SUP'COM

المدرسة العليا للمواصلات بتونس

ÉCOLE SUPÉRIEURE DES COMMUNICATIONS DE TUNIS

Telecommunications Engineering Degree

Major:

Communicating Systems and Connected Objects (SCOC)

End of Studies Project Report

Topic:

Implementation and Validation of FPGA core IP Integrated into a MultiProcessor based System on Chip (MPSoC)

Realized by:

Amine Dagdoug

Supervised by:

Mr Zied Marrakchi, Mentor Graphics Tunisia

Mr Marouene Haouati, Mentor Graphics Tunisia

Mrs Manel Ben Romdhane, SUP'COM

Work proposed and realized within:

Mentor®
A Siemens Business

Academic Year: 2020-2021

Ecole Supérieure des Communications de Tunis
SUP'COM
2083 Cité Technologique des Communications - Elghazala - Ariana - Tunisie
Tél. +216 70 240 900 – site web : www.supcom.mincom.tn

****

Mentor Graphics's supervisor: Dr. Zied Marrakchi

Signature



****

Academic Supervisor: Dr. Manel Ben Romdhane

signature



Acknowledgements

I would like to express my sincere thanks to all those who have contributed, either closely or by far, to the successful completion of my Final internship and to the development of this modest work.

I would like to begin by addressing the members of the honorable jury, whom I thank for agreeing to examine this report in the hope that they will find the quality of clarity and reasoning that they expect.

I want to thank the manager of Mentor Graphics Dr. Zied Marrakchi for giving us the opportunity to join Mentor Graphics, developing a solid link between our studies and our working life.

I want to thank Mr. Marouene Haouati, my technical supervisor, for being always available to answer all my questions, even the futile ones, and for explaining and guiding me throughout this internship.

My deep gratitude goes to Dr. Manel Ben Romdhane, who mentored me during this internship, for the valuable advice and assistance she provided throughout the whole working period.

To all the professors and teachers who have contributed to our training and education from our first cycle to the end of our university studies.

Résumé

Dans ce projet, nous exposons la topologie *Mesh-of-Tree* d'un FPGA et nous développons un générateur de la description matérielle de son réseau d'interconnexion. Etant donné que nous voulons exploiter la scalabilité de la topologie *Mesh-of-Tree*, nous considérons la *Mesh-of-Tree* comme des blocs répétés appelés Tiles qui peuvent se jouxter pour générer le layout d'un FPGA. Après avoir validé notre générateur, nous explorons la topologie en termes d'efficacité de surface et la meilleure taille de mot utilisé pour configurer le design du FPGA utilisé. Finalement, nous intégrons notre FPGA IP dans un environnement MPSoC comme un accélérateur matériel en implémentant son interface d'interconnexion avec l'environnement MPSoC.

Mots clés: FPGA, HDL, RTL, Hypergraphe, Structure de données, CPU, Processeur

Abstract

In this project, we expose the Mesh-of-Tree FPGA topology and develop a generator of the hardware description of its interconnect network. Since we want to exploit the scalability of the Mesh-of-Tree topology, we consider the Mesh as repeated blocks named Tiles that can be abutted to generate the whole FPGA layout. After validating our generator, we explore the topology in terms of area efficiency, as well as, the best word width used to configure the FPGA design. Finally, we integrate the FPGA IP into an MPSoC platform as a configurable accelerator and implement its interface of interconnection.

Key words: FPGA, HDL, RTL, Hypergraph, Data structure, CPU, Processor

Acronyms

ASIC	:	Application Specific Integrated Circuit
AXI	:	Advanced eXtensible Interface
BLE	:	Basic Logic Element
CB	:	Connection Box
CLB	:	Configurable Logic Block
CPU	:	Central Processor Unit
DMSB	:	Downward Mini Switch Box
FPGA	:	Field Programmable Gate Array
HDL	:	Hardware Description Language
HNL	:	Hierarchical Netlist
IP	:	Intellectual Property
LUT	:	Lookup Table
MPSoC	:	MultiProcessor System On chip
MSB	:	Mini Switch Box
Mux	:	Multiplexer
RTL	:	Register Transfer Level
SB	:	Switch Box
SoC	:	System on Chip
SRAM	:	Static Random Access Memory
TTM	:	Time To Market
UMSB	:	Upward Mini Switch Box

Table of Contents

Acknowledgements iii

Résumé iv

Abstract iv

Acronyms v

List of figures viii

List of tables x

General introduction 1

1 MESH OF TREE AND MPSOC ARCHITECTURE FUNDAMENTALS 3

1.1 Introduction 3

1.2 Basic concepts 3

1.2.1 FPGA architecture 3

1.2.1.1 FPGA's mesh-based architecture 3

1.2.1.1.1 Configurable Logic Blocks 4

1.2.1.1.2 Switch Box 6

1.2.1.1.3 Connection Box 6

1.2.1.2 FPGA's Tree-Based Architecture 7

1.2.1.2.1 Downward Network 7

1.2.1.2.2 Upward Network 9

1.2.1.2.3 Connection with the outside 11

1.2.1.3 Mesh vs Tree Architectures 11

1.2.2 Unified FPGA's Mesh and Tree Architecture 14

1.2.2.1 Introducing Mesh-of-Tree architecture 14

1.2.2.2 Tree Clusters Local Interconnect 15

1.2.2.3 Mesh routing link-up 16

1.2.2.4 Input/Output Pads Connection 17

1.2.2.5 FPGA CAD flow 18

1.2.2.6 Design Verification 19

1.2.3 MPSoC overview 21

1.2.3.1 SoC 21

1.2.3.2 Microprocessor 22

1.2.3.3 OpenPiton architecture 22

1.3 Objectives and Contribution 24

1.4 Conclusion 25

2 MESH-OF-TREE ARCHITECTURE DESIGN 26

2.1 Introduction 26

2.2 Design of the existing generator and interconnection network 26

2.2.1 Mesh-of-Tree Generator Overview 26

2.2.2 Design of the Mesh-of-tree interconnect 30
2.2.2.1 FPGA Basic Building Blocks 30
2.2.2.2 Conceptual Overview 34
2.2.2.3 Behavioral Overview 36

2.3 Loader Design 37
2.3.1 Architecture Configuration 37
2.3.2 Address Decoder 38
2.3.2.1 Decoder at the Mesh Level 39
2.3.2.2 Decoder at the Tree Level 40
2.3.3 Multiplexers Configuration 42
2.3.4 Address width 42
2.3.5 Loader conceptual overview 45

2.4 Enhancement of the existing solution 45
2.4.1 N:1 Multiplexer design 45
2.4.2 TestBench Generation 49

2.5 Integrating the Mesh-of-Tree into MPSoC 49
2.5.1 Connecting the eFPGA to an AXI interface 49
2.5.2 Connecting the eFPGA to OpenPiton 50

2.6 Conclusion 51

3 VALIDATION OF THE FPGA IP GENERATOR AND INTEGRATING IT INTO AN MPSOC ARCHITECTURE 52

3.1 Introduction 52

3.2 Validation of the FPGA Hardware Architecture 52
3.2.1 Generation of the hardware design of the FPGA 52
3.2.2 Design Simulation and Debugging 55

3.3 FPGA Integration into MPSOC 60
3.3.1 AXI slave interface generation 60
3.3.2 Simulation of the eFPGA within the AXI Slave Interface 61
3.3.2.1 Simulation of the write operation 61
3.3.2.2 Simulation of the read operation 63
3.3.3 Connection of the eFPGA to openpiton 64
3.3.3.1 Add the eFPGA as a connected device to I/O Crosssbar 64
3.3.3.2 Add an AXI interface to the Top system 67
3.3.3.3 Connect OpenPiton to the Mesh-of-Tree 70
3.3.3.4 Validation of the connection 71

3.4 Conclusion 74

Conclusion and perspectives 75

References 76

List of figures

Figure 1.1: Mesh based FPGA Architecture. 4
Figure 1.2: Illustration of BLEs in a CLB. 4
Figure 1.3: Basic Logic Block 5
Figure 1.4: FPGA 4 Inputs Look-Up Table Composition. 5
Figure 1.5: Switch Box. 6
Figure 1.6: Connection Box. 6
Figure 1.7: Simplified overview of Tree-based FPGA Topology. 7
Figure 1.8: Tree-based FPGA Topology. 8
Figure 1.9: Downward Network 9
Figure 1.10: DMSB Internal Composition 9
Figure 1.11: Upward Network 10
Figure 1.12: UMSB Internal Composition 11
Figure 1.13: Input/Output Pads Connection 11
Figure 1.14: Number of series switched in Tree-based and mesh-based interconnect. 12
Figure 1.15: Mesh and Tree cases: a layout aspect. 13
Figure 1.16: Mesh of Tree architecture presentation: a grid of 3*3 Mesh Clusters. 14
Figure 1.17: Mesh of Tree Cluster topology. 15
Figure 1.18: Mesh of Tree Switch Box architecture: inter-clusters interconnect. 17
Figure 1.19: External Input and Output pads connection: Boundary Switch Box case. 18
Figure 1.20: Generalized software flow. 18
Figure 1.21: Comparison of design verification techniques. 20
Figure 1.22: MPSoC architecture. 21
Figure 1.23: An overview of a basic microprocessor architecture. 22
Figure 1.24: OpenPiton architecture 23
Figure 1.25: OpenPiton prototype architecture. 23
Figure 1.26: Project general steps 24
Figure 2.1 :Generator black box overview. 26
Figure 2.2: Architecture Description File. 27
Figure 2.3: Architecture Description File. 28
Figure 2.4: Simplification of the Tree in 2.3. 28
Figure 2.5: Mesh-of-Tree architecture parameters. 29
Figure 2.6: Mesh-of-Tree generator general overview. 29
Figure 2.7: FPGA Tile presentation. 30
Figure 2.8: Tiles for Mesh-based FPGA. 31
Figure 2.9: Distinct use of tile's modules. 32
Figure 2.10: Use of tile's modules twice. 32
Figure 2.11: Different tile's modules. 33
Figure 2.12: Class diagram of the 9 types of tiles. 34
Figure 2.13: General class diagram of the Mesh-of-Tree. 35
Figure 2.14: Inheritance 35
Figure 2.15: Activity diagram of the FPGA IP generator. 36
Figure 2.16: LB implementing and function in LUT. 37
Figure 2.17: General overview of the Mesh of Tree Loader. 38
Figure 2.18: A detailed overview of the Mesh of Tree Loader containing a Tile with its corresponding loader. 39
Figure 2.19: A Switch Box Loader 40
Figure 2.20: Loaders of clusters at level 0 and clusters at level greater than 0. 40
Figure 2.21: Tree of 2 hierarchical levels with their corresponding levels. 41
Figure 2.22: Configuration in both LUT and SBs. 42
Figure 2.23: Elements to identify to get into a LB Configuration. 43

Figure 2.24: Elements Design of Loaders used to configure the Mesh-of-Tree.....45
Figure 2.25: 2-input Mux. 46
Figure 2.26: 3-input Mux. 47
Figure 2.27: 4-input Mux. 47
Figure 2.28: Symbol of 8:1 Mux. 48
Figure 2.29: TestBench. 49
Figure 2.30: AXI Slave interconnection. 50
Figure 2.31: Address mapping description. 50
Figure 2.32: OpenPiton prototype architecture with eFPGA. 51
Figure 3.1: Example of an architecture file. 52
Figure 3.2: Build and hardware design generation of an FPGA architecture. 53
Figure 3.3: Build and hardware design generation of an FPGA architecture 4*4. 54
Figure 3.4: Build and hardware design generation of an FPGA architecture 4x4. 54
Figure 3.5: Generation of 25 tiles of a 4*4 FPGA architecture. 55
Figure 3.6: Debugging flow. 56
Figure 3.7: Bitstream description file example. 57
Figure 3.8: Example of the content of the SRAMs of a ble. 57
Figure 3.9: Example of a net in the router.log file. 58
Figure 3.10: Example of a net in the router.log file. 59
Figure 3.11: TestBench. 59
Figure 3.12: Shell script. 60
Figure 3.13: Result of comparison. 60
Figure 3.14: Write and read operation mapping. 61
Figure 3.15: Simulating the write operation. 62
Figure 3.16: Mesh-of-Tree instance configuration. 62
Figure 3.17: Mesh-of-Tree inpads assignment. 62
Figure 3.18: Simulating the read operation. 63
Figure 3.19: Read Data from outpads. 63
Figure 3.20: Changes in the XML file. 64
Figure 3.21: Generation of crossbar ports and associated wires. 65
Figure 3.22: Generation of valrdy to credit converter and credit to valrdy converter. 66
Figure 3.23: Add interfaces to chipset_impl model. 67
Figure 3.24: NOC/AXI bridge instantiation. 68
Figure 3.25: Add AXI interface to chipset. 69
Figure 3.26: Add AXI interface to chipset. 69
Figure 3.27: Add AXI interface to system and chipset instantiation. 70
Figure 3.28: Top system and AXI slave interconnection. 71
Figure 3.29: Main, configuration and loading bitstream functions. 72
Figure 3.30: Inpads configuration functions. 72
Figure 3.31: Outpads read functions. 73
Figure 3.32: Data transfer result. 73

List of tables

Table 1.1: Comparison between Mesh and Tree topologies.....12
Table 2.1: Truth table of 2:1 Mux. 46
Table 2.2: Truth table of 3:1 Mux. 46
Table 2.3 : Truth table of 4:1 Mux. 47
Table 2.4: Truth table of 8:1 Mux. 48

General introduction

In the past few years, devices have swept the globe. In fact, we are facing an exponential increase in their capabilities and use by people all over the world. Most of these devices perform a specific application, integrated on a chip (SoC, System on Chip). In fact, SoCs are mainly used in applications that require high performance and high adaptability with low utilization. This growing market and the demand for higher performance drove the industry to build a more powerful chip. That's why the industry is moving to a many-core architecture in both mobile and enterprise platforms. This type of systems, however, still have difficulties of flexibility and reduced time-to-market (TMM).

Thus, in order to meet the requirements and retain its market competitiveness, SoC designers moved to Field Programmable Gate Array (FPGAs), a trade-off between designing constraints, which meets the requirements owing to its reconfigurability. This new era of FPGAs has begun, since it allows us to verify designs by achieving rapid prototyping (up to 30MHz at the moment) of high complexity digital circuits. Nevertheless, thanks to FPGAs, we are able to easily test and validate new applications, topologies' concepts and the possibility to introduce optimization, evolution but also propose new functionalities. This interesting property of the FPGA pushed us to move directly to explore its different topologies which are mainly the Tree, Mesh and Mesh of Tree. The Mesh-of-Tree architecture, is proven to be promising since it ensures a good layout scalability and a better interconnect efficiency. In this context, this project is proposed, to realize an FPGA Mesh-of-Tree-based Intellectual Property (IP): implement, validate, explore the developed IP and integrate it into a Multiprocessor System on Chip (MPSoC) platform as a configurable accelerator. In fact, the idea behind using FPGAs as instantiated IPs, is to reuse them in more complex systems as embedded FPGAs (eFPGAs).

My end of study project is elaborated in Mentor Graphics which is a US-based multinational corporation founded in 1981, headquartered in Wilsonville, Oregon, and has numerous sites around the world, among them Tunis R & D center. It employs roughly 5,200 people worldwide. Since March 2017, Mentor Graphics is a subsidiary of Germany industrial giant Siemens [1].

The company is specialized in electronic design automation (EDA) for electrical engineering and electronics, providing software and hardware design solutions that enable customers to develop better electronic products faster and more cost-effectively. Most of Mentor Graphics clients are integrated circuits (IC) manufacturers that use its software to simulate, emulate, or prototype their designs. We can cite, for example, Intel, ARM, Samsung, Apple, Qualcomm,

etc [1].

Mentor Tunisian site was created in 2017 and employs today, 18 engineers. This site performs research and development activities around the Mentor Graphics multi FPGA prototyping tool named Veloce Prototyping System (VPS). The objective is to enhance the tool's performance and keep it up to date with the new technologies and the increasing complexity of board and chip designs [1].

This project report is composed of 3 major chapters. The first chapter will start by introducing the company. Then, we will outline the basic concepts for all domain related to our project as well as the requirements and objectives. In the second chapter, we will present the design of the existing solution and the proposed enhancement. Then, we simulate, validate our work and present the integration of the eFPGA into a many-core architecture as a configurable accelerator. Finally, we will sum up and present the perspectives of the project.

CHAPTER 1

Mesh of Tree and MPSoC architecture fundamentals

1.1 Introduction

We will present in this first chapter the FPGA and its major components. We will also present its importance in design verification and the MPSoC architecture within we will integrate our IP FPGA as an embedded FPGA (eFPGA). Finally, the project's goals and contributions will be highlighted.

1.2 Basic concepts

In this section, we will define some technical terms that I have used in this project. In the first sub-section, we will present the different FPGA architecture. In the second sub-section, we will present the unified FPGA's mesh and tree Architecture and its appropriate FPGA software flow. In the last part, we will briefly describe the MPSoC architecture and the OpenPiton manycore processor.

1.2.1 FPGA architecture

In this subsection, we will introduce the mesh FPGA architecture as well as the tree FPGA architecture. Then, we will present and demonstrate the significance of combining these two architectures into a single architecture known as mesh of tree.

1.2.1.1 FPGA's mesh-based architecture

Mesh-based architecture is the most common among academic and commercial FPGAs. They are also known by the name island-style FPGA. It is organized into a bidirectional grid of configurable logic blocks (CLBs) surrounding by vertical and horizontal routing network as presented in Figure 1.1 [2]. Each network has a fixed number of routing channels, which is known as the channel width of the architecture. The routing network consists of two types of blocks: the switch boxes (SBs) and the connection boxes (CBs). At the grid periphery, I/O cells or also called I/O blocks, are placed to connect the FPGA to its external environment. CLB, SB and CB are presented in the next subsubsection.

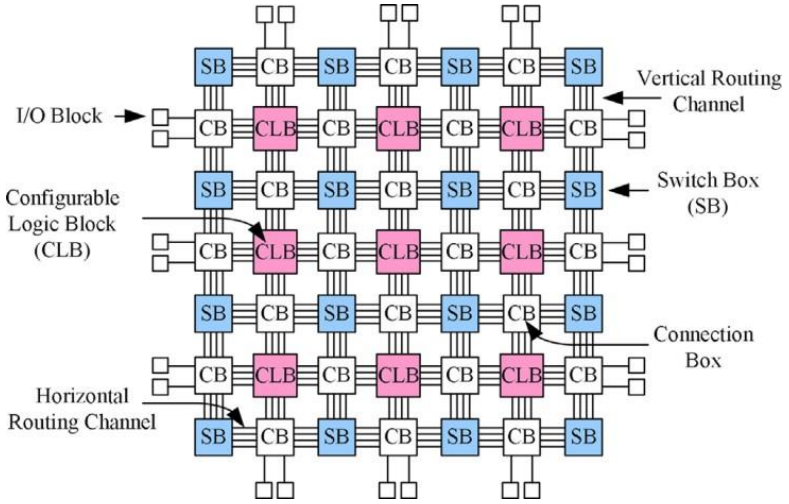


Figure 1.1: Mesh based FPGA Architecture.

1.2.1.1.1 Configurable Logic Blocks

CLBs are the basic logic part and the configurable resource of an FPGA. They can implement a wide variety of logic functions, which guarantees the flexibility of the FPGA. They contain boolean logic, which is the equivalent of gates in an application-specific integrated circuit (ASIC). CLB can contain one or more basic logic elements. Figure 1.2 illustrates a group of BLEs interconnections in a CLB.

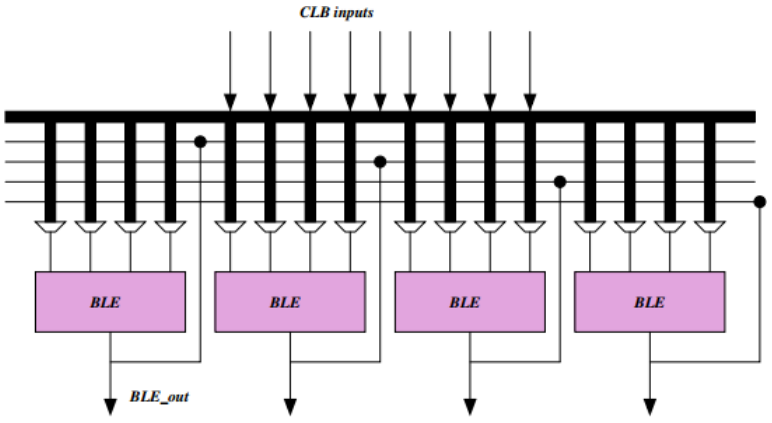


Figure 1.2: Illustration of BLEs in a CLB.

As described in Figure 1.3 [3], a BLE contains 3 components.

- One look-up table (LUT) is programmable with k inputs to hold 2^k configuration bits and one output. It is used to implement a combinatorial circuits in the FPGA. The 2^K configuration bits were hold in static random access memory (SRAM) is a type of

random access memory that uses flip-flops to store data.

- One D-type Flip-Flop (D-FF) is used to implement sequential circuits in the FPGA and use clock as reference signal.
- One 2-to-1 multiplexer (MUX) is used to choose between the LUT output and the latched output.

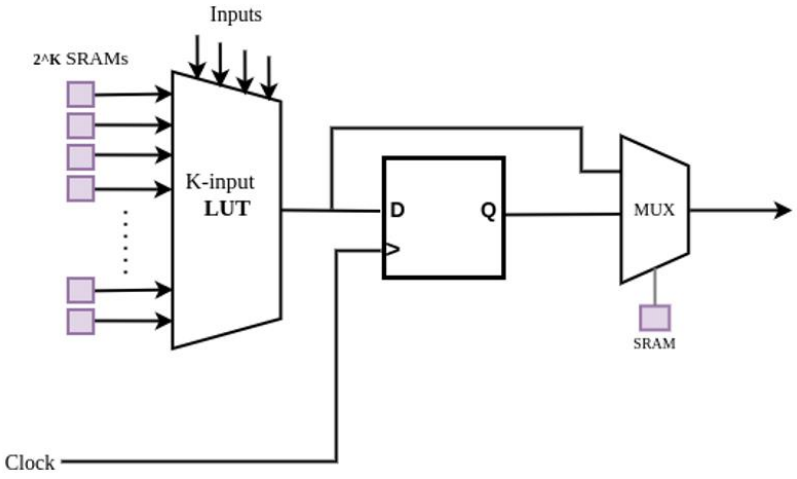


Figure 1.3: Basic Logic Block

Figure 1.4 depicts an example of look-up table with four inputs [2].

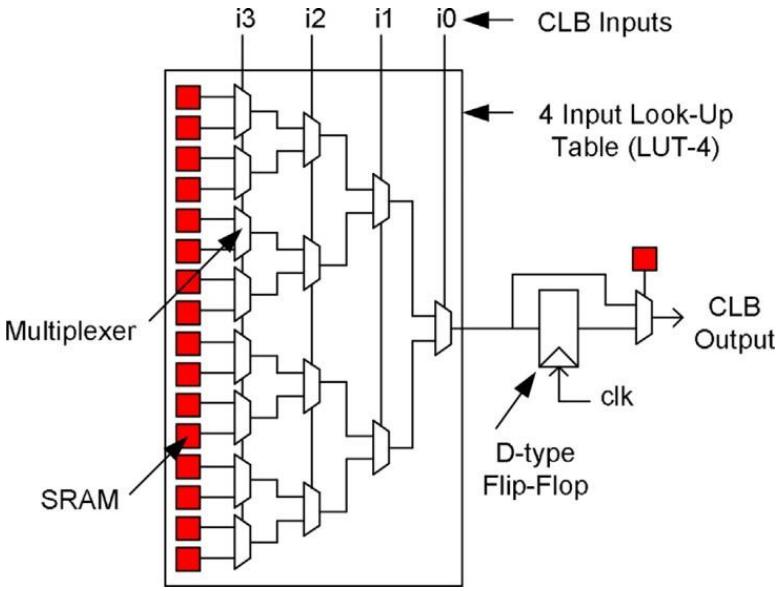


Figure 1.4: FPGA 4 Inputs Look-Up Table Composition.

1.2.1.1.2 Switch Box

The Switch Box, shown in Figure 1.5, is positioned at each intersection between the horizontal and vertical routing channels to interconnect and identify all possible links between them.

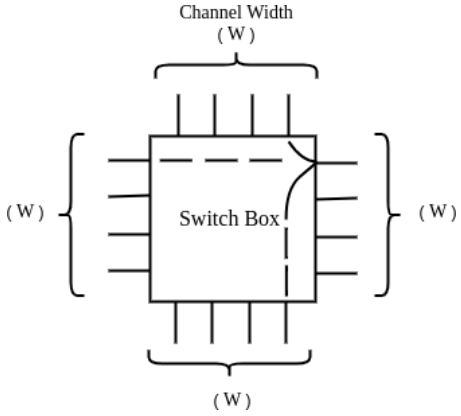


Figure 1.5: Switch Box.

1.2.1.1.3 Connection Box

As previously introduced, a CB, as given in Figure 1.6 [4] is a group of switches used to connect a CLB to adjoining route in the routing network. Furthermore, it is used to connect the inputs and outputs of CLBs to the routing paths.

Referring to [5], the routing interconnect dominates almost 80% of the surface area while the logic blocks take the rest of the FPGA area. Moreover, this type of architecture constrains that a signal must go across a considerable number of switches, to get routed in the routing network which probably leads to exceed the logic blocks delay.

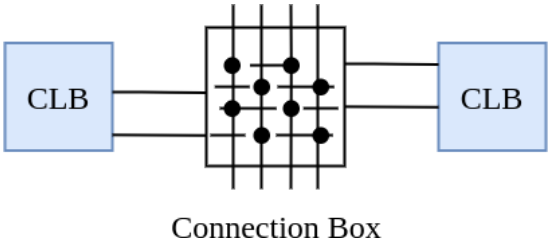


Figure 1.6: Connection Box.

Thus, decreasing the crossed switches number may boost the circuit speed and improve the FPGA area use. That’s why, another architecture named as the tree-based FPGA architecture is suggested.

1.2.1.2 FPGA's Tree-Based Architecture

As already mentioned, the Tree-based FPGA architecture comes to overcome the overdue routing delays' problem between logic blocks. Also named as the Hierarchical FPGA topology, this architecture interconnection operates as the Butterfly Fat-Tree (BFT) topology shown in Figure 1.7 where S is a switch box and W0 is the channel width.

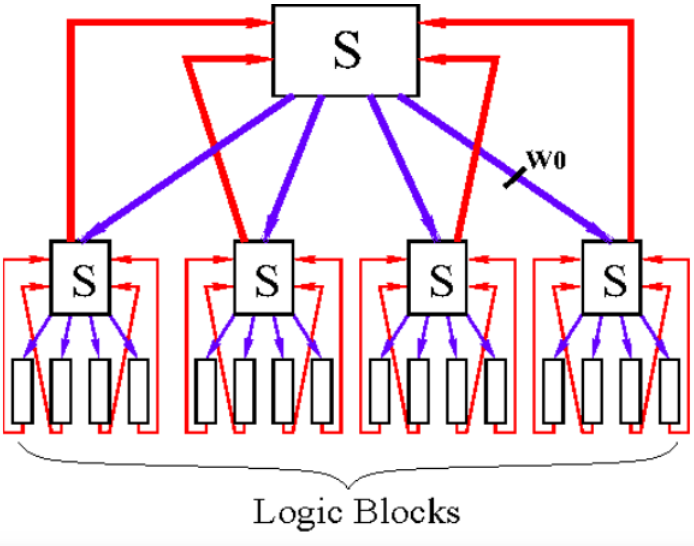


Figure 1.7: Simplified overview of Tree-based FPGA Topology.

In fact, in this latter, logic blocks are assembled in clusters, which are in their turn assembled recursively to form clusters of higher level in the tree structure (one cluster and its slaves clusters gathered inside it). The figure 1.8 illustrates the topology where a cluster of level "L+1" has "W" incoming and outgoing routing wires of level "L" and contains slaves clusters of lower level "L" associated with a switch box of level "L". SBs in the Tree topology are mainly divided into Upward and Downward Mini Switch Boxes respectively (MSBs) fully connected crossbar. As the architecture contains two unidirectional networks Downward and Upward, it uses respectively Downward MSBs (DMSBs) and Upward MSBs (UMSBs) to connect logic blocks to various levels of the tree.

1.2.1.2.1 Downward Network

The BFT topology is based on the downward topology. As illustrated in Figure 1.9 each DMSB in the (L, c) cluster is linked to each subcluster (slave of cluster (L, c)) by an input pin where L is the tree level and c is the cluster index.

The DMSB, a one-way crossbar switch box, as mentioned in subsubsection 1.2.1.2, is the foundation of the downward network. It allows its inputs to reach any of its outputs. It has also an important role to fulfill downward interconnection. It guarantees the interconnection between

a cluster of level L and all its slaves of level L − 1. So, the number of DMSBs at a specific level depends on the cluster inputs at the next lower level [4].

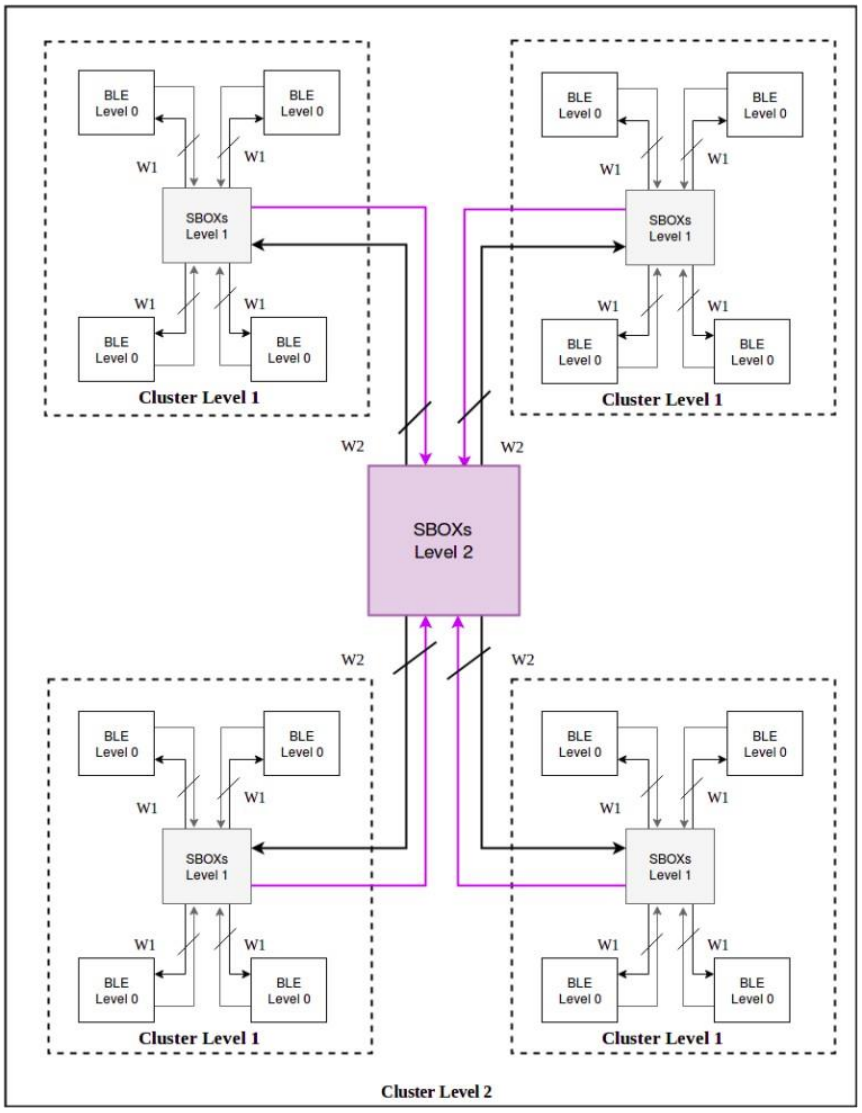


Figure 1.8: Tree-based FPGA Topology.

It should be noted that the number of a cluster DMSBs at a level L, nbDMSBs(L), is equal to number of inputs of a cluster at level L-1 (nbInputs(L − 1)). But in the case where $L = 0$, the number of DMSBs of a cluster at level 0 is equal to the number of inputs of LB.

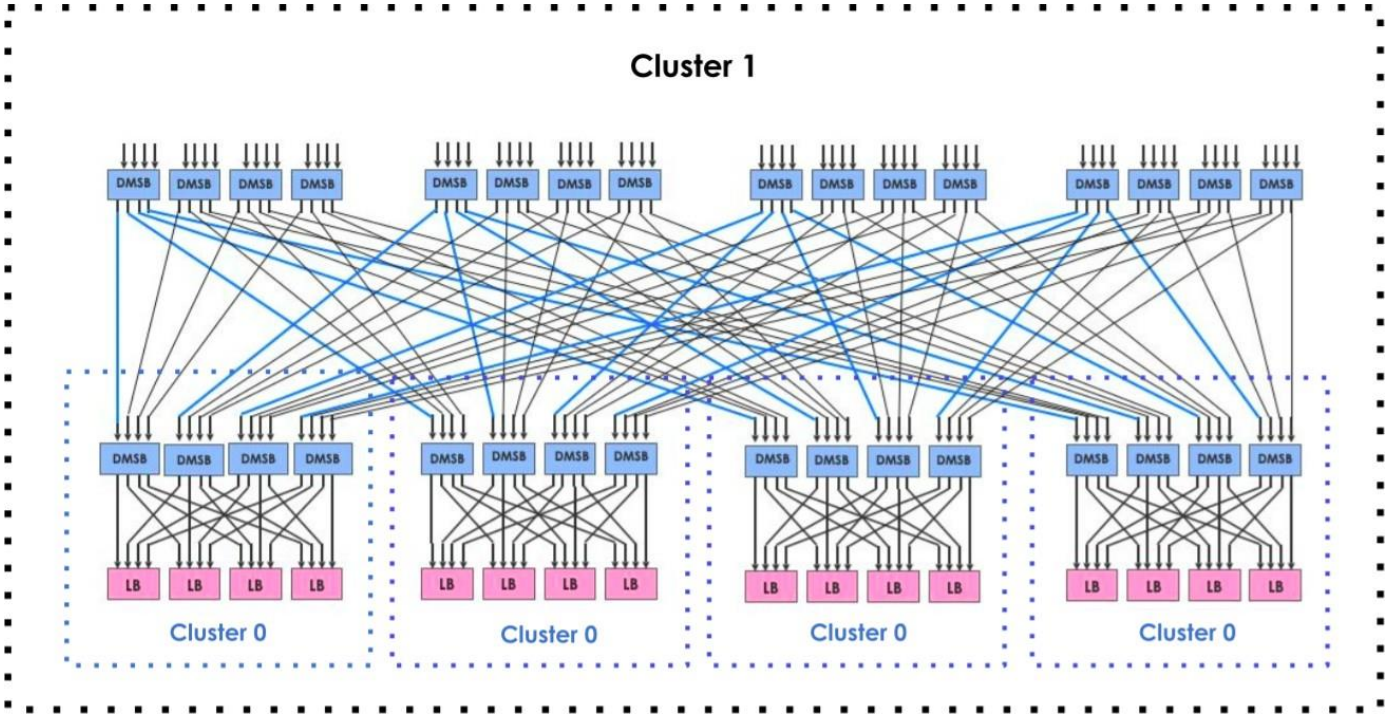


Figure 1.9: Downward Network

Figure 1.10 presents the internal composition of a DMSB with 4 inputs and 4 outputs. If a DMSB has N inputs and O outputs, then it is composed of O number of N multiplexers and log2 (N) × O SRAMs.

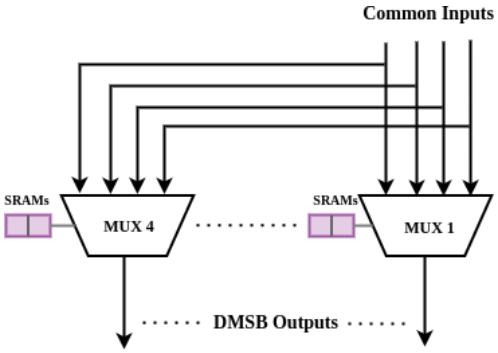


Figure 1.10: DMSB Internal Composition

1.2.1.2.2 Upward Network

UMSB, a one-way crossbar switch box, is the foundation of upward network. UMSBs have also an important role to fulfill downward interconnection. They ensure the interconnection between a cluster of level l and all clusters of level l + 1. Thus, they are arranged in a way that every LB belonging to the same owner cluster can reach exactly the same set of DMSBs at each

level as presented in Figure 1.11.

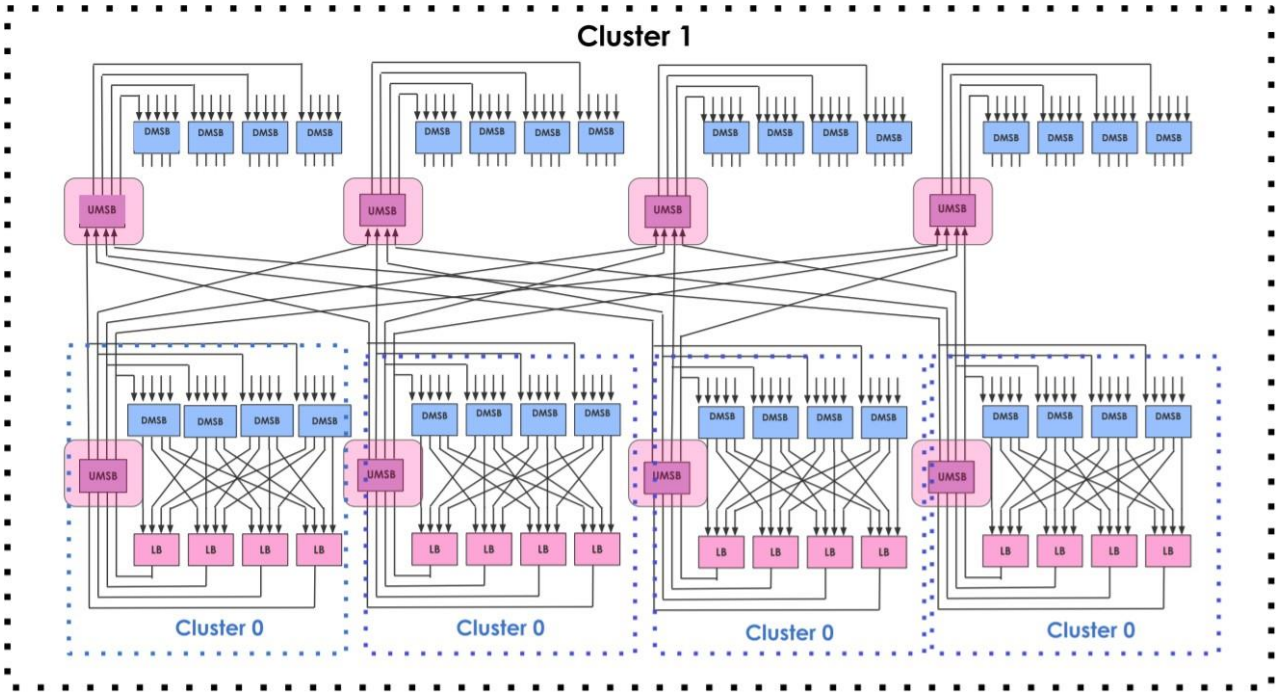


Figure 1.11: Upward Network

As shown in Figure 1.11, The UMSB allows LBs outputs to be achieved by DMSBs. Indeed, feedbacks are the wires driven by the UMSBs because they lead information from a lower level cluster to a higher level cluster.

Using the equations (1.1) and (1.2) [6], the number of feedbacks and UMSBs in the upward interconnect are defined.

$$nb_{UMSBs}(L) = nb_{Outputs}(L-1) \tag{1.1}$$

$$nb_{Feedbacks}(L) = nb_{Outputs}(L) \times nb_{UMSBs}(L) \tag{1.2}$$

With:

$nb_{UMSBs}(L)$ Number of cluster UMSBs at level = L.

$nb_{Outputs}(L)$ Number of outputs of a cluster at level = L.

Excepting the case when L = 0, $nb_{UMSBs}(0) = nb_{LBOutputs}$.

Figure 1.12 depicts the internal structure of a DMSB with four inputs and four outputs. If a DMSB has N inputs and O outputs, it is made up of O N multiplexers and log2 (N) × O SRAMs.

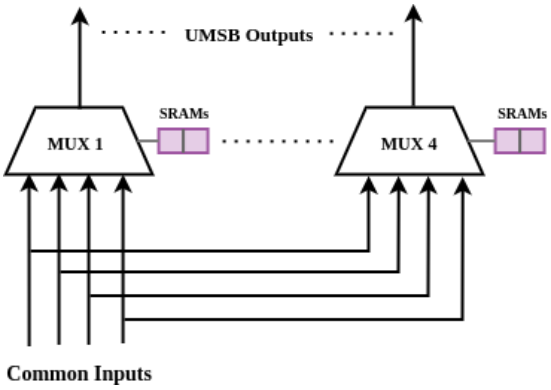


Figure 1.12: UMSB Internal Composition

1.2.1.2.3 Connection with the outside

Input and Output pads ensure the external connection. In fact, as figure 1.13 illustrated, input pads are directly connected to UMSBs and output pads are linked to DMSBs through a DMSBOUT cluster in the same specific level called I/O level. The DMSBOUT cluster drives signals from all DMSBs in the I/O level to output pads. So, External input and output level can achieve all logic blocks in tree-based architecture.

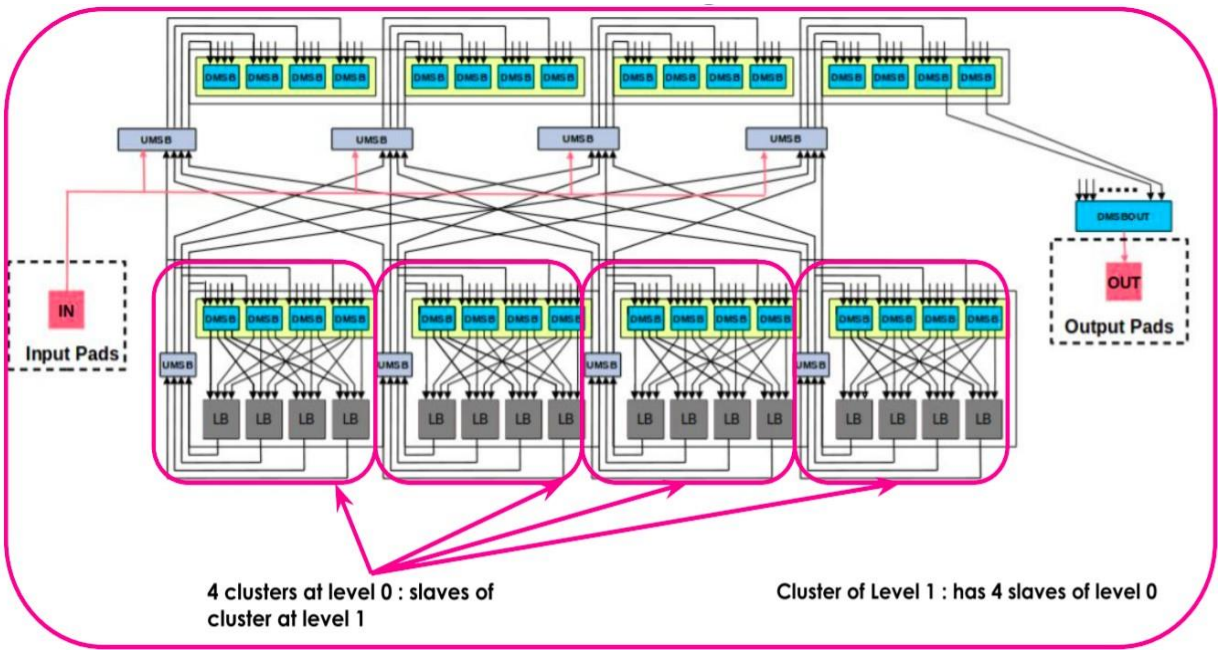


Figure 1.13: Input/Output Pads Connection

1.2.1.3 Mesh vs Tree Architectures

In order to compare the Mesh and Tree topologies, we will base our comparative table 1.1 on the interconnect and logic areas [5], the ease of physical layout generation and scalability [3]

and the Manhattan distance. In fact, he mesh topology has a greater routing delay since the number of crossed Switch Boxes (SBs) connecting two Logic Blocks keeps increasing linearly the Manhattan 1 distance "d". In the case of the Tree case, the number is a logarithmic function of the distance "d" like illustrates figure1.14. However, the Manhattan distance metric of two nodes $n(x_n, y_n)$ and $m(x_m, y_m)$ is defined in the equation (1.3) [7]:

$$d(n,m) = |x_n - x_m| + |y_n - y_m| \tag{1.3}$$

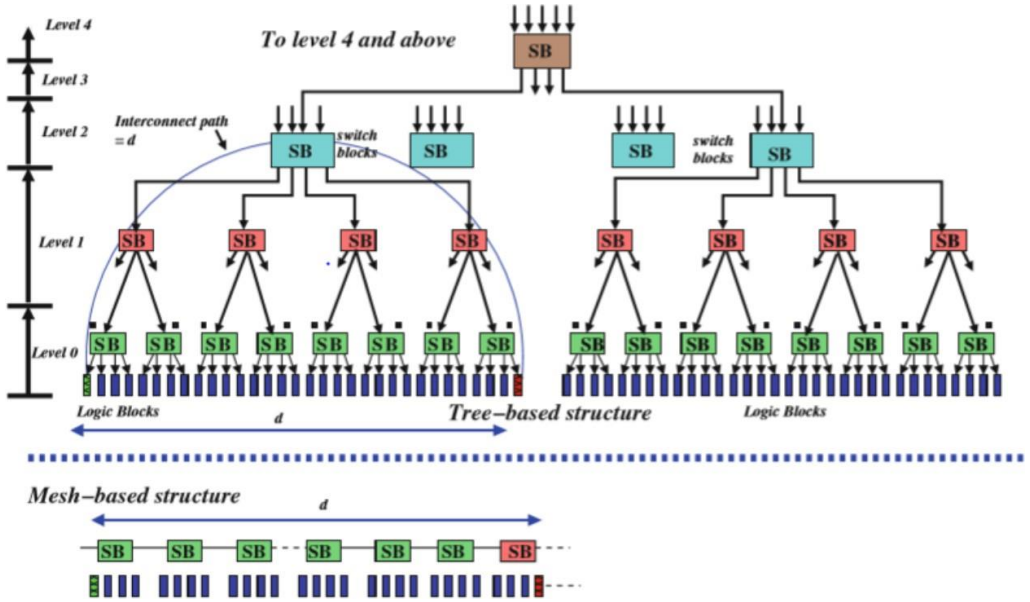


Figure 1.14: Number of series switched in Tree-based and mesh-based interconnect.

The comparison is summarized in this table 1.1:

Performance\Topology	Mesh	Tree
Interconnect area	90%	73%
Logic area	10%	27%
Physical Layout Generation	Easy	Not Easy
Scalability	Scalable	Not Scalable
Manhattan Distance	Linear	Logarithmic

Table 1.1: Comparison between Mesh and Tree topologies.

Providing maximum flexibility while conserving a minimal area cost is the big challenge for FPGA. In fact, Configurable logic blocks in the Mesh-based FPGA, which are the intrinsic building blocks since they contribute and provide reconfigurable logic gates, are arranged in two-dimensional mesh with routing resources distributed throughout the mesh. In fact, a signal must cross a considerable number of switches in the routing path which leads to a large routing

path propagation delay. This latter, can exceed logic blocks delay which ultimately limits the performance of the FPGA.

In the Mesh topology the overall interconnect area takes 90% of the surface while in the Tree-based topology it occupies 73%. And the remaining surface in the two topologies is occupied by the logic blocks. It exposes that the Mesh surface area is dominated by the interconnection network where routing resources occupy 90% of the surface area while logic blocks occupy only 10%. When compared to the Mesh, the tree topology has an interconnection area efficiency of nearly 20%. In fact, in the tree topology the logic blocks are grouped in clusters which are in their turn gathered recursively to form a parent clusters of higher level in the tree structure.

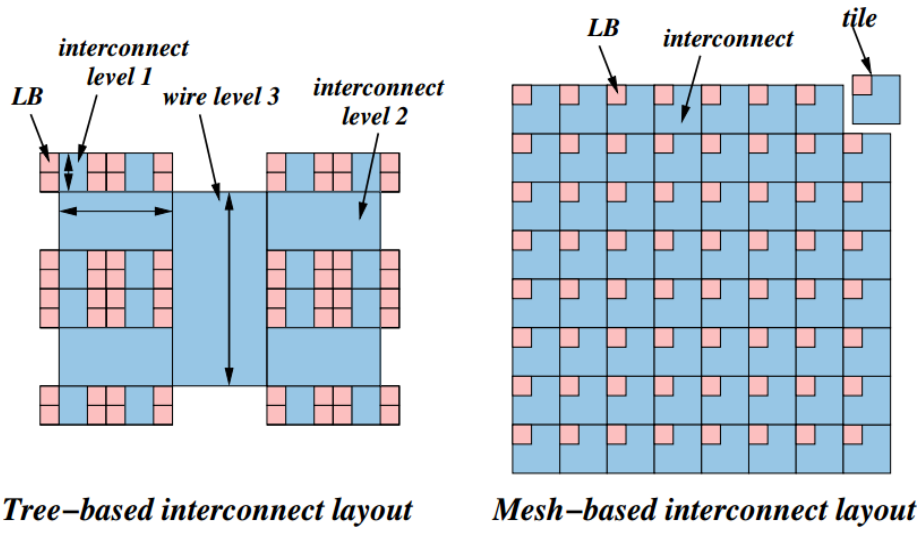


Figure 1.15: Mesh and Tree cases: a layout aspect.

Nevertheless, we cannot deny the fact that the Tree topology shows a better area efficiency compared to the Mesh, as shown in Figure1.15, but at the same time it is penalizing in terms of physical layout generation. On the other hand, the mesh-based topology has a good physical scalability since it is structured as an array of repeated tiles. Thus, the Mesh of Tree topology [8]was invented to take advantage of the positive aspects of both mesh and tree topologies which gives logic blocks connected into a Tree-based clusters, these clusters are connected in the Mesh topology with an external interconnect.

Even though the announced advantage seems promising, no FPGA designers, until now, has implemented this new Mesh of Tree FPGA architecture which is required for the topology assessment.

1.2.2 Unified FPGA's Mesh and Tree Architecture

The unified architecture Mesh of Tree, is proposed to exploit advantages of both Tree and Mesh architectures.

1.2.2.1 Introducing Mesh-of-Tree architecture

As presented in Figure 1.16, it is composed of a mesh of nodes [9], each node in the mesh is structured as tree. The use of the Tree topology is proven to provide a better performance [9], because it ensures a faster signal sharing between clustered logic blocks.

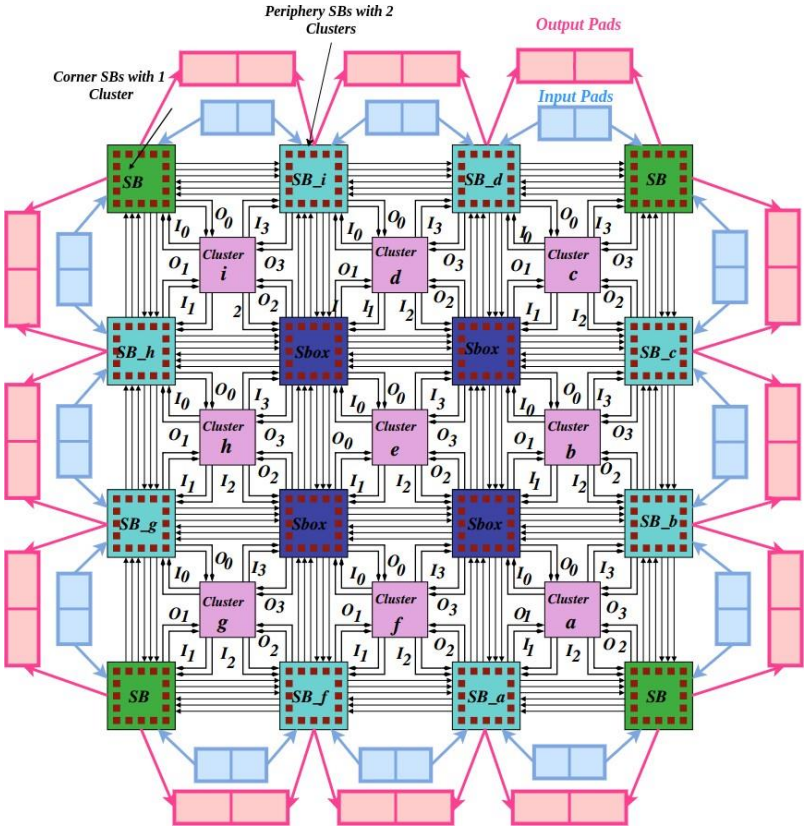


Figure 1.16: Mesh of Tree architecture presentation: a grid of 3*3 Mesh Clusters.

Known as Three-Dimensional FPGA [3] [10], this high density architecture reports an optimized intra-cluster interconnect due to Tree and inter-clusters interconnection due to Mesh topology. The resulting unification is a mesh of clusters, where each cluster encloses the intra-cluster interconnect as well as logic blocks.

The topology presents two different scales of clusters, clusters in the mesh level surrounded by the Mesh connection named Mesh Clusters. The other clusters 'scale is the levels of the Tree, named as Tree Clusters.

1.2.2.2 Tree Clusters Local Interconnect

Tree clusters consists of logic blocs (LBs) that communicate in a programmable interconnect. The intra-cluster interconnect is organized as a tree structure and has a topology similar to a Tree-based FPGA.

This distribution has a great impact on routability and eliminates the restriction of placing Logic blocs in the cluster. (c.f Figure1.16 and Figure1.17).

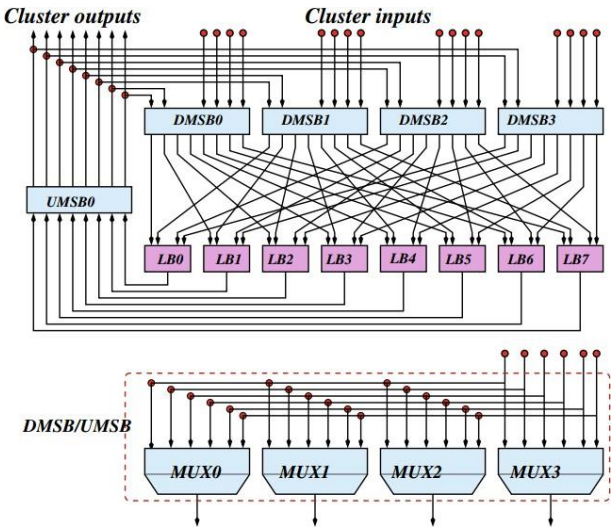


Figure 1.17: Mesh of Tree Cluster topology.

Each top cluster, as pictured Figure1.16, (in the hierarchical level of the tree) is connected to its 4 adjacent channel wires (also called tracks). Cluster inputs and outputs are equally divided by the 4 sides of cluster. Thus, we always have the same number of inputs and outputs in all cluster ‘sides. These input and output signals are connected to clusters at a certain level "L" of the tree.

- The 4 input adjacent channels from the neighboring Switch Boxes in the mesh, are connected as inputs to the all UMSBs clusters at level "L+1". By connecting them this way, the 4 inputs can reach all Logic Blocks located in the whole Tree.
- The 4 Cluster output signals are connected to the 4 output adjacent Switch Boxes. All output wires are connected to all the Tree DMSBs located at the level "L+1". Thus, all Tree’ LBs can output on all the 4 SBs sides.

This way of network interconnection affects in a positive manner the routability by excluding

constraints when placing LBs in Clusters [3].

1.2.2.3 Mesh routing link-up

Wires in the Mesh unit are single-driver unidirectional wires. It is proven in [11] that single-driver link up contribute to 25% enhancement in area density. Mesh clusters are encircled by 4 tracks connected by Switch Boxes. Unlike the Mesh-based topology, the Mesh of Tree do not use Connection Boxes (CBs) to connect channels to clusters input and output wires. Indeed, this type of interconnect is optimized by using the cluster local interconnect as CB [12].

The flagship component of the Mesh of Tree based interconnect is Switch Box (SB). In fact, SBs ensure signals driven in the FPGA. Moreover, SBs connect horizontal and vertical channels each another as well as to clusters. As demon- started in Figure1.16, each cluster inputs and outputs are connected to the 4 switch boxes situated at each cluster sides. Like illustrates Figure 1.16, the Mesh-of-Tree interconnect topology is constructed with a matrix of Switch Boxes (SBs) presented in Figure 1.16.Switch Boxes ensure signals driven in the FPGA.

From a link-up point of view, there are 3 types of SBs.

- Internal SBs are connected to 4 clusters, 4 neighboring switch boxes and do not have direct connection with Mesh of Tree I/O pads the case of Figure 1.18 (4 clusters :a, b, c, d and 4 switch boxes a, b, c, d).
- Boundary SBs are connected to 2 clusters, 3 neighboring switch boxes and have direct connection with Mesh of Tree I/O pads (c.f Figure1.19).
- Corner SBs are connected only one cluster and 2 adjacent switch boxes and have direct connection with Mesh of Tree I/O pads.

Characterized with a Tree-based interconnect topology, SBs have Down to Up and Up to Down interconnect respectively (upward and downward networks link- up).

As it contains a similar Tree-based interconnect, SBs parameters are the following [3]:

- The upper SB DMSBs number $nb_{DMSBL2} = ChannelWidth$.
- The lower SB DMSBs number $nb_{DMSBL1} = nb_{ClusterInputs}/4$, ($nb_{ClusterInputs}$ is the number of mesh cluster inputs).
- The SB UMSBs number $nb_{UMSB} = nb_{ClusterOuputs}/4$, ($nb_{ClusterOuputs}$ is the number of mesh cluster outputs).
- Number of Feedbacks $nb_{Feedbacks} = nb_{ClusterOuputs}/4 \times nb_{ClusterNeighbors}$ ($nb_{ClusterNeighbors}$ is the number of neighbor clusters connected to the SB).

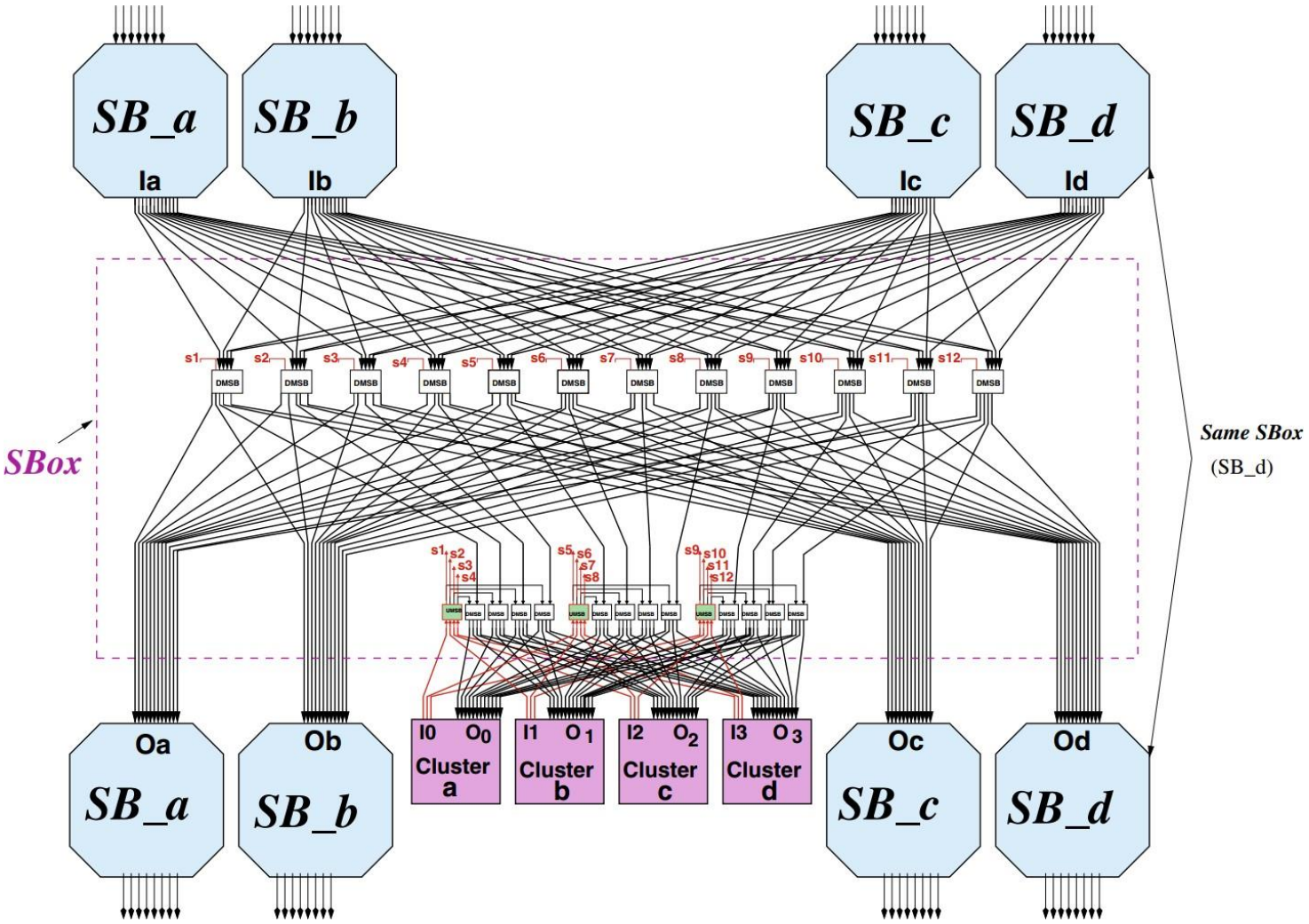


Figure 1.18: Mesh of Tree Switch Box architecture: inter-clusters interconnect.

1.2.2.4 Input/Output Pads Connection

Input and output pads are directly connected to SBs situated at the Mesh of Tree Boundaries which is the case of Corner and Boundary SBs. Like presents Figure1.19, a boundary SB connected with the outside through I/O pads. Like previously mentioned, the internal SB interconnect is Tree-based which means that it contains upward and downward networks. Thus, input pads are connected to the upward networks through UMSBs and output pads are connected to DMSBs via DMSBOUT: the downward network.

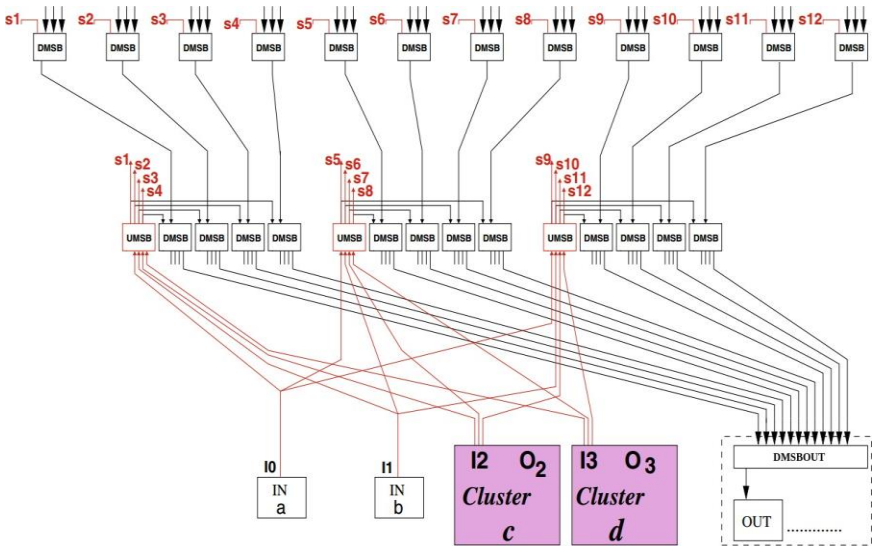


Figure 1.19: External Input and Output pads connection: Boundary Switch Box case.

1.2.2.5 FPGA CAD flow

FPGA designing require two major axes that come together to fulfill the complete realization and utilization of a new FPGA IP. The first which is mainly our center of interest is the hardware flow that starts from an FPAG architecture modelization to a concrete description of the hardware of the FPGA in a HDL (Hardware description Language). The second is the software flow also named CAD flow (Computer Aided Design) which represents a set of steps that will enable us to get any design to be configured into our FPGA (generated by the hardware flow).

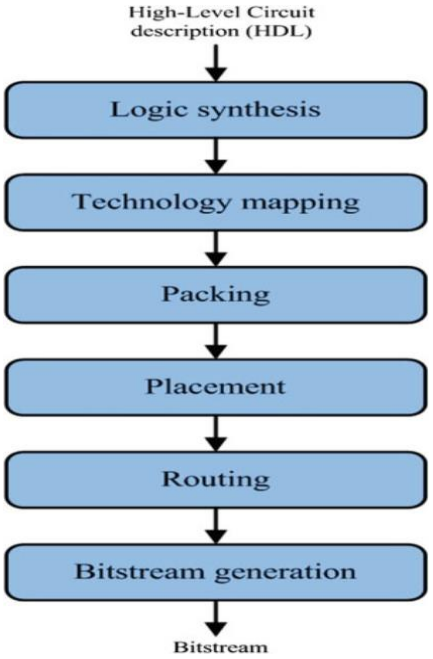


Figure 1.20: Generalized software flow.

CAD flow, is a set tools that enables the mapping of applications to FPGAs. This flow effectiveness is crucial, it determines the quality of an FPGA based implementation. The software flow begins by taking the hardware description design of an application in a HDL and converts it into a bitstream which is a binary representation of the application that will be programmed on the FPGA.

This conversion process in 1.21 of the circuit description into a format that can be loaded in the FPGA, highlights how an application circuit can be programmed on an FPGA, can be represented and divided into these steps below [13].

- Synthesis: Transforms a HDL of a design into a netlist.
- Technology mapping: Transforms a netlist of gates and registers into a netlist of K-input Look-Up-Tables (LUTs) and registers.
- Packing (also named clustering): Pack LUTs and registers into clusters in order to minimize area and maximize routability. Therefore, logic blocks with common inputs or ones outputs are others inputs will be packed together into the same cluster.
- Placement: Clusters are mapped to physical location within the FPGA.
- Routing: Assign nets to routing fabric so that each routing fabrics doesn't share the same net.
- Bitstream generation: Generate bitstream information which contains the description of which static random-access memory (SRAM) bits to be programmed to 0 or 1.

The output of this CAD flow configures the state of the memory bits in an FPGA which controls and determines the implemented logic functions on FPGAs.

1.2.2.6 Design Verification

The design verification is the most critical step of the hardware life cycle. Its main goal is to prove that the design meets the requirements. In fact, it consists of verifying and validating the expected behavior of the design. According to [14], the cost of verification is about 40% of the overall design cost. Figure 1.21 illustrates a comparison between Simulation, Emulation and Prototyping. This latter, are the most advanced and powerful verification techniques based on the four major factors: compile time, visibility, capacity and performance.

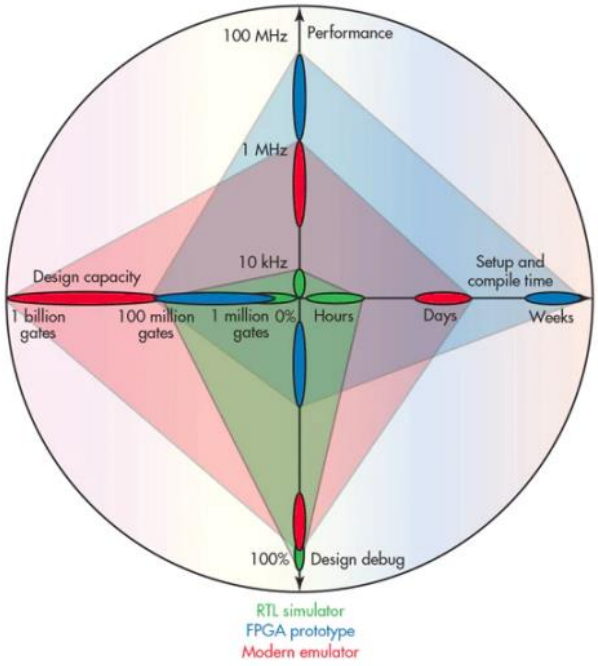


Figure 1.21: Comparison of design verification techniques.

As for simulators, which are software tools that provide a general behavior of the design, starting from a conceptual model. This technique offers full- visibility into the register transfer level (RTL) representation of the design. The set-up for implementation is instantaneous, and the cost is about thousands of dollars. Nevertheless, the execution speed is very limited (a few kHz).

Whereas, emulation is the use of a special computer (popularly called Emulator), which automatically maps the RTL of a design to its internal programmable gate array to perform the functional verification of the design. This method presents a high-visibility level in the RTL and a much faster execution speed than the simulation (about 1MHz). However, emulators are the most expensive (about millions of dollars) and take several days to several weeks for set-up.

As far as FPGA prototyping is concerned, it is a technique used for verifying the functionality and the performance of hardware design, as application-specific integrated circuits (AISC), and system on chip (SOC), by porting their RTL to an FPGA, and it can be used post tape out for software development. This technique is the one that offers the best execution speed (5MHz-100MHz) but the least visibility compared to other verification techniques in terms of debugging. It costs about ten thousand dollars and takes several weeks to several months for

set-up. Different from simulation and emulation, FPGA-based prototyping enables "real world" testing. This means the prototyped design is put into actual hardware and real external interfaces such as double data rate (DDR), Peripheral Component Interconnect (PCI), Ethernet, etc...

We will therefore implement a new powerful and optimized FPGA IP that can be used to prototype more complex systems and applications.

In the next subsubsection we will present the the MPSoC architecture within we will integrate our IP FPGA as an eFPGA.

1.2.3 MPSoC overview

Multiprocessor System on Chip (MPSoC) is a system on chip (SoC) containing multiple processors [15]. Nowadays industry, is moving to many cores architectures hence the use of high performant applications. However, MPSoCs have difficulties of reduced TTM and flexibility. Thus, integrating an FPGA, as a configurable accelerator, into an MPSoC seems to be promising c.f Figure 1.22.

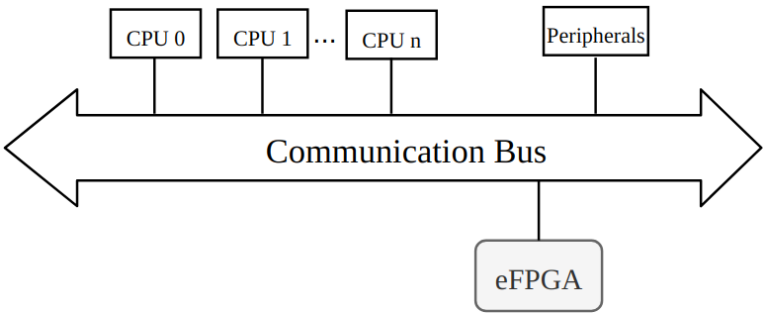


Figure 1.22: MPSoC architecture.

1.2.3.1 SoC

The increased technological development in the field of semiconductors has made it possible to manufacture entire systems of numerous and complex integrated circuits. All over the past decades, the Integrated Circuits design flow has become more developed and strong. Indeed, the EDA (Electronic Design Automation) tools and their increasing innovation are behind this prosperous result. Among the most used systems in the integrated circuits design industry, we have the ASIC (Application Specific Integrated Circuits) and SoC (System on a chip). While the ASIC manifests as a customized designed chip for a specific application, the SoC which is a system on an integrated circuit that integrates hardware and software IP (Intellectual Property) that makes available more than one design methodology to define the behavior of the proposed system, it also contains a high-level integration appropriate for performance and efficiency requiring applications.

1.2.3.2 Microprocessor

The microprocessor, also known as the central processing unit, is an electronic circuit within a computer that executes the instructions that constitute a computer program. Its three main units are an arithmetic logic unit (ALU) which handles arithmetic and logic operations, a control unit (CU) that directs the processor operations and the third components are registers used to stores data as input or output of operations. Memory, a location where information is kept while not in current use, is used to store programs, to store instructions whenever the microprocessor needs them to be executed [16].

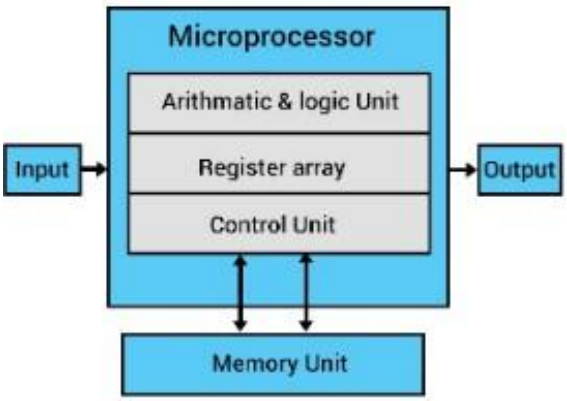


Figure 1.23: An overview of a basic microprocessor architecture.

1.2.3.3 OpenPiton architecture

OpenPiton is a many-core generator developed by the Princeton Parallel Group. This generator is written with Verilog and can build a scalable processor from 1 to 500 million general-purpose cores. It has a configurable core and uncore and can generate RTL for both ASIC and FPGA. For the FPGA, it has many supported boards like the VC707 (with 50 MHz), and the VCU118 (with 100 MHz) [17]. For the ASIC, a SoC with 25 cores was implemented and can run with 1 GHz. This SoC can use many different cores. One of them is Ariane, which means it can run a Unix-like operating system.

Figure 1.24 shows the architecture of OpenPiton that will be the processor in our MPSOC and which will communicate with our eFPGA.

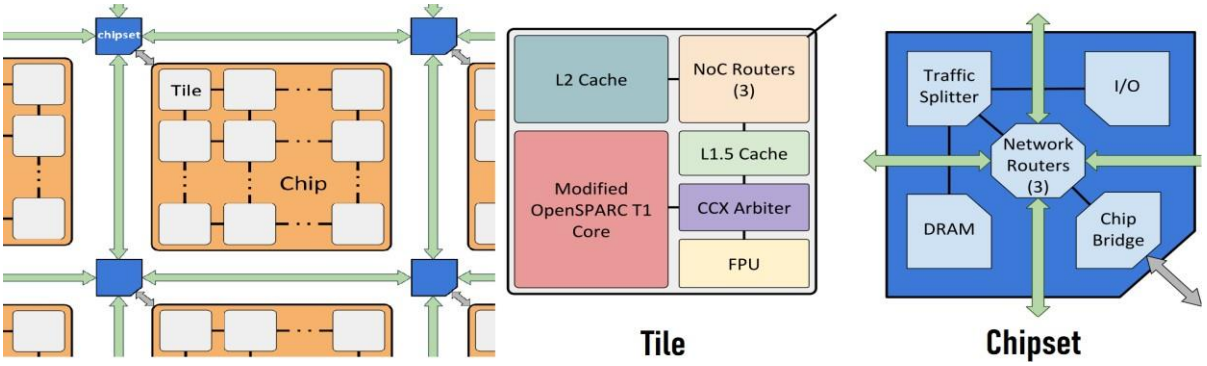


Figure 1.24: OpenPiton architecture.

OpenPiton's architecture is intended to be scalable. This architecture, as depicted in Figure 1.24, is made up of three components: Tile, Chip, and Chipset [17].

To construct the chip, tiles are linked together via a network on chip (NOC). Chips are also linked to one another via a chipset, which contains an additional network on the chip.

As shown in figure 1.25, the top module of the FPGA prototype is named system, and it is divided into chip and chipset logical blocks.

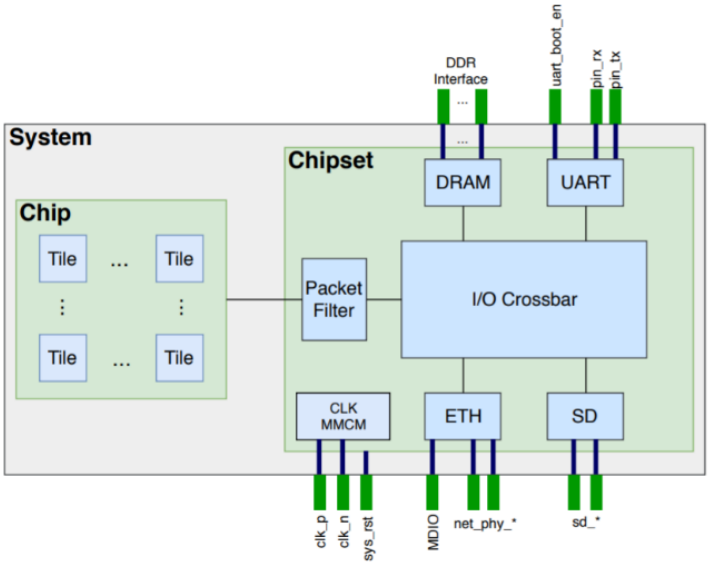


Figure 1.25: OpenPiton prototype architecture.

In turns, the chipset is composed out of the next submodules:

- The I/O Crossbar is a programmable crossbar that routes NOC packets. In this module, each INPUT/OUTPUT has its own port.
- The Packet Filter is a programmable module that alters NOC request packets. This module examines the destination address of a NOC packet request and then modifies the

port field in the I/O Crossbar. This module can also be configured using the device.xml file.

- The CLK MMCM is a clock generation IP from Xilinx. It generates all of the clocks required by the system, including the chipset clock and the memory clock.
- The DRAM is a programmable module that connects the main memory to the chipset.
- The UART is used in bare metal programming to print the output result. This module is made up of a NOC/AXI bridge and a UART16550 IP.
- The BOOTROM contains the first line of code executed by the processor.
- The SD card interface is used to connect an SD card to the processor. The operating system is stored on this SD card.
- The ETH IP from Xilinx is used to connect the processor to a network.

1.3 Objectives and Contribution

FPGA' interesting properties pushed us to move directly to explore its different topologies and optimize its physical conception and utilization so that can be used to prototyping more complex systems and applications, as well as, integrating it as a configurable accelerator.

This end of study project objectives are summed up in this Figure 1.25.

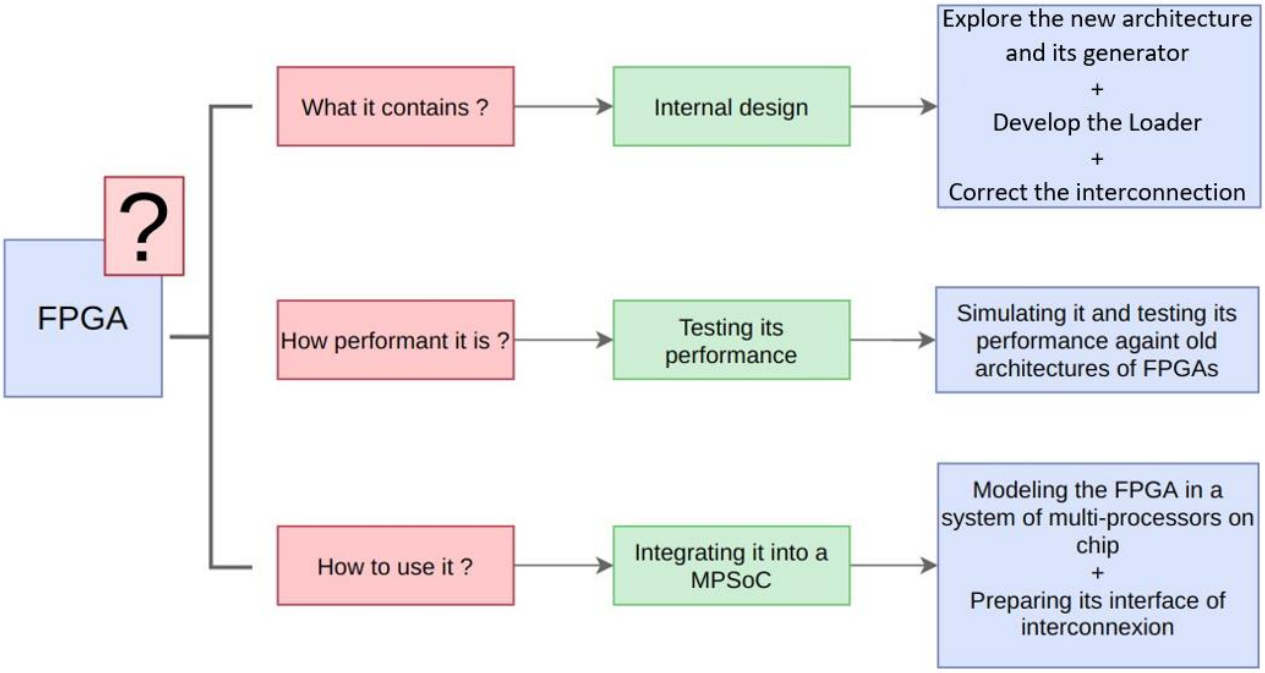


Figure 1.26: Project general steps.

In fact, the first objective of this project is to explore the new mesh-of-tree architecture and understand its Verilog generator which based on C++. The second objective of this project is to develop the loader of our FPGA which is responsible of configuring it. The next mission is to simulate and debug the entire IP generator to correct the interconnection error and fix the mismatching between the software flow and the hardware flow. The final goal is to integrate the FPGA IP into an MPSoC (Multiprocessor System on Chip) platform as a configurable accelerator.

1.4 Conclusion

In the first chapter, we initiate with a brief presentation of the basic elements towards a good understanding of the next chapters. We started by introducing the different FPGA architectures such as Mesh architecture and tree architecture, and then we proposed the unified mesh-of-tree architecture. In a second step, we introduced the MPSoC architecture in which we will integrate our FPGA IP as an eFPGA, finally we focused on the objectives and contribution.

CHAPTER 2

Mesh-of-Tree Architecture Design

2.1 Introduction

Following the presentation of the Mesh-of-tree architecture in the previous chapter, we needed to explain the components required to build this FPGA and how they are interconnected. Indeed, we will present the FPGA generator based on the Mesh-of-Tree. Then, we will present how we will address each component of the whole architecture to configure it. Finally, we will describe how we will connect this IP to OpenPiton in order to automate the operation of loading bitstream and assigning inputs.

2.2 Design of the existing generator and interconnection network

Since our goal is to generate the hardware description of the Mesh-of-Tree architecture, it is crucial to know how its components are connected and how signals are routed to enter their corresponding components. In this part, we start by presenting the whole architecture connection that is already implemented in our generator, so that we can later develop the loader and validate the whole architecture.

2.2.1 Mesh-of-Tree Generator Overview

The generator takes as input the Mesh of Tree FPGA description file and generates the necessary files to describe the given FPGA architecture (c.f Figure2.1).

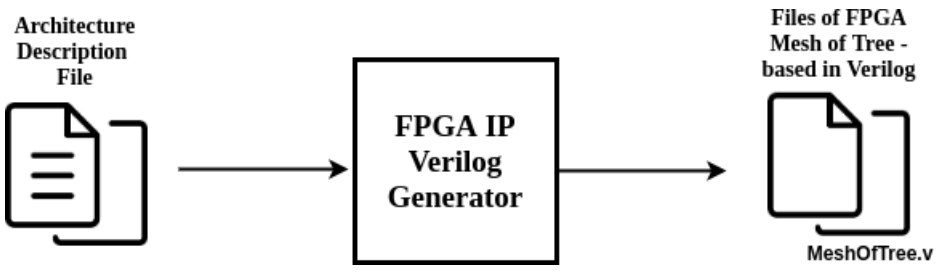


Figure 2.1 :Generator black box overview.

The system is fed a human-readable Mesh-of-Tree FPGA architectural description file. The interconnect generator Mesh-of-Tree uses this file to generate the necessary architecture files. Figure3.2 shows an example of this file format.

```
nx = 2
ny = 3
in_rate = 2
out_rate = 2
network width = 13
nb_levels 2
      .level 0 nb_slaves 4 nb_inputs 16 nb_outputs 4
      .level 1 nb_slaves 4 nb_inputs 64 nb_outputs 16
      .basic ble nb_inputs 4 nb_outputs 1
      .IOlevel 1
      .word_width 8
```

Figure 2.2: Architecture Description File.

The file contains the following information:

- nx, ny : are respectively the number of rows and columns of the Mesh Of Tree which corresponds to the number of clusters, here $2 * 3 = 6$ clusters.
- in_rate: is the number of entries in each input block of the grid.
- out_rate: is the number of outputs in each output block of the grid.
- network_width: is the number of one-way connection tracks between SBs in the mesh.
- nb_levels: is the number of levels in the Tree.
- level 0: describes the number of slaves (BLE), the number of inputs and the number of outputs of level 0. In the example in figure 2.3, level 0 of the tree has 4 slaves (4 BLEs) with 16 inputs and 4 outputs (see figure 2.3).
- level 1: describes the number of slaves in level 1 (level 0 cluster), the number of inputs and the number of outputs. In the example in Figure 2.3, level 1 of the tree has 4 slaves (4 level 0 clusters) with 64 inputs and 16 outputs.
- IOLevel: it specifies the level of the Tree clusters connected directly to the Switch Boxes (IO stands for Input/Output, the SBs wires represent the I/O pads to the Tree: because the SBs are connected directly to the external IO pads).
- word_width: is the size of the word that allows the FPGA configuration.

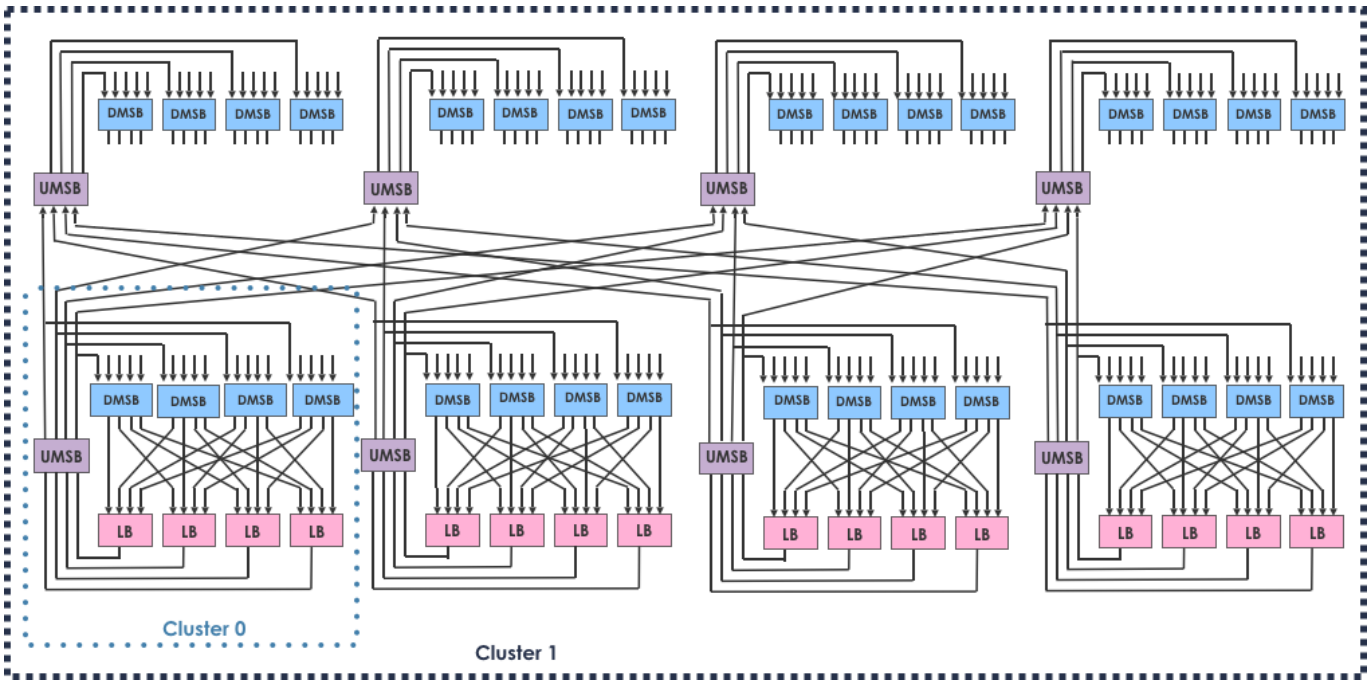


Figure 2.3: Architecture Description File.

The 2 levels Tree is simplified in Figure2.4 where we have represented its headlight components.

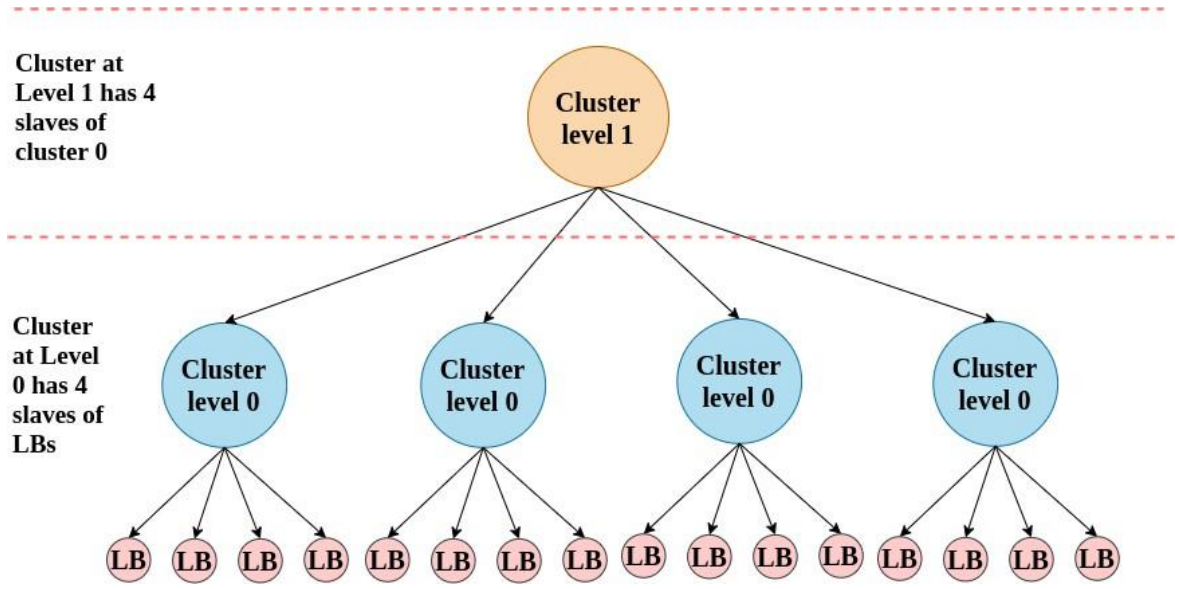


Figure 2.4: Simplification of the Tree in 2.3.

Once the parsing is complete, the parser loads the various parameters into the archParam file and builds all the necessary architecture parameters.

```
//user params
//--Mesh (Nx * Ny ) : clusters size
unsigned _Nx;
unsigned _Ny;
unsigned _channelWidth;
unsigned _inRate;  // inputs pads rate
unsigned _outRate; // inputs pads rate
//--Tree
unsigned _nbTreeLevels;
vector<unsigned> _nbTreeSlaves;     // salves per level
vector<unsigned> _nbTreeClusterInputs;  // cluster inputs per level
vector<unsigned> _nbTreeClusterOutputs; // cluster outputs per level
unsigned _nbLeafInputs; //BLE inputs
unsigned _nbLeafOutputs;// BLE outputs
unsigned _IOLevel; //
unsigned _wordWidth;
//other params
//--Tree cluster
unsigned _nbBLEs;
vector<unsigned> _nbTreeClusters;         // clusters per level
vector<unsigned> _nbTreeClusterFeedbacks; // cluster feedbacks per level
vector<unsigned> _nbTreeDMSBs;            // nb cluster DMSBs per level
vector<unsigned> _nbTreeUMSBs;            // nb cluster UMSBs per level
vector<unsigned> _minWiresInTreeDMSB;
vector<unsigned> _maxWiresInTreeDMSB;
vector<unsigned> _nbTreeDMSBsWithMaxWires;
vector<unsigned> _nbTreeDMSBsWithMinWires;
vector<unsigned> _minFeedsInTreeDMSB;
vector<unsigned> _maxFeedsInTreeDMSB;
vector<unsigned> _nbTreeDMSBsWithMaxFeeds;
vector<unsigned> _nbTreeDMSBsWithMinFeeds;
vector<unsigned> _minWiresOutTreeUMSBs;
vector<unsigned> _maxWiresOutTreeUMSBs;
vector<unsigned> _nbTreeUMSBsWithMaxWires;
vector<unsigned> _nbTreeUMSBsWithMinWires;
```

```
//--SB                *** Sbs cases ***
unsigned _nbSbDMSBsL1;
unsigned _nbSbDMSBsL2;
unsigned _nbSbUMSBs;
vector<unsigned> _nbSbFeedbacks;
unsigned _minWiresInSbDMSBsL1; // inputs of sbDMSBL1 depend only on nbSbDMSB L1 & L2
unsigned _maxWiresInSbDMSBsL1;
unsigned _nbSbDMSBsL1WithMaxWires;
unsigned _nbSbDMSBsL1WithMinWires;
vector<unsigned> _nbWiresInSbDMSBsL2;
vector<unsigned> _minFeedsInSbDMSBsL1;
vector<unsigned> _maxFeedsInSbDMSBsL1;
vector<unsigned> _nbSbDMSBsL1WithMaxFeeds;
vector<unsigned> _nbSbDMSBsL1WithMinFeeds;
vector<unsigned> _minFeedsInSbDMSBsL2;
vector<unsigned> _maxFeedsInSbDMSBsL2;
vector<unsigned> _nbSbDMSBsL2WithMaxFeeds;
vector<unsigned> _nbSbDMSBsL2WithMinFeeds;
vector<unsigned> _minWiresOutSbUMSBs;
vector<unsigned> _maxWiresOutSbUMSBs;
vector<unsigned> _nbSbUMSBsWithMaxWires;
vector<unsigned> _nbSbUMSBsWithMinWires;
// --adresses for bitstreaner
vector<unsigned> _addressWidth;
vector<unsigned> _MSB;
```

Figure 2.5: Mesh-of-Tree architecture parameters.

As shown in Figure 2.6, we focused on the highlighted part. Indeed, after reading the description file, the generator of the Mesh-of-Tree architecture generates the hardware description of the FPGA in Verilog which we have chosen as the hardware description language (HDL). This choice of Verilog is constrained by the data structure used to develop the generator and also by the advantages of Verilog via VHDL. Indeed, the primary distinction between Verilog and VHDL is that Verilog is based on the C programming language, whereas VHDL is based on the Ada and Pascal programming languages and, on the other hand, is less complex.

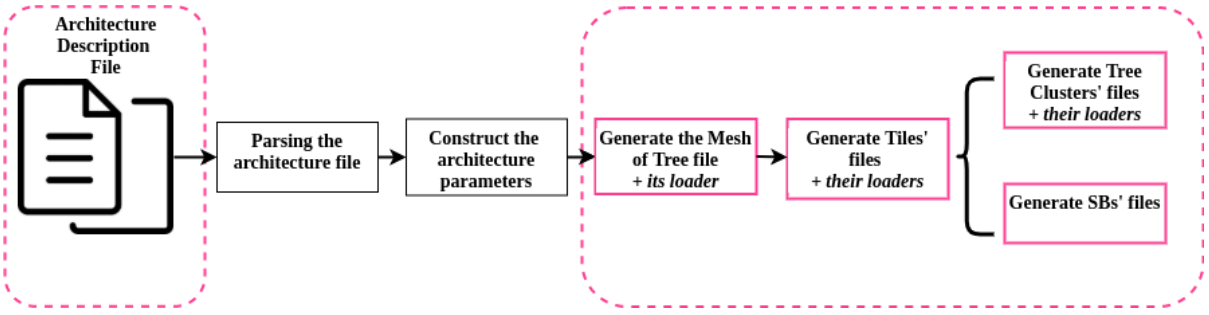


Figure 2.6: Mesh-of-Tree generator general overview.

The data structure used is the HNL (Hierarchical Netlist), developed and used by Mentor Graphics to create FPGA netlists. It allows us to describe the hardware components and their interconnections. The HNL has proven to be powerful, compact, easy to use and reusable. It allows a compact representation of the FPGA netlist and thus optimizes the implementation of the generator interconnection.

HNL is a Mentor proprietary solution implemented in C++ for representing hypergraphs in Verilog.

2.2.2 Design of the Mesh-of-tree interconnect

After providing an overview of our generator, we will go into greater detail in this subsection by presenting the Mesh-of-Tree FPGA building blocks modeling.

2.2.2.1 FPGA Basic Building Blocks

Since we are going to design the hardware description of the FPGA, we need to generate the mesh of figure 2.7 by breaking it down into tiles or building blocks.

In fact, FPGA fabrication is a complex and demanding process that necessitates a massive manpower investment if we do not consider optimization and automation of both the software and hardware design flow. Because we're concentrating on the hardware, which is one of the most time-consuming and labor-intensive steps, we will look at transistor-level design and physical layout.

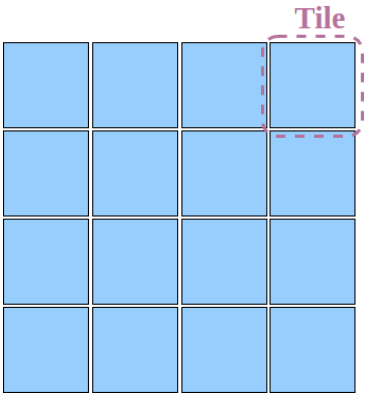


Figure 2.7: FPGA Tile presentation.

The literature mentioned in [18][19] presented an approach to the generation of the physical layout of FPGA hardware, which considers the FPGA as repetitive blocks arranged in rows and columns [18] in order to reduce manual work in the generation of the physical layout.

Figure 2.7 shows an FPGA tile, repeated several times to build the FPGA network [20]. These are considered as building blocks because they can be replicated to build a structure.

Authors of [21][22] used tiles considering the automation of FPGA layout generation. While authors of [23] worked on optimizing tiles for area in FPGAs, Authors of [3] focused on generating tiles for a mesh-based FPGA and named them modules. The tile generator in [3] starts by generating the netlist and a partial layout of FPGA tile as illustrated in Figure 2.8.

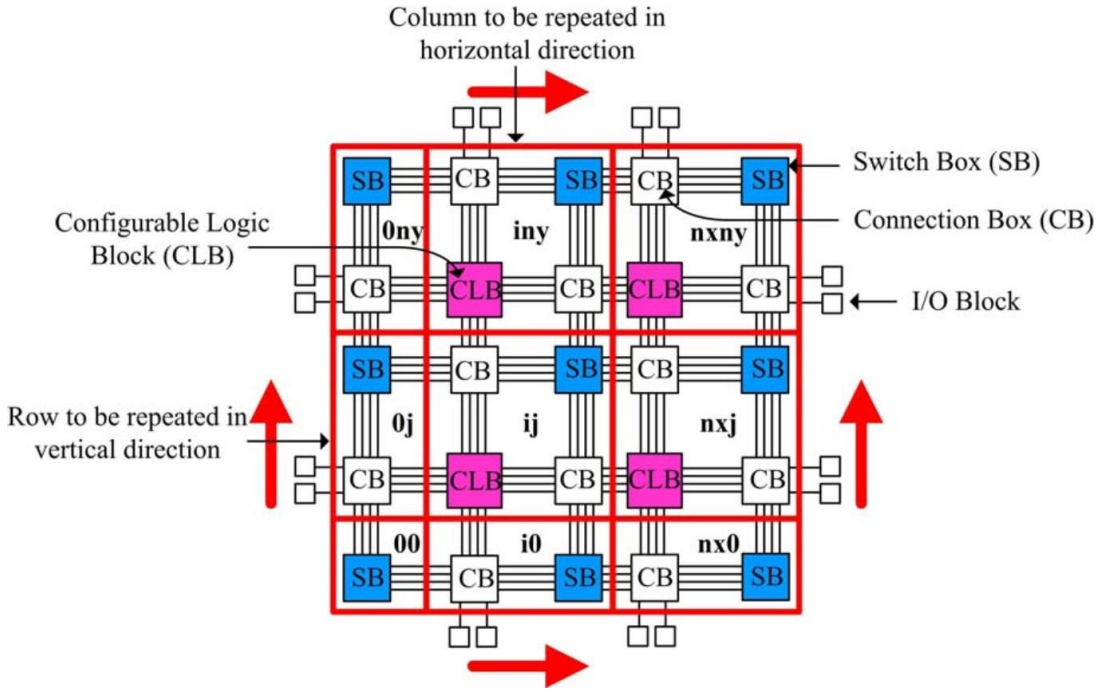
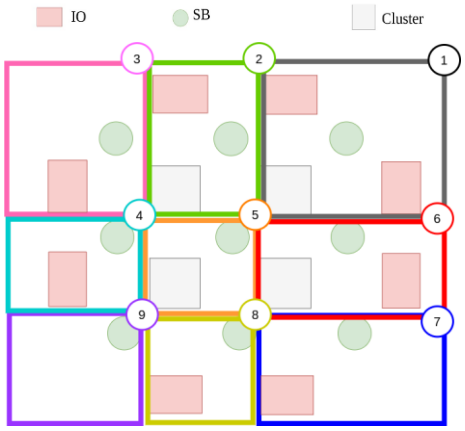


Figure 2.8: Tiles for Mesh-based FPGA.

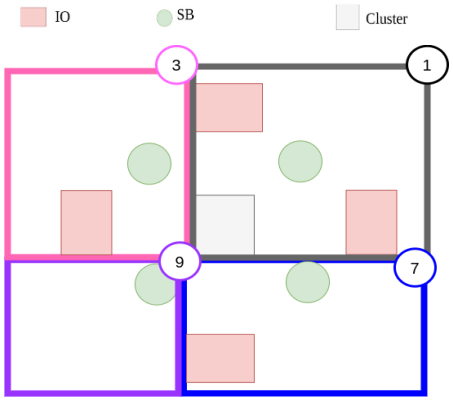
After being inspired by the promising FPGA-based constructive solution. We will now introduce this tile-based solution that uses this last great idea of tiles to facilitate the fabrication and scalability of our FPGAs.

Our Mesh-of-Tree Interconnect, also known as a grid of Clusters, is a matrix of Tree-based Clusters linked by Switch Boxes and connected to the outside world via I/O Pads (Input and Output Pads). Figure 1.16 in the previous chapter depicts the main components of the Tree Mesh.

The solution presented in 2.2.2.1 considers the mesh as a matrix of blocks called tiles. In fact, there are 9 different types of tiles to build any FPGA dimension with the Mesh of Tree topology (see Figures 2.10 and 2.11). By considering this construction method, the interconnection of Mesh FPGAs is easier to develop and validate and makes FPGA construction scalable by replicating tiles of the same type.

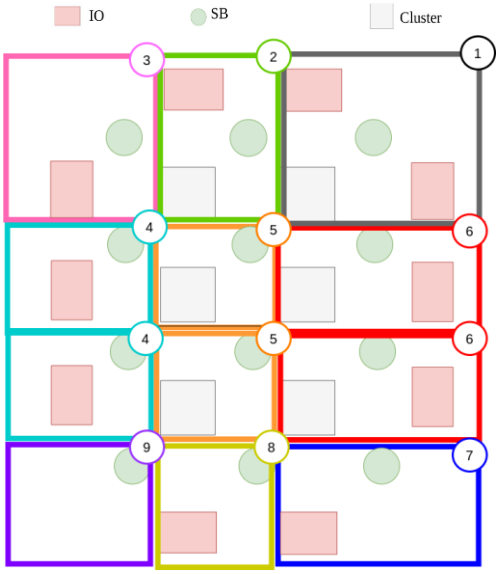


(a)A 2x2 Mesh of Tree using all the 9 Tiles.

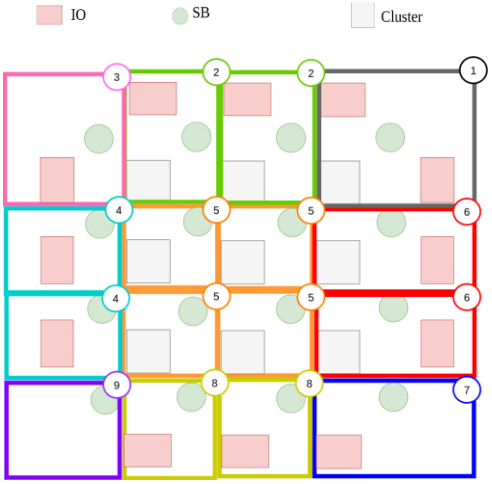


(b) The utilization of only 4 tiles (the 4 corners) to construct the FPGA.

Figure 2.9: Distinct use of tile's modules.



(a) A 3x2 Mesh of Tree using all the 9 Tiles with reuse of Tile' modules (4), (5) and (6) to construct the whole mesh.



(b) A 3x3 Mesh of Tree using all the 9 Tiles with reuse of Tile' modules (2), (4), (5), (6) and (8) to construct the whole mesh.

Figure 2.10: Use of tile's modules twice.

As we have already explained, we considered 9 different tiles to build a Mesh-of-Tree based FPGA. As mentioned earlier, these modules can be reused in the same FPGA and instantiated as many times as needed to achieve the required mesh sizes. The different tile modules are explained in Figure 2.11 and Figure 2.12.

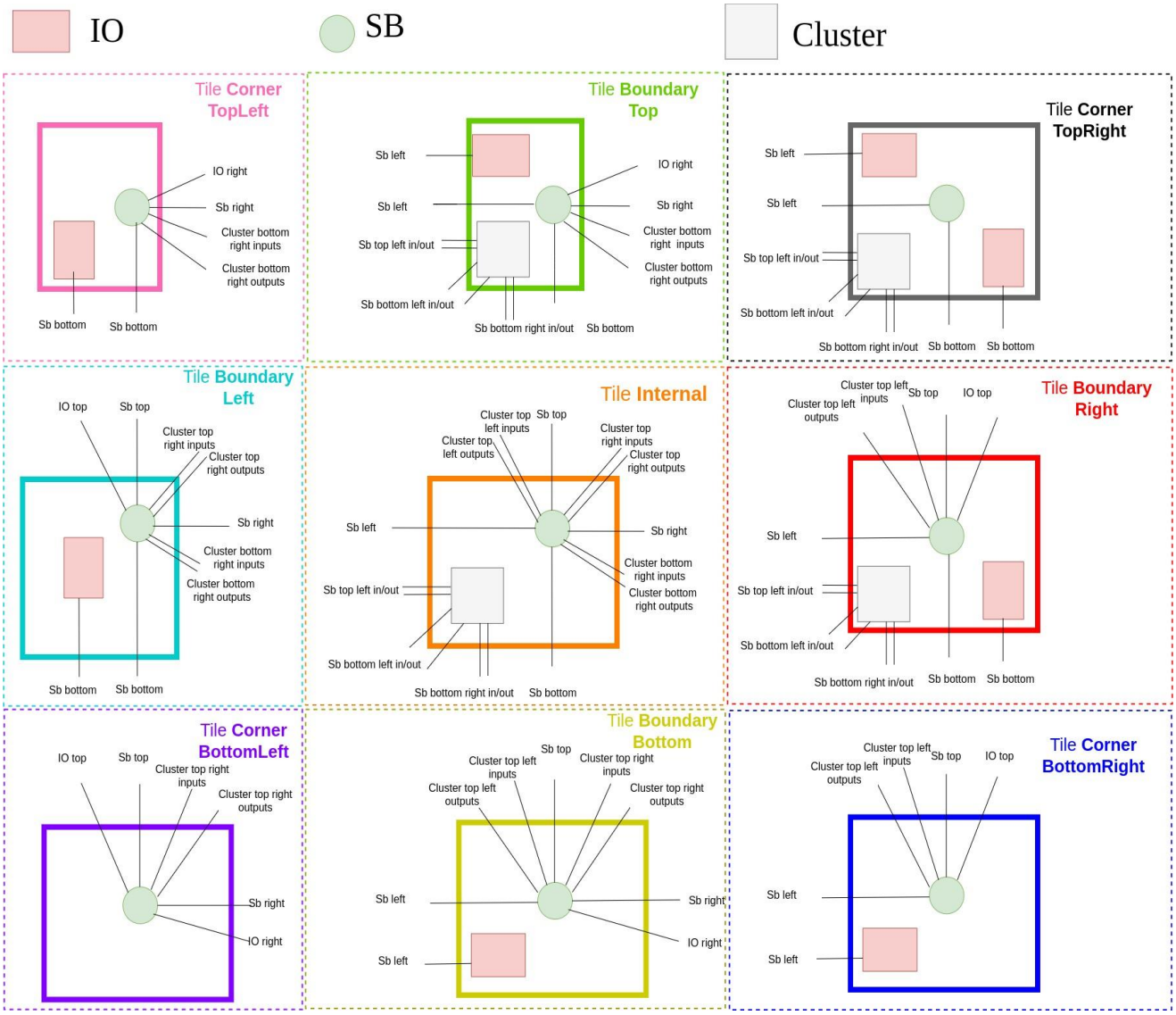


Figure 2.11: Different tile's modules.

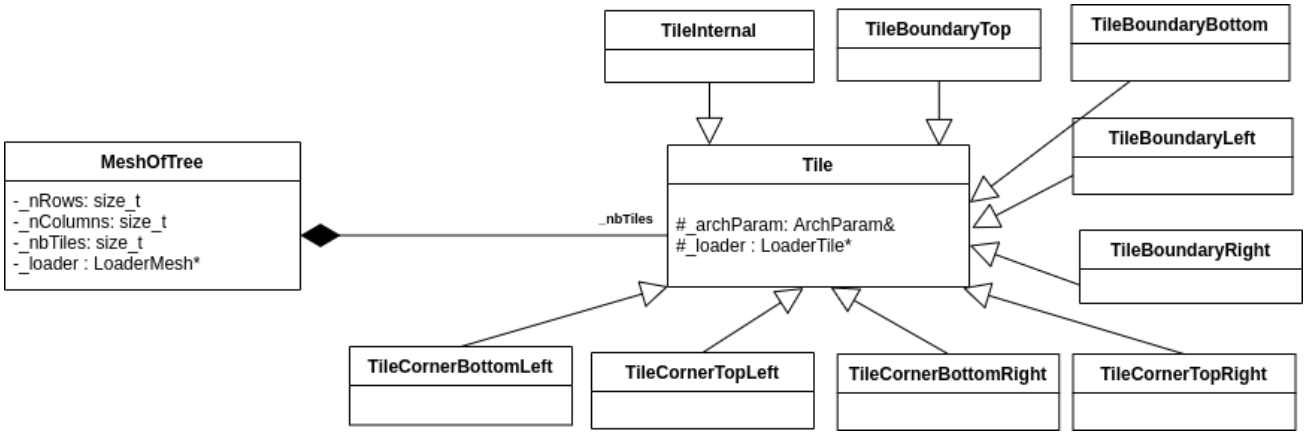


Figure 2.12: Class diagram of the 9 types of tiles.

As containers of I/O or SB and/or Cluster pads, depending on their type, the tiles are replicated to form a complete FPGA core. A tile must locate incoming or outgoing signals so that we can connect them to the appropriate port in its adjacent tiles replicated on its 4 sides.

Since these tiles will be processed at the transistor level and built into physical tiles, the position, direction of each port and its components (Cluster, SB, IO pads) are the main factors to reveal these 9 types. As shown in Figure 2.11, each tile contains an SB. SBs are responsible for routing signals through the mesh. As for the I/O pads, we only consider them in the corners and boundaries of the mesh, therefore we only find them in the corner and boundary tiles.

If we consider the CornerTopRight tile where the SB (see Figure2.13) is directly connected to the I/O pads inside the same tile. Thus, the SB routes the input signals from the I/O pads to the Cluster. The outputs of the LBs (located in the tree) are routed directly through the same SB to reach the output pads in the same tile.

2.2.2.2 Conceptual Overview

This hypergraph aspect of the Mesh of Tree architecture was implemented to achieve the required generator output. In fact, this topology is embodied and designed in the class diagram in Figure2.13. In that figure, we have presented the most important classes responsible for describing the main components of the Mesh of Tree. In short, the Mesh-of-Tree is composed of $nb_{Tiles} \times Tile$. Each tile is composed of a Switch Box (Sb) and a Cluster. The cluster in the tile is structured as a tree of sub-clusters. Indeed, the Cluster class contains child clusters as well as MSBs (DMSBs and UMSBs) and BLEs (if the cluster level is equal to 0).

Each class in the diagram (see Figure 2.13) contains a pointer to decoders also called loaders. In fact, the loaders are used to load the configuration in the whole FPGA and thus to select a component and configure its strobes.

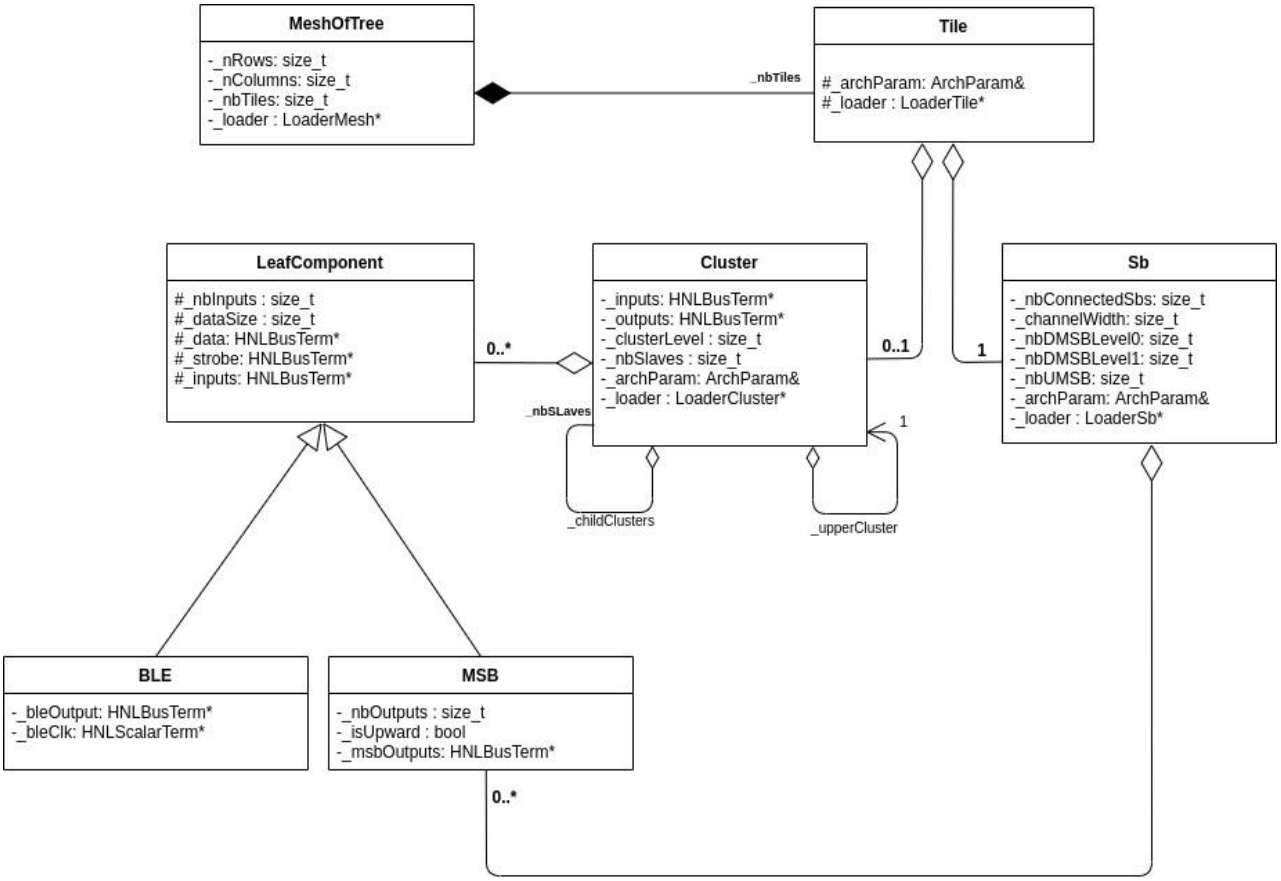


Figure 2.13: General class diagram of the Mesh-of-Tree.

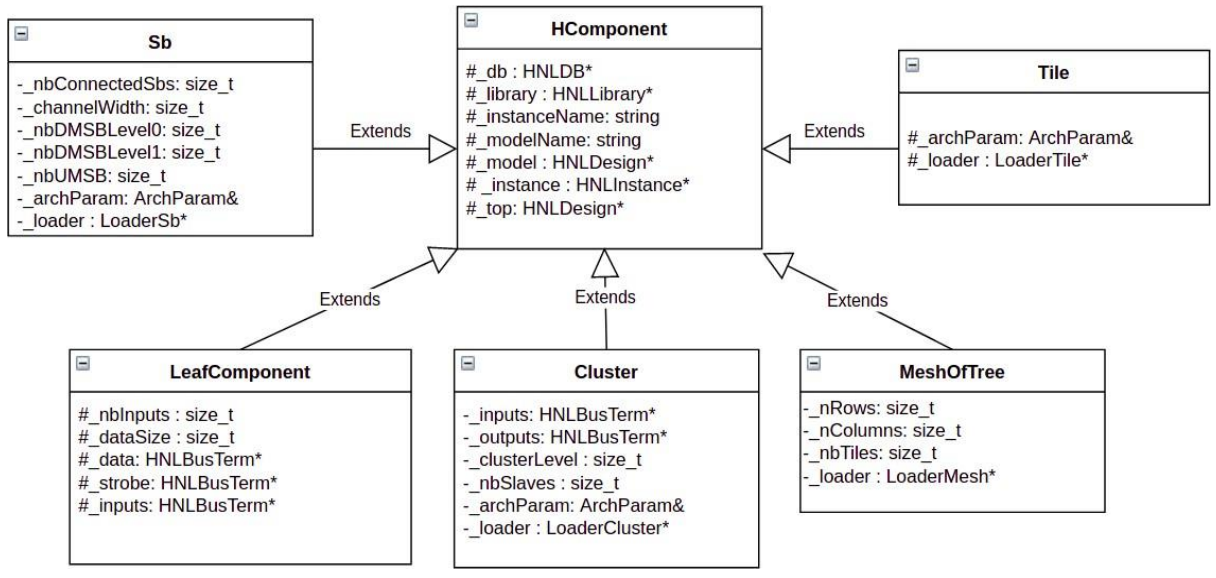


Figure 2.14: Inheritance

In figure2.14 we show the HComponent super class, it gathers the common attributes and function to describe a component in a HDL (Hardware Description Language) using the HNL data structure.

2.2.2.3 Behavioral Overview

In this section, we will discuss the behavioral part of our generator. As shown in Figure 2.15, the system first obtains the architecture parameters to calculate the number of tiles needed to build the full mesh. Then, it creates a tile design that depends on the cluster and SB designs. As for the SB design, we create DMSBs and UMSBs within the SB. For clusters, we create DMSBs, UMSBs and sub-clusters for clusters of level > 0, and UMSBs, DMSBs and LBs for clusters of level 0.

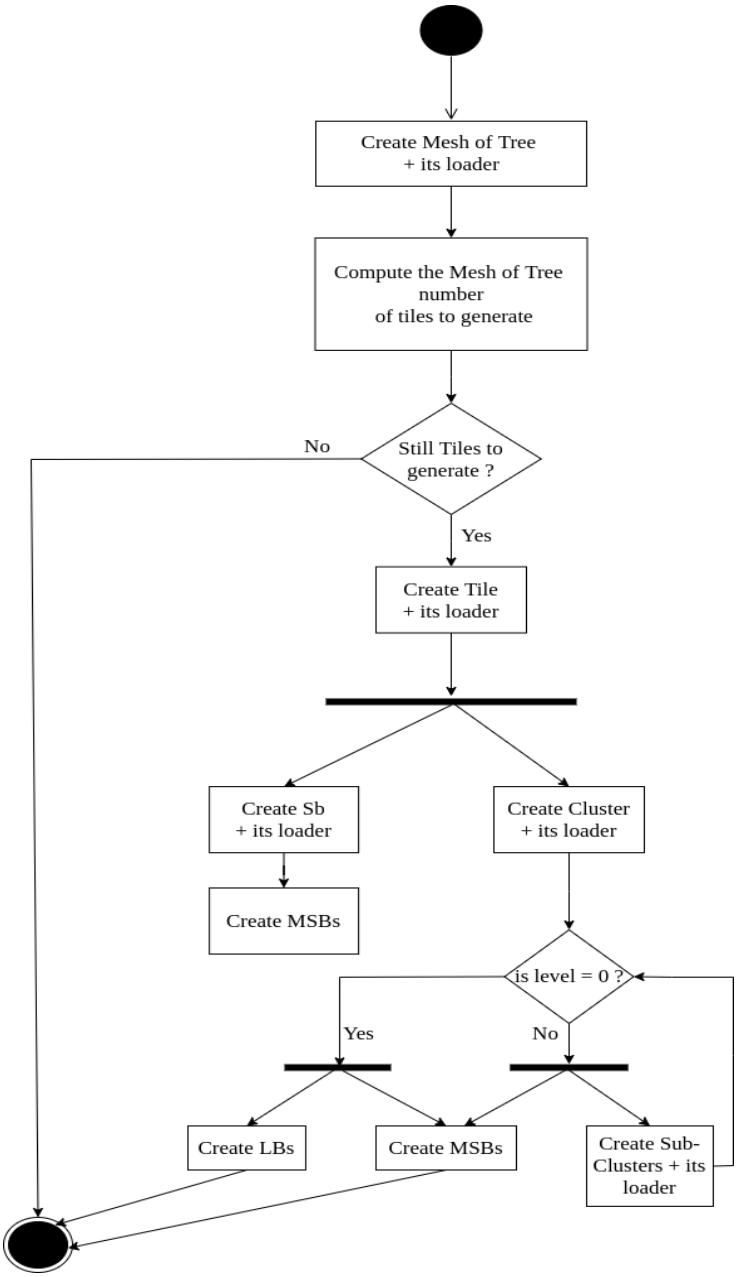


Figure 2.15: Activity diagram of the FPGA IP generator.

2.3 Loader Design

After designing and implementing the required interconnect to realize the description of the FPGA Mesh of Tree based, another essential step must take place. This step is the configuration of the FPGA which is the first development step in my end of study project. In fact, after creating and instantiating the required components of the FPGA design we have to address each one of them to configure them. In fact, this step of addressing and configuring each component is primordial since it allow us to validate the implemented design as well as to prepare the FPGA IP to get into the software flow in order to load bitstreams of application to get loaded in the FPGA.

In fact we will present in this section the Loader architecture.

2.3.1 Architecture Configuration

In order to reconfigure and implement a function on the FPGA we need first to reconfigure it by inserting logic functions into the SRAMs of the leaf components of the Mesh-of-Tree which are Logic Blocks (LBs).

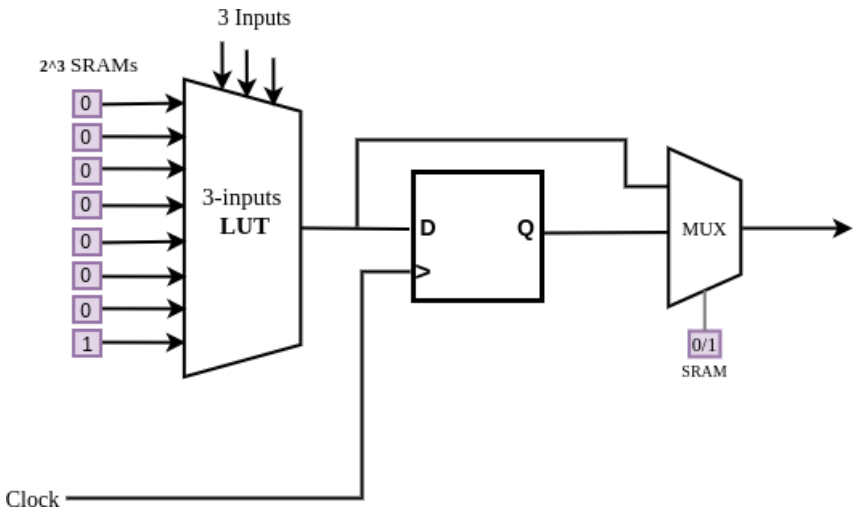


Figure 2.16: LB implementing and function in LUT.

At this stage it is important to know that the implementation of logic blocks, their connection and the routing of the whole topology (by configuring the appropriate SRAMs) are the main tasks for reconfiguring the FPGA. In fact, SRAMs serve to manage multiplexers select inputs in the case of switches all along the routing interconnect. As the case of logic functions in LBs, SRAMs configure logic functions of each LUT within each LB.

As shown in Figure 2.16, where we see a LUT configured as a function and, so that when the inputs are all one, it corresponds to the SRAM that contains 1.

2.3.2 Address Decoder

To configure the Mesh of Tree FPGA with a specific design configuration and to make this process as relevant as possible, we assign for each element of the architecture a specific address. With this address, we send for each element the corresponding data to be placed in the SRAMs cells of each element. Therefore, the instances of each tile in the mesh must be distinguished by meaningful indexes and addresses, and the same for SBs and clusters. Thus, we need different decoders to define the configurations in the whole Mesh of Tree architecture.

It is called a loader or decoder because it loops data through the Mesh of Tree. Figure 2.17 shows a general overview of a Mesh of Tree FPGA loader.

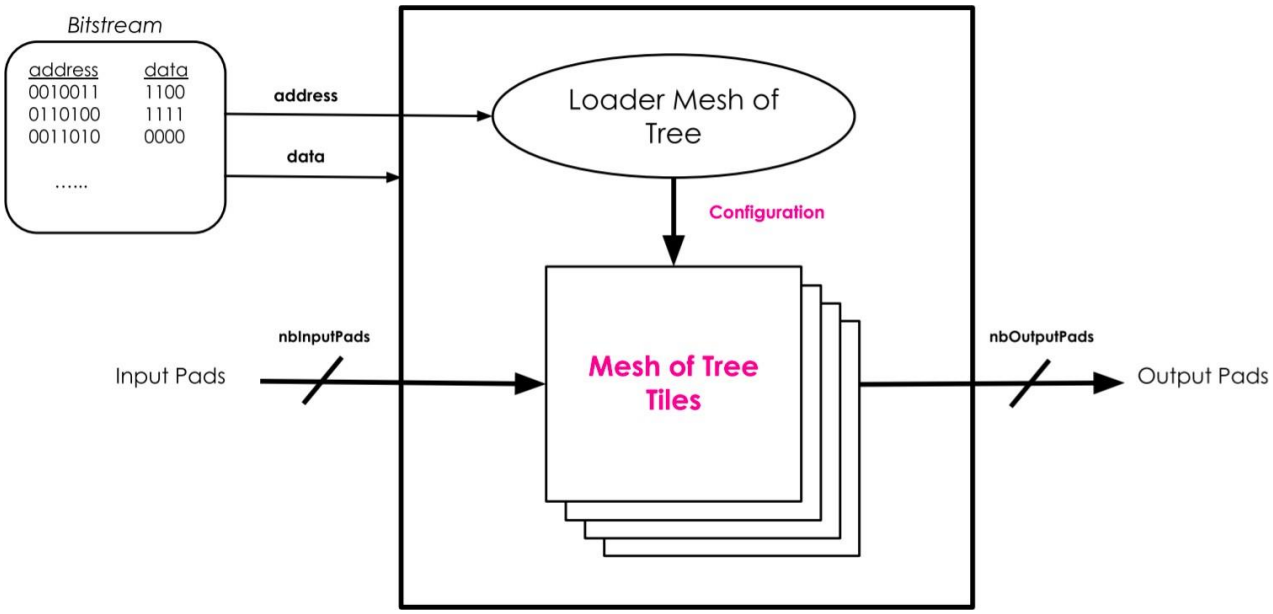


Figure 2.17: General overview of the Mesh of Tree Loader.

As we work on the hardware flow of the new Mesh of Tree architecture, we receive from the software team (those working on the Mesh of Tree FPGA CAD flow) the bitstream of a given design to be configured in the FPGA. The latter contains a sequence of addresses and data. As for the addresses, they are identifiers given to the FPGA multiplexers to access their memory and define a specific configuration. In fact, each element of the topology has one and only one corresponding address. This address is used to configure its corresponding element by configuring the SRAMs that control its multiplexers. As for the data, the loader selects an element according to the given address, the SRAMs activate (write activations called strobes) the writing of the data in the right memory cells.

2.3.2.1 Decoder at the Mesh Level

As shown in Figure 2.18, at the mesh level, the loader receives the input address and outputs the remaining address which decodes the tile levels with the tile selection thread. The tile selection thread allows us to decode the corresponding tiles identified in the address. Entering the tile ladder to load the configuration, we have 9 different types, 5 types with only SB. As for the remaining 4 tiles, they contain both an SB and a cluster. Therefore, as explained 2.18 we select according to the address the corresponding element. If the address (address range encoding the selected element, whether it is an SB or a Cluster) is equal to 0, then we will reconfigure the SB and select it. Otherwise, if it is equal to 1, we will reconfigure the Cluster by selecting it first.

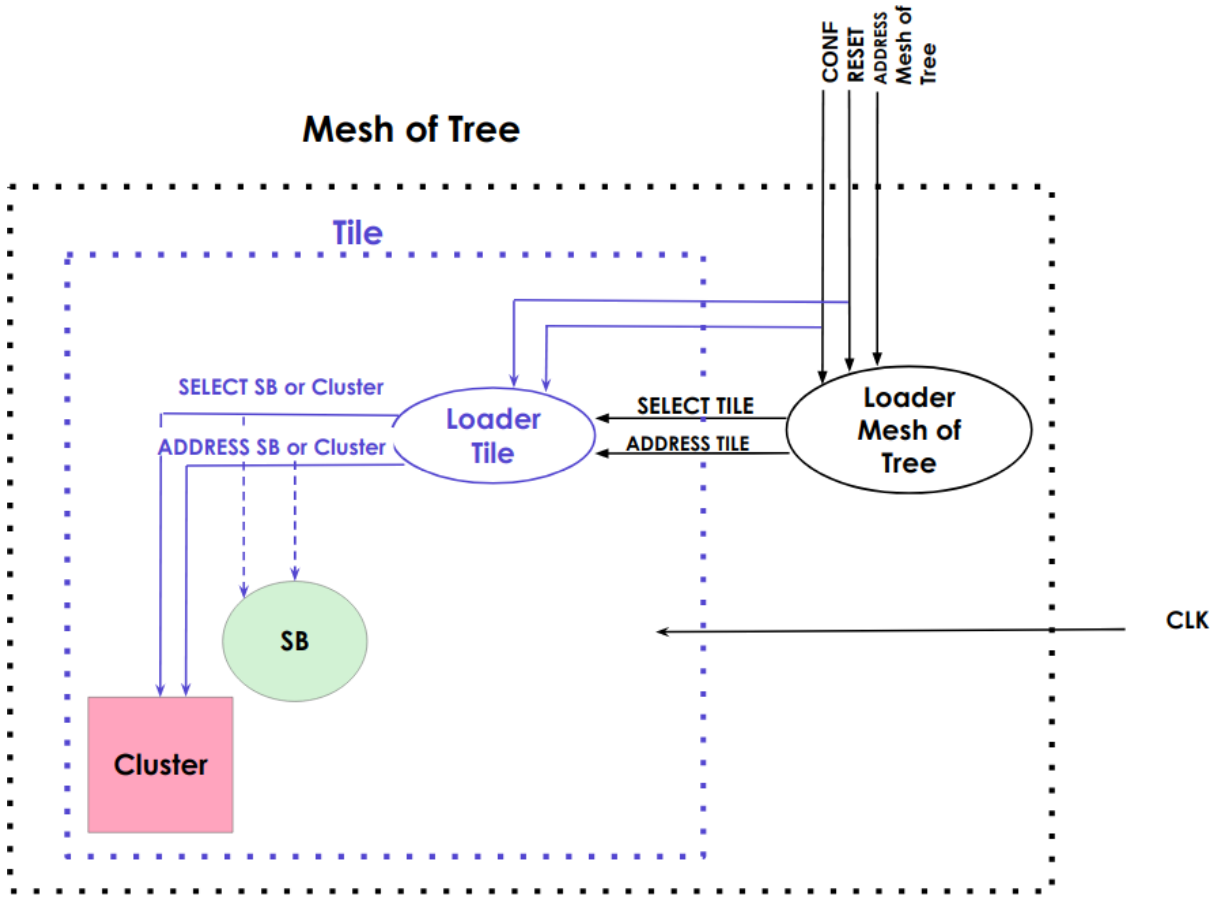


Figure 2.18: A detailed overview of the Mesh of Tree Loader containing a Tile with its corresponding loader.

After selecting an SB, the loader begins loading that SB. In fact, we set DMSBs and UMSBs SRAM strobes in accordance with the provided address. Figure 2.19 shows how to set up this arrangement.

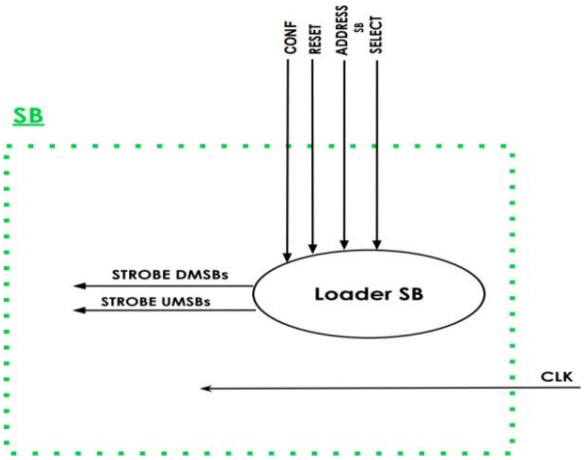
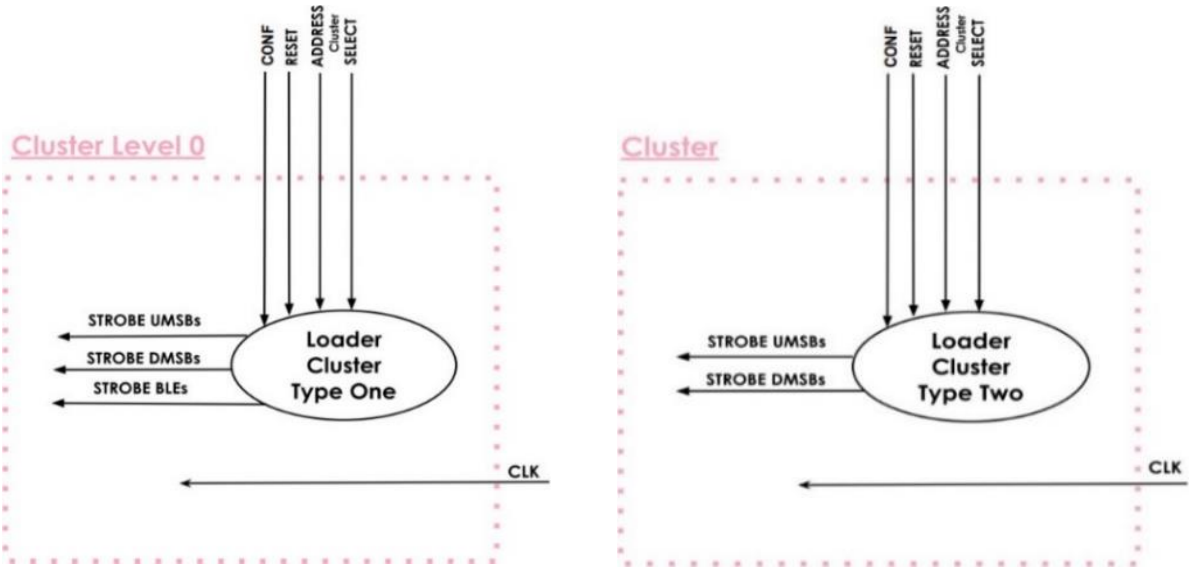


Figure 2.19: A Switch Box Loader.

2.3.2.2 Decoder at the Tree Level

After selecting a cluster, we move to the tree level. We have 2 different types of loaders. The first type, known as Loader Type 1, is in charge of clusters at the level 0 level. The clusters at level 0, are those that contain LBs as leaf components. Thus To load their configuration, we use a loader, as shown in Figures 2.20a and 2.21, which outputs SRAM strobes for UMSBs, DMSBs, and BLEs. In the second type, called Loader Type 2, the loaders only outputs the SRAM strobes for UMSBs and DMSBs since there are no BLEs, which is the one and only difference between them. In fact, the type 2 loader corresponds to clusters whose level is different from 0 as illustrated in Figure 2.20b and Figure 2.21.



(a)Loader of clusters at level = 0. (b) Loader of clusters at level > 0.

Figure 2.20: Loaders of clusters at level 0 and clusters at level greater than 0.

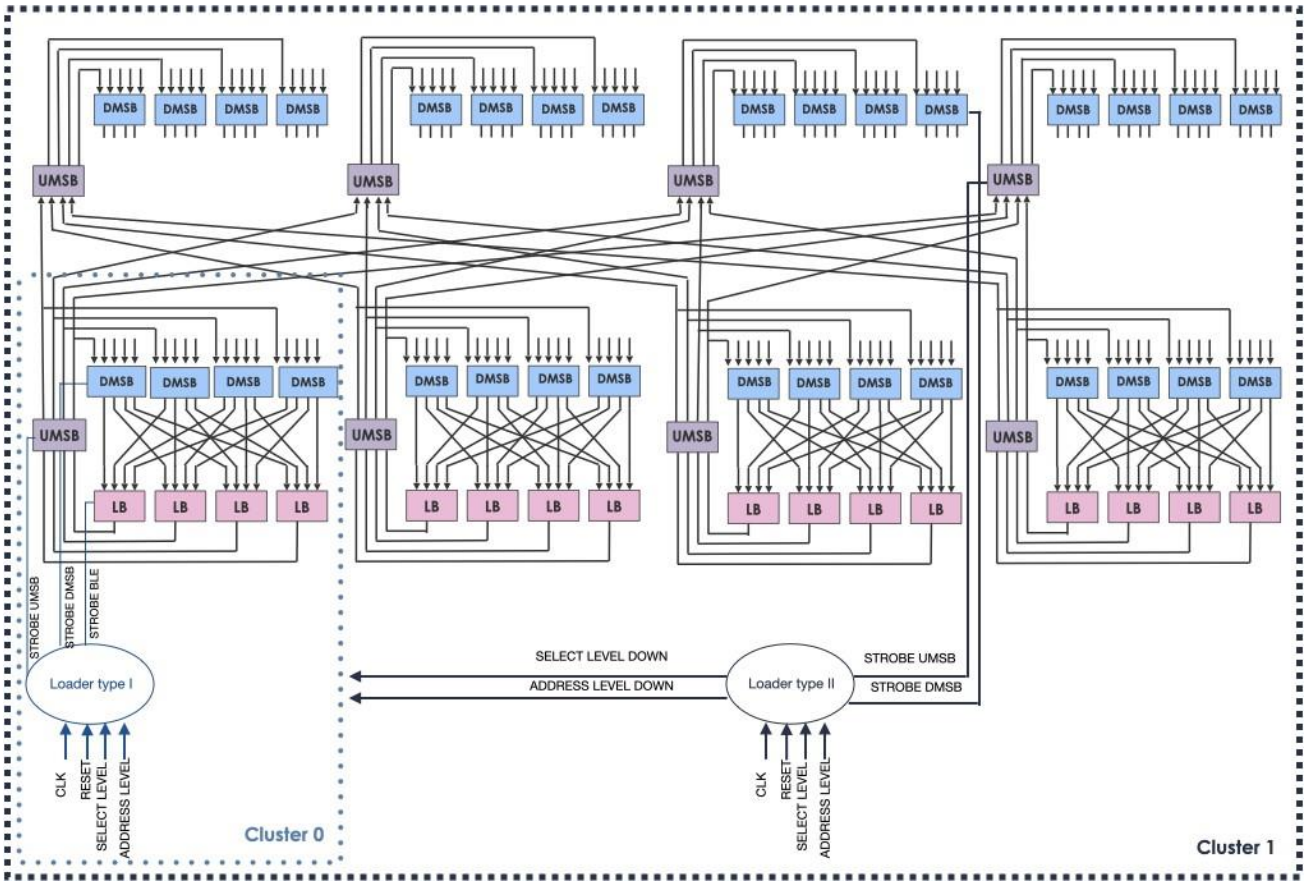


Figure 2.21: Tree of 2 hierarchical levels with their corresponding levels.

For example, for a tree with 3 levels, indexed from top to bottom 2, 1 and 0, since the tree is a hierarchical architecture where a cluster at level 1 has a parent at level 2 and slaves at level 0. Thus, the configuration is established from top to bottom, from level 2 to 1 and finally 0. Each selected cluster, decodes its received address to activate the corresponding strobes of the appropriate element, then it sends the remaining address to its owner.

The address is split into two parts:

1. MSB (Most Significant Bit) indicates an element index.
2. LSB (Less Significant Bit) indicates the configuration of the corresponding element.

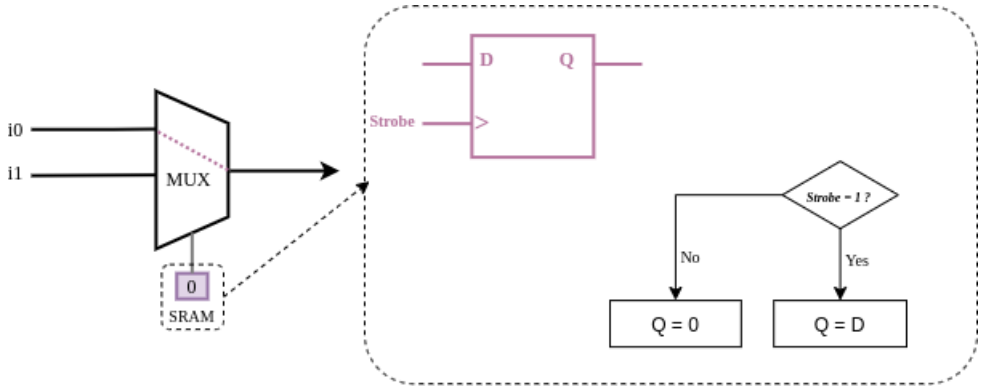
If we are at level 2’s cluster: when its 'loader' selector is at 1, we determine by decoding the address part of the MSB, the element to configure. Indeed, this element can be either a UMSB, a DMSB or one of its level 1 slaves. If the element is a UMSB or a DMSB, then the LSB part is the multiplexers configuration. Otherwise, if the element is one of the cluster slaves, the LSB will be the address that will be sent to the loader type 1 of the selected cluster slave.

2.3.3 Multiplexers Configuration

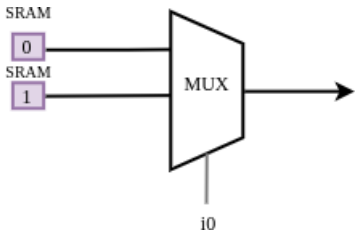
As we explained, strobes are the activators of SRAMs. By decoding the addresses, we move to the dedicated loaders to obtain a fine-grained configuration. Indeed, the multiplexers, as shown in Figure 2.22, are used in 2 different ways, those that implement the LUT and those that interconnect the architecture inside the SBs (DMSBs and UMSBs). For the LUTs, in order to configure their multiplexers, the data is implemented in their SRAM memory cells, which means that the data serves as a truth table for the LUTs. As for the UMSBs and DMSBs, the configuration of their multiplexers allows the link of the whole architecture to be defined and therefore the right input to be activated and linked to the output. Thus, in this case, the data are implemented as multiplexer selectors.

2.3.4 Address width

The calculation of the address width of each element of the architecture is crucial when generating the hardware description files of the embedded FPGA (eFPGA). These parameters must be calculated in the same way in the hardware and software tools, in order to obtain an appropriate configuration, which corresponds to the SB and LB used in both environments.



(a)Switch boxes configuration.



(b)LUT configuration.

Figure 2.22: Configuration in both LUT and SBs.

An address is used to identify an element in the whole architecture, by identifying it, the data enters its memory cells to configure it. For example, if we want to configure a LUT in the Mesh

of Tree that exists in the Tree of the Tile of index "i", we will have to calculate the addresses in order: first the address of the Tile "i", then the Tree, the Tree Slave and then the LUT to get to configure it.

As in Figure 2.22 above, to configure a LB in the tree, the Mesh-of-Tree address must support all the design elements so that we can configure all the existing elements. Thus, the width of the address must support all elements of the architecture. The formulas 2.1 describe in detail how we calculate the address width of all elements to support the entire FPGA configuration.

The address width of a tree mesh depends on the number of tiles and the width of the tile addresses.

$$ADD(MeshOfTree)_{width} = log_2(nb_{Tiles}) + Max_{i=1}^{i=nb\ Tiles} ADD(Tiles)_{i\ width} \quad (2.1)$$

The address width of each tile depends on its number of elements (which are Sb and/or Cluster) and the address width of the two elements which gives the equation 2.2:

$$ADD(Tile)_{width} = log_2(2) + Max(ADD(SB)_{width}, ADD(Cluster(l-1))_{width}) \quad (2.2)$$

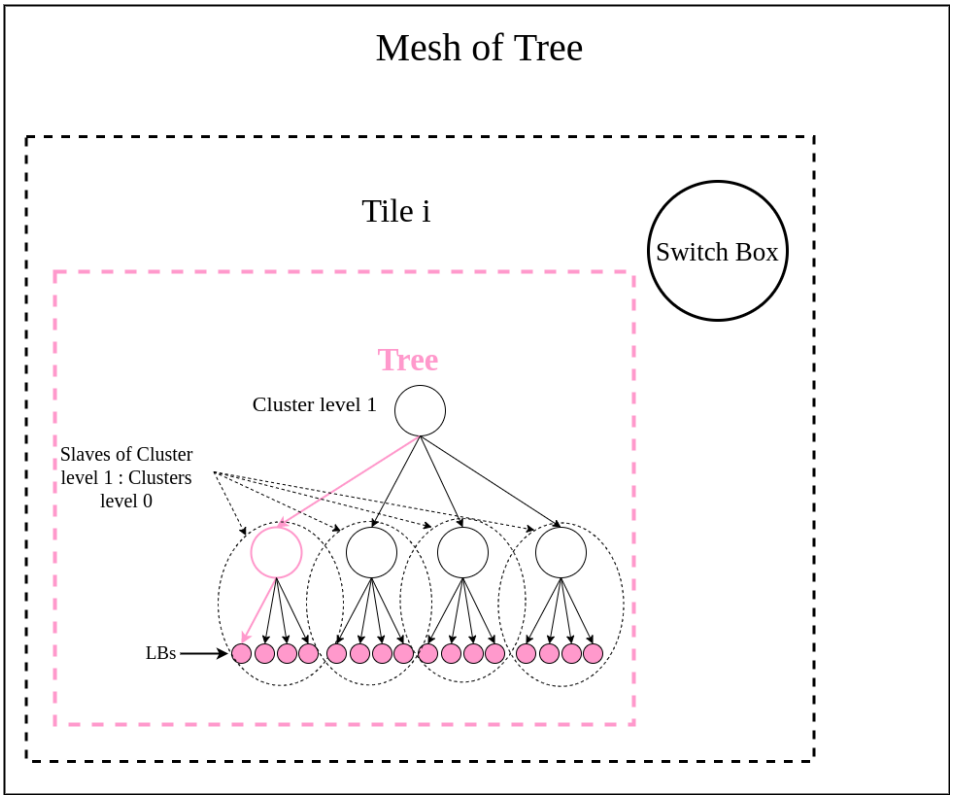


Figure 2.23: Elements to identify to get into a LB Configuration.

Regarding the address width of the SB in the above equation 2.3, an SB contains 3 elements which are the UMSBs, the DMSBs at level 1 and level 2 and therefore the address width of the SB must support the addressing of these elements and their contents.

$$ADD(SB)_{width} = log_2(3) + Max(ADD(DMSBL2)_{width}, ADD(DMSBL1)_{width}, ADD(UMSB)_{width}) \quad (2.3)$$

For a DMSB or an UMSB, its address width parameter depends on the number of inputs and outputs. The equations 2.4, 2.5, 2.6 and 2.7 define the address width for DMSBs and UMSBs. The DATAwidth parameter used is the width of the word:

$$ADD(DMSBs)_{width} = [\log_2\left(nb_{DMSBs} * \frac{nb_{SRAMs}(DMSB)}{DATA_{width}}\right)] \quad (2.4)$$

$$ADD(UMSBs)_{width} = [\log_2\left(nb_{UMSBs} * \frac{nb_{SRAMs}(UMSB)}{DATA_{width}}\right)] \quad (2.5)$$

$$nb_{SRAMs}(DMSB) = \left[log_2\left(DMSB_{nbInputs}\right)\right] * DMSB_{nbOutputs} \quad (2.6)$$

$$nb_{SRAMs}(UMSB) = \left[log_2\left(UMSB_{nbInputs}\right)\right] * UMSB_{nbOutputs} \quad (2.7)$$

As for the "L" level tree that has from "0" to "L-1" levels, the L-1 level cluster that exists encapsulated with an SB inside a tile, its address width is calculated as follows in the following equation 2.8:

$$ADD(Cluster(L-1))_{width} = \log_2(nb_{Slaves} + 2) + Max(ADD(Cluster(L-2))_{width}, ADD(UMSBs)_{width}, ADD(DMSBs)_{width}) \quad (2.8)$$

The latter has nb_{Slaves} clusters of level "L-2" which address width is calculated in the following equation 2.9 and so on until reaching the slaves of level 0.

$$ADD(Cluster(L-2))_{width} = \log_2(nb_{Slaves} + 2) + Max(ADD(Cluster(L=0))_{width}, ADD(UMSBs)_{width}, ADD(DMSBs)_{width}) \quad (2.9$$

The level 0 clusters are those that encapsulate the LBs, hence their address width formula is given in the equation 2.10 below:

$$ADD(Cluster(L=0))_{width} = \log_2(nb_{Slaves} + 2) + Max(ADD(LBs)_{width}, ADD(UMSBs)_{width}, ADD(DMSBs)_{width}) \quad (2.10)$$

A new formula computation is required because they contain LBs. The address width of the LBs is, in fact, determined by the number of inputs and extra SRAM. The following equations 2.11 and 2.12 explain the last equation in more depth.

$$ADD(LBs)_{width} = [\log_2\left(nb_{LBs} * \frac{nb_{SRAMs}(LB)}{DATA_{width}}\right)] \quad (2.11)$$

$$nb_{SRAMs}(LB) = 2^{LUTnbInputs} + 1 \quad (2.12)$$

2.3.5 Loader conceptual overview

Figure2.25 shows the different loader classes implemented and used to configure FPGA components.

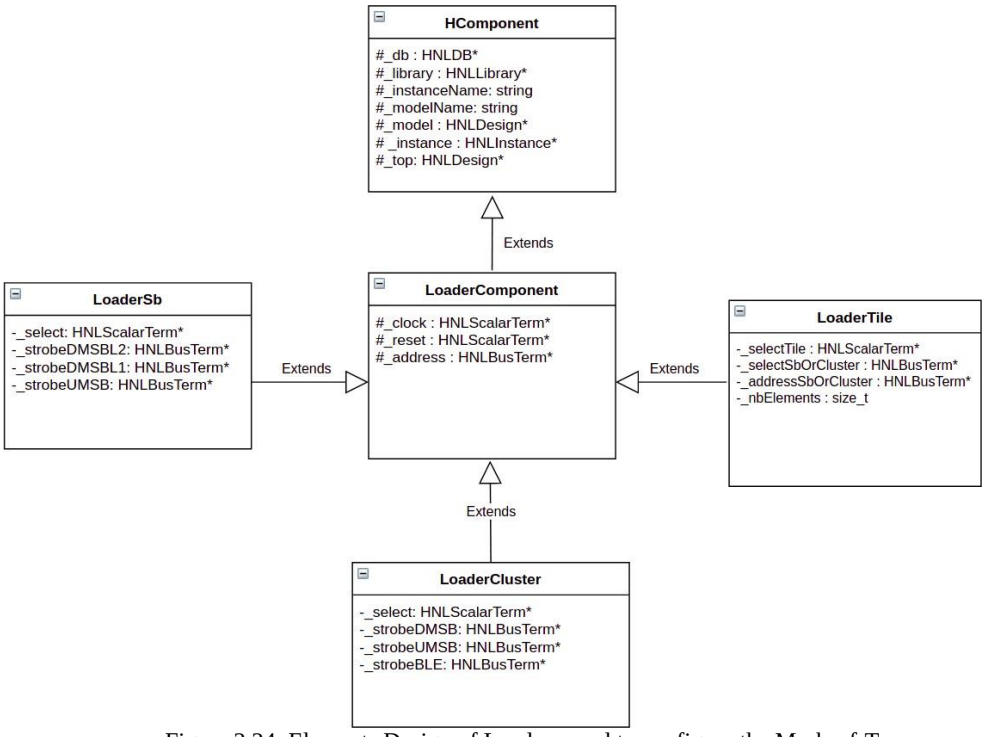


Figure 2.24: Elements Design of Loaders used to configure the Mesh-of-Tree.

In order to make the generated architecture more robust and to automate its validation, we will propose in the next subsection some modifications in the IP generator that can improve it.

2.4 Enhancement of the existing solution

We now go on to enhancing our solution after developing and adding the bitstream loader to our IP generator. We'll start by replacing the old multiplexer by another which is more efficient, then we will adapt the hardware generator code to enable it to produce the testbench automatically.

2.4.1 N:1 Multiplexer design

Because the multiplexer is the most fundamental component of our architecture, it is required to construct it with more optimized multiplexers. Indeed, we will use multiplexers based on 2:1 multiplexers, which will improve the energy efficiency of our approach.

In fact the multiplexer is a combinatorial circuit that selects one of several input signals based on the state of the select line. A 2^N:1 multiplexer with 'N' select lines can select one among 2^N inputs. In other words, the multiplexer connects the output to one of its inputs depending on the value on the select lines. A multiplexer (or commonly called a MUX) is also called a data

selector. Common functions of a multiplexer include concentrating multiple lines of data onto a single line. It can also be used as a data selector or a clock selector.

Multiplexers are defined by the number of inputs. A 2:1 mux, as basic element, has 2 input lines and one selection line. The state of the selection line decides which of the inputs propagates to the output. The truth table of a 2x1 mux is given below. As it shows, when SEL is 1, OUT follows IN2 and when SEL is 0, OUT follows IN1.

SEL	OUT
0	IN1
1	IN2

Table 2.1: Truth table of 2:1 Mux.

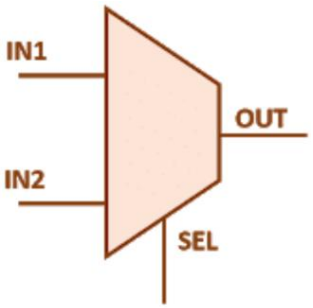


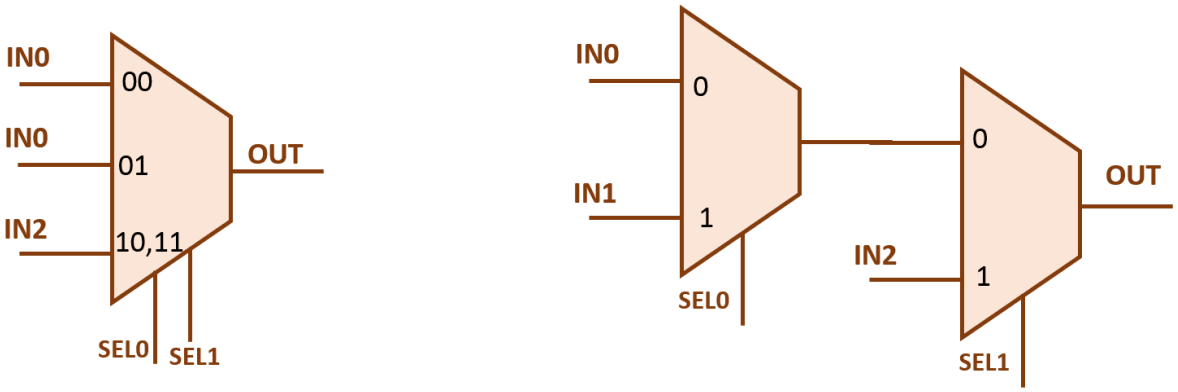
Figure 2.25: 2-input Mux.

A 3-input mux or a 3:1 mux has 2 selection lines and 3 inputs. As a mux with 2 selection lines can represent at most 4 inputs, a 3:1 mux repeats some inputs for 2 combinations. The truth table for a 3 input mux is given in Figure 2.27. For the SEL values "10" and "11", IN2 is selected at the output. This is one of the 3 possible scenarios, repeating IN0 or IN1 is also possible.

SEL	OUT
00	IN0
01	IN1
10	IN2
11	IN2

Table 2.2: Truth table of 3:1 Mux.

The schematic symbol and structural representation (in terms of 2:1 muxes) for a 3:1 mux is shown in Figure 2.28 below. In fact, a 3x1 mux has been constructed using two 2x1 muxes.



(a)Schematic symbol for 3:1 Mux (b) Structural symbol for 3:1 Mux using 2:1 Muxes

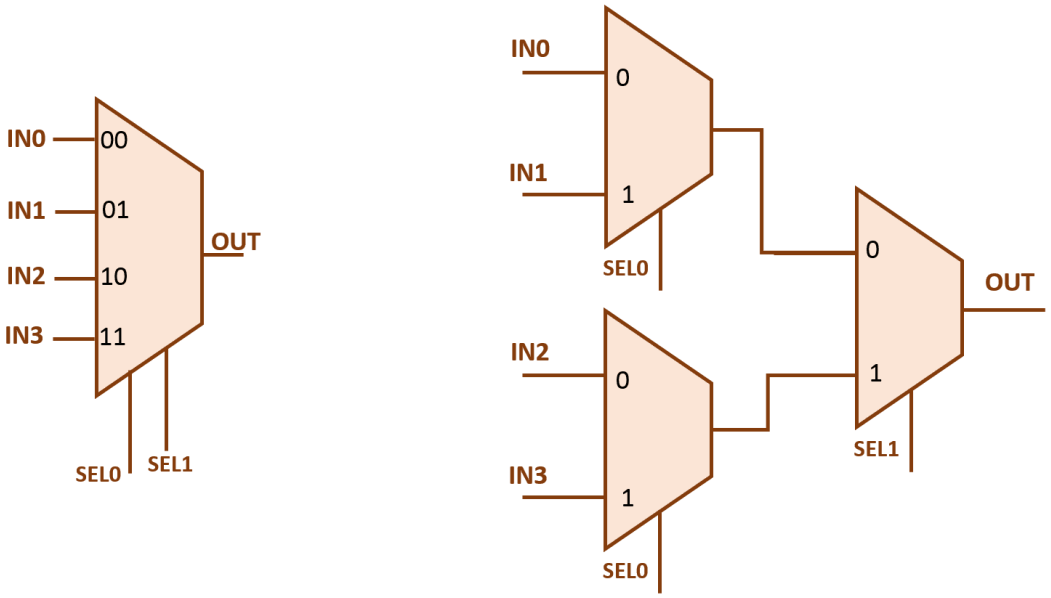
Figure 2.26: 3-input Mux.

A 4-input mux or a 4:1 mux has 2 selection lines and 4 inputs. The truth table for a 4:1 mux is shown below:

SEL	OUT
00	IN0
01	IN1
10	IN2
11	IN3

Table 2.3 : Truth table of 4:1 Mux.

Figure 2.31 below shows the schematic symbol and structural symbol of a 4:1 mux using 2:1 muxes. It is obvious that a 4:1 mux has been constructed from three 2:1 muxes.



(a)Schematic symbol for 4:1 Mux Figure (b) Structural symbol for 4:1 mux using 2:1 Muxes

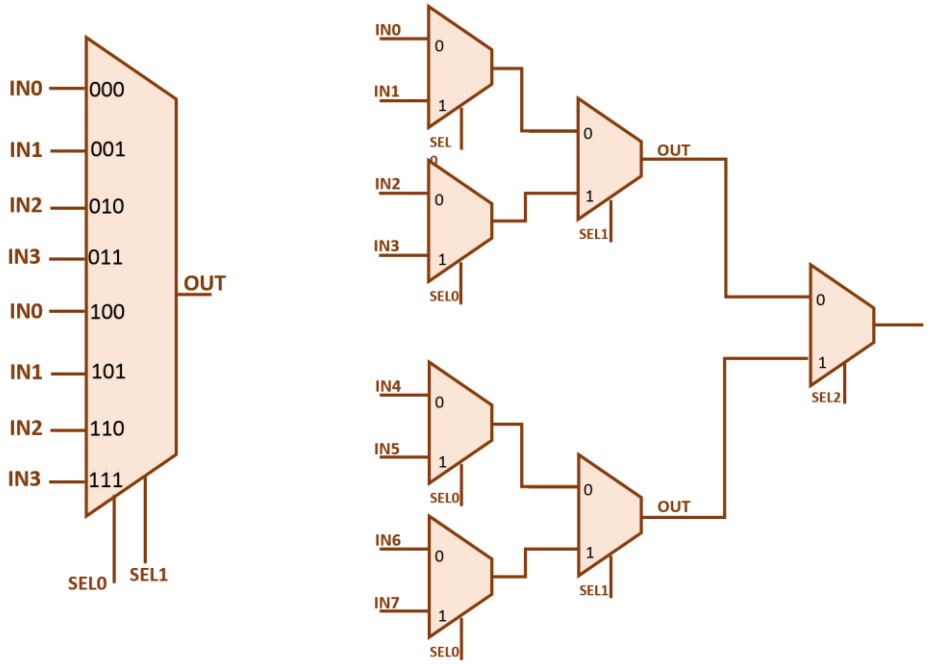
Figure 2.27: 4-input Mux.

An 8-input mux or an 8x1 mux has 3 selection lines and 8 inputs. The truth table for an 8x1 mux is shown below:

SEL	OUT
000	IN0
001	IN1
010	IN2
011	IN3
100	IN4
101	IN5
110	IN6
111	IN7

Table 2.4: Truth table of 8:1 Mux.

The structural representation using 2x1 muxes, and the schematic symbol for the same is as shown below in Figure 2.32. An 8 input mux can be implemented using 7 2x1 muxes.



(a)Schematic symbol for 8x1 mux Figure (b) Structural symbol for 8:1 mux using 2x1 Muxes

Figure 2.28: Symbol of 8:1 Mux.

16-input mux: A 16x1 mux can be implemented from fifteen 2:1 mux. It has 4 selection lines and 16 inputs. The output follows one of the inputs depending on the state of the selection lines.

2.4.2 TestBench Generation

To facilitate and accelerate the testing and validation of the architecture, we pushed our generator to be able to automatically generate a testbench file basing on the descriptive parameters of our architecture such as the size of the input and output blocks and the size of the data and basing on a generic testbench file written in verilog in which we instantiated the Top module of the Mesh Of Tree.

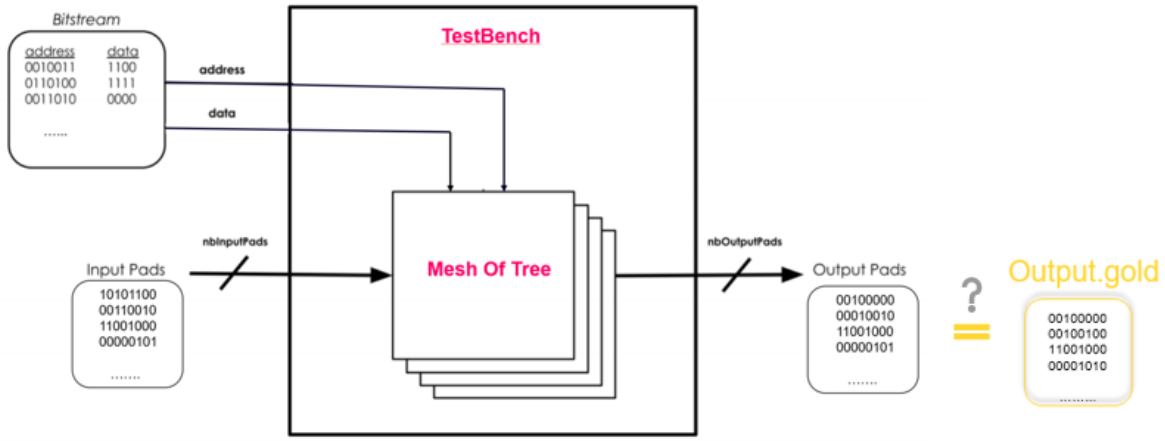


Figure 2.29: TestBench.

In fact, this Verilog TestBench File contains an instantiation of the top-level design entity which is the Mesh-of-Tree Verilog file and reads from the simulation input pads file and writes the output into the simulation output pads file. In addition, this will be compared to another output.gold file that contains the predicted output values.

2.5 Integrating the Mesh-of-Tree into MPSoC

We intend to integrate the FPGA IP as a reconfigurable accelerator into an MPSoC architecture. Indeed, because of this integration, we can investigate the new FPGA architecture as a device or accelerator, and then take advantage of the benefits of both the Mesh of Tree design and the many-core architectures.

2.5.1 Connecting the eFPGA to an AXI interface

In order to integrate our embedded FPGA (eFPGA) into an MPSOC, we must connect it to an AXI interface that will be connected later to openpiton. In fact, we created this AXI slave interface using the verilog IP generator, and it is linked to an AXI bridge, which will act as an intermediary between the AXI interface and our IP. In the table below, the FPGA pin names are mapped to AXI addresses. This AXI Bridge will, in fact, use this mapping table to ensure the transaction between the two modules. And all of these modules are linked using the HNL library.

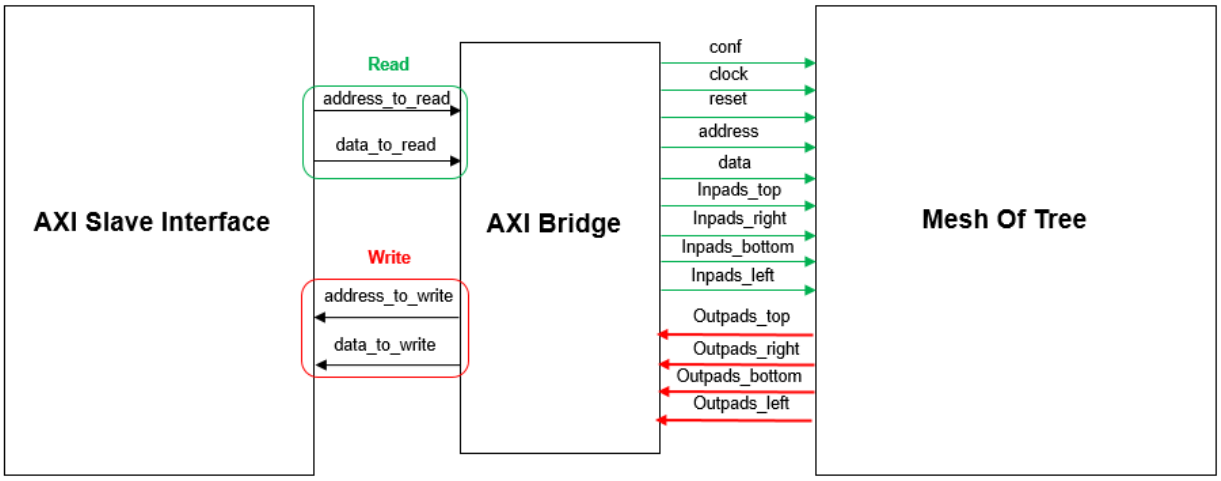


Figure 2.30: AXI Slave interconnection.

address_to read	Reg name	Type
0x00	outpads_top	OUT
0x01	outpads_right	OUT
0x02	outpads_bottom	OUT
0x03	outpads_left	OUT

address_to_write	Reg name	Type
0x00	conf	IN
0x01	clock	IN
0x02	reset	IN
0x03	address	IN
0x04	data	IN
0x05	inpads_top	IN
0x06	inpads_right	IN
0x07	inpads_bottom	IN
0x08	inpads_left	IN

Figure 2.31: Address mapping description.

2.5.2 Connecting the eFPGA to OpenPiton

OpenPiton has been designed as a platform with a high level of configurability and integrity of a many-core processor. It is based on a generator that takes many arguments to generate a specific architecture[24].

To achieve the connection objective between the embedded FPGA and top system of OpenPtion, we are supposed to do some modification in the RTL code of OpenPiton to support the bidirectional communication with the eFPGA,

Indeed, the chipset module contains various peripherals, these peripherals are programmable, and they can be added or removed from the design. In our case, we are testing the eFPGA IP

which was encapsulated into an AXI interface and as OpenPiton does not have an AXI interface and use NOC only as communication protocol between its peripherals, so we had to instantiate a new AXI/NOC Bridge module in the chipset.

As illustrate in the figure 2.36, the NOC/AXI Bridge will be the intermediate between the I/O Crossbar and the eFPGA.

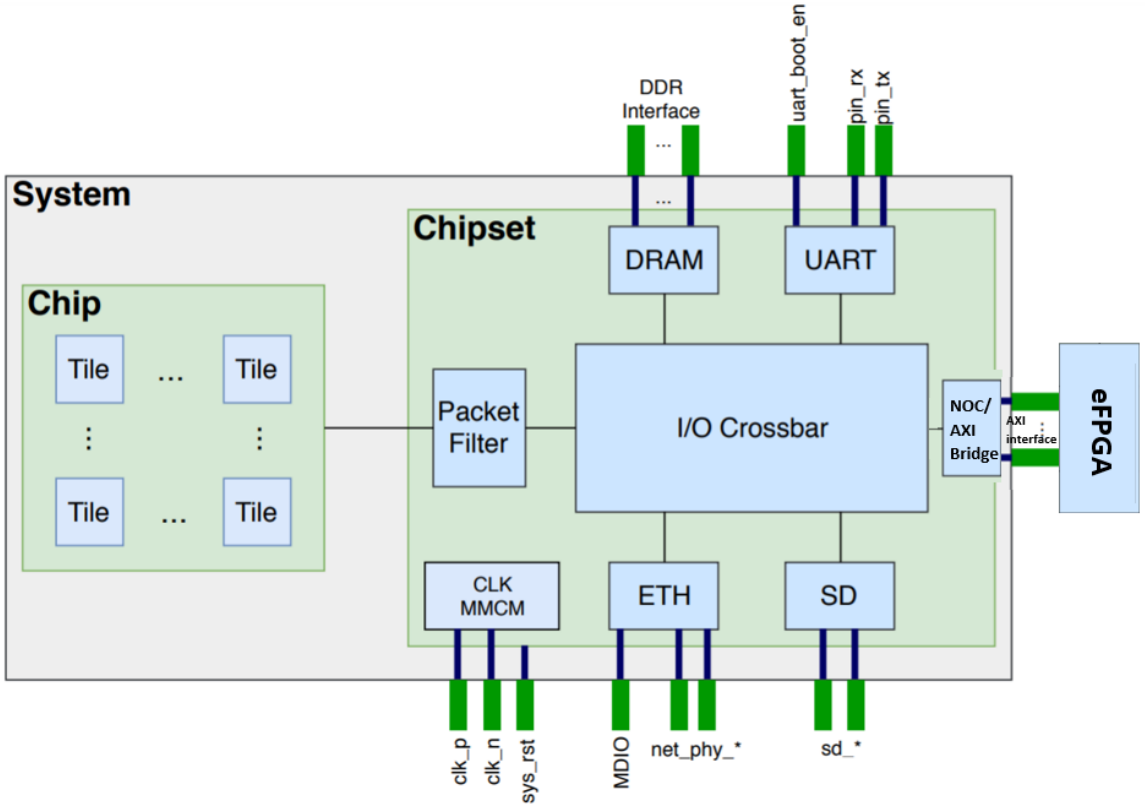


Figure 2.32: OpenPiton prototype architecture with eFPGA.

Following the connection procedure, we will write C code to test this eFPGA by reading the bitstream file, loading its contents into our IP, assigning the input pads, and finally extracting the contents of the output pads via AXI communication. All of this via an AXI communication.

2.6 Conclusion

In this section, we presented the design of the Mesh-of-Tree hypergraphic interconnect. In a second step, we presented the different implemented components used to configure the existing architecture. In a third step, we have presented the improvements made to the existing solution. Finally, we proposed and described some modifications to allow OpenPiton communicate with the eFPGA IP.

CHAPTER 3

Validation of the FPGA IP Generator and integrating it into an MPSoC Architecture

3.1 Introduction

In this chapter, we will first simulate the output of the FPGA IP generator that we created. The results will then be validated using a debugging and simulation flow that will be presented later. Finally, we will propose an integration of the FPGA in the environment of a multiprocessor system on chip.

3.2 Validation of the FPGA Hardware Architecture

This section will concentrate on the validation of the tree architecture's generated mesh.

3.2.1 Generation of the hardware design of the FPGA

Following the change and completion of the Mesh of Tree-based FPGA IP C++ generator, which uses the HNL data structure to dump the FPGA hardware description into Verilog. We will validate our generated files by simulating and checking their behavior.

In fact, as a first step, we provide the generator with a design, as described in the architecture file in Figure 3.1.

```
nx = 2
ny = 2
in_rate = 2
out_rate = 2
network width = 12
nb_levels 2
   .level 0 nb_slaves  4  nb_inputs  16     nb_outputs 4
   .level 1 nb_slaves  4  nb_inputs  64     nb_outputs 16
   .basic ble          nb_inputs    4      nb_outputs 1
   .IOlevel 0
   .word_width  8
```

Figure 3.1: Example of an architecture file.

The generator parses the file and extracts the parameters required to create the Mesh-of-Tree. In Figure 3.2, the generator successfully built and generated all of the FPGA files required to

describe the entire design. In this use case, the design of 2x2 requires the instantiation of nine types of tiles: TileCornerTopRight, TileCornerTopLeft, TileCornerBottomRight, TileCornerBottomLeft, TileBoundaryTop, TileBoundaryBottom, TileBoundaryRight, TileBoundaryLeft, and finally TileInternal. Each tile component will be created, and thus we find SBs files and Tree files because it has 2 levels and thus files of cluster at level 1 and level 0.

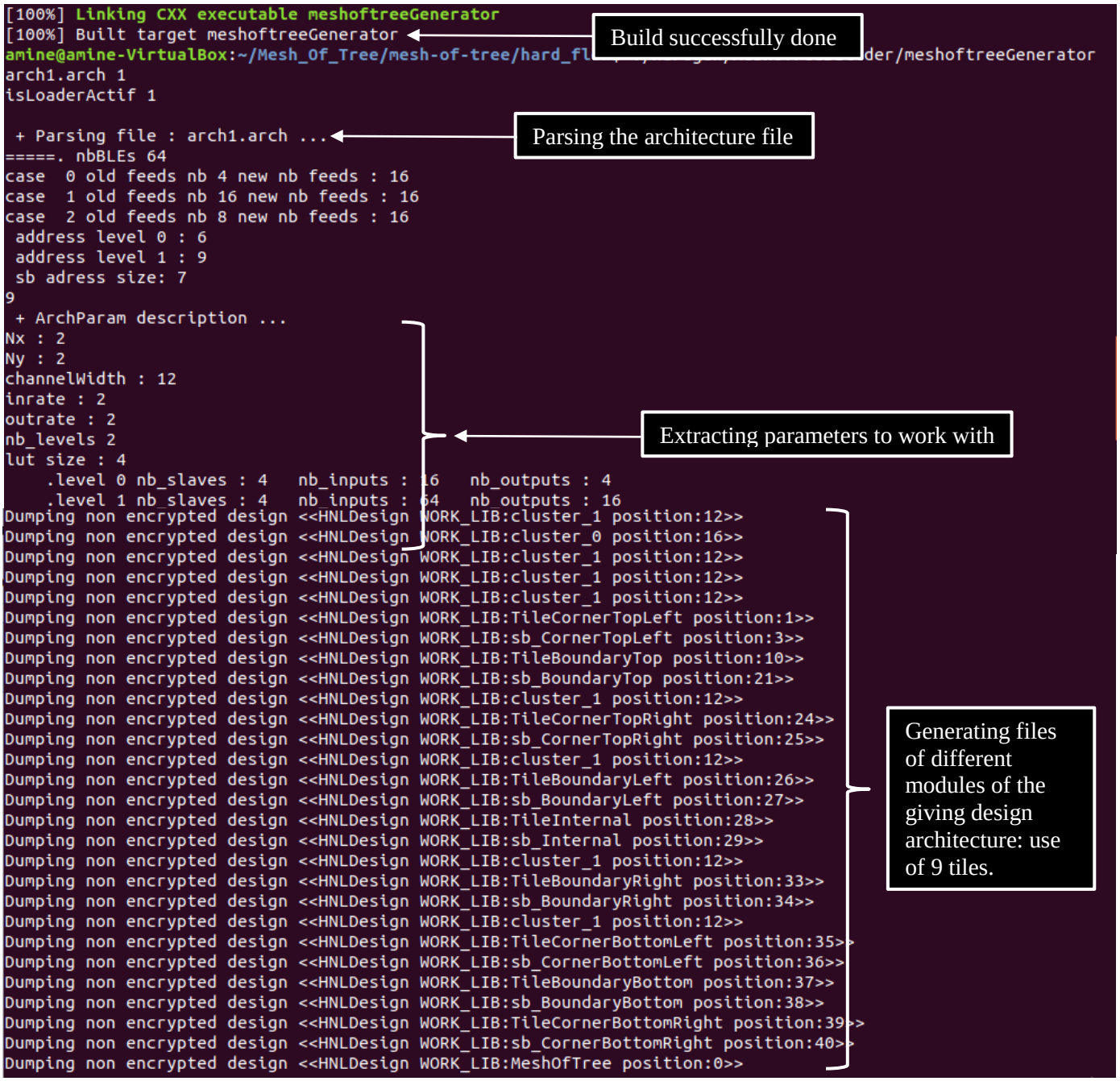


Figure 3.2: Build and hardware design generation of an FPGA architecture.

As for the larger designs (4*4), we have in figure 3.4, the generation of 25 tiles of 9 types distributed like this:

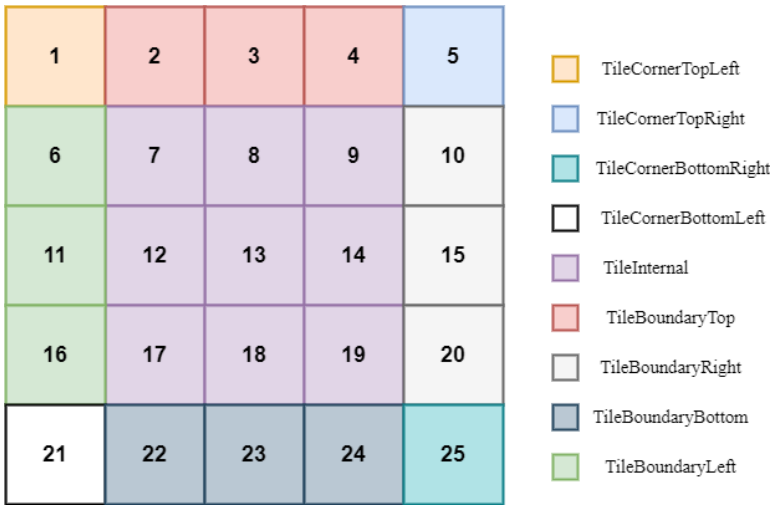


Figure 3.3: Build and hardware design generation of an FPGA architecture 4*4.

In this case we have taken advantage of the scalability of the mesh architecture since we have made the duplication of tiles of the same type.

```
[ 98%] Building CXX object hardgen/meshOTreeBuilder/CMakeFiles/meshoftreeGenerator.dir/RGraphBuilder.cpp.o
[100%] Linking CXX executable meshoftreeGenerator
[100%] Built target meshoftreeGenerator
amine@amine-VirtualBox:~/Mesh_Of_Tree/mesh-of-tree/hard_flow$ ./hardgen/meshOTreeBuilder/meshoftreeGenerator
arch1.arch 1
isLoaderActif 1

 + Parsing file : arch1.arch ...
=====. nbBLEs 256
case  0 old feeds nb 4 new nb feeds : 16
case  1 old feeds nb 16 new nb feeds : 16
case  2 old feeds nb 8 new nb feeds : 16
 address level 0 : 6
 address level 1 : 9
 sb adress size: 7
9
 + ArchParam description ...
Nx : 4
Ny : 4
channelWidth : 12
inrate : 2
outrate : 2
nb_levels 2
lut size : 4
    .level 0 nb_slaves : 4   nb_inputs : 16   nb_outputs : 4
    .level 1 nb_slaves : 4   nb_inputs : 64   nb_outputs : 16
BLE : inputs: 4,outputs : 1
 IO level : 0
```

Parsing architecture file

Figure 3.4: Build and hardware design generation of an FPGA architecture 4x4.

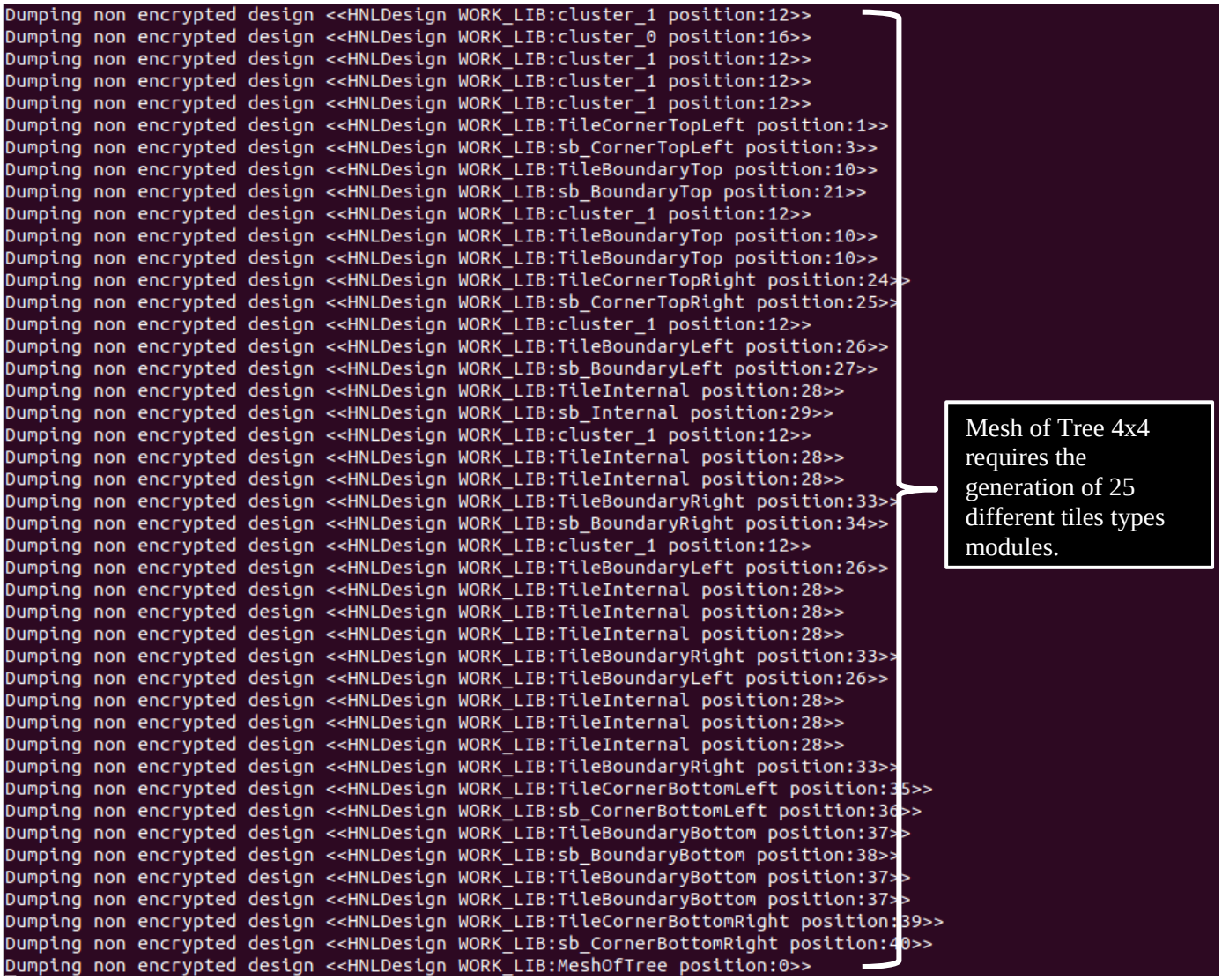


Figure 3.5: Generation of 25 tiles of a 4*4 FPGA architecture.

3.2.2 Design Simulation and Debugging

To validate our generated FPGAs including the existing solution and the changes and improvements made, we need to simulate the whole architecture with the RTL DMSBs, UMSBs, BLEs and Loaders codes. To simulate our testbench, we created a Questasim project that included all of the generated files as well as the primitive files in order to obtain a scope describing the behavior of the architecture's various signals. In fact, Questasim is a multi-language environment by Mentor Graphics, for simulation of hardware description languages.

As illustrated in the figure below, we used this debugging flow to validate the entire project: software and hardware.

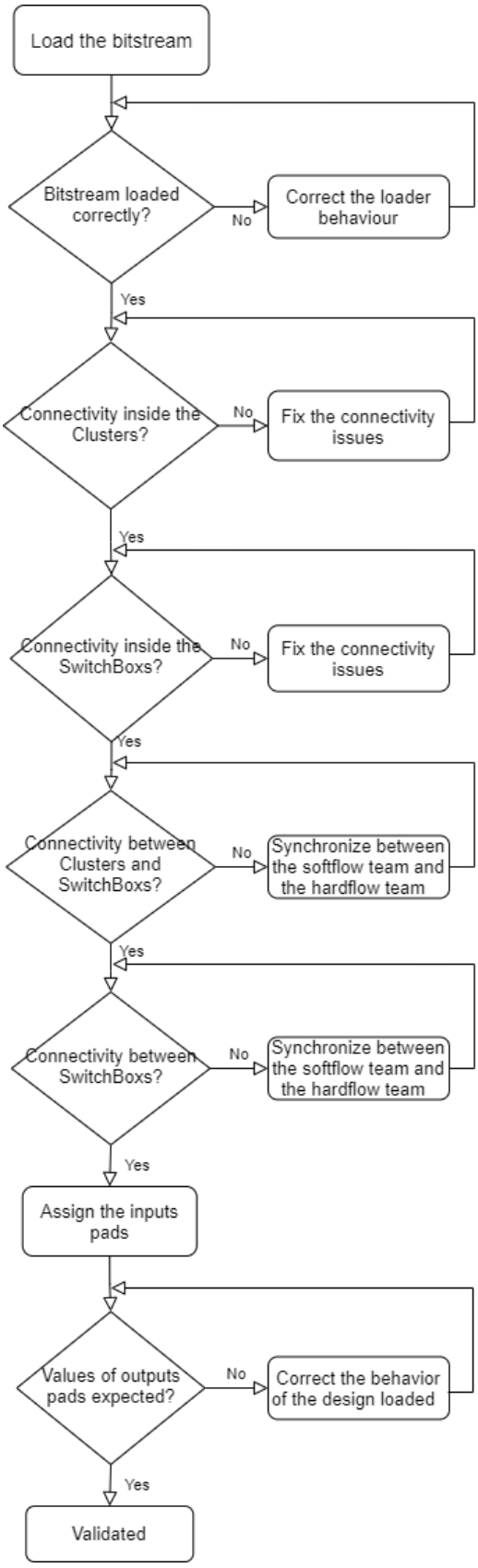


Figure 3.6: Debugging flow.

As a first step, we simulate the developed loader to ensure that the bitstream was correctly loaded into our architecture's SRAMs. To achieve this goal, we use a straightforward procedure that entails inspecting the SRAM's content mentioned in Questasim and confirming that it corresponds to the data specified in the bitstream description file that correspond to this SRAM address. It should be noted that the bitstream description file, as mentioned in Figure 3.7, is a file created by the software team that describes the content of the SRAMs and their location in the FPGA.

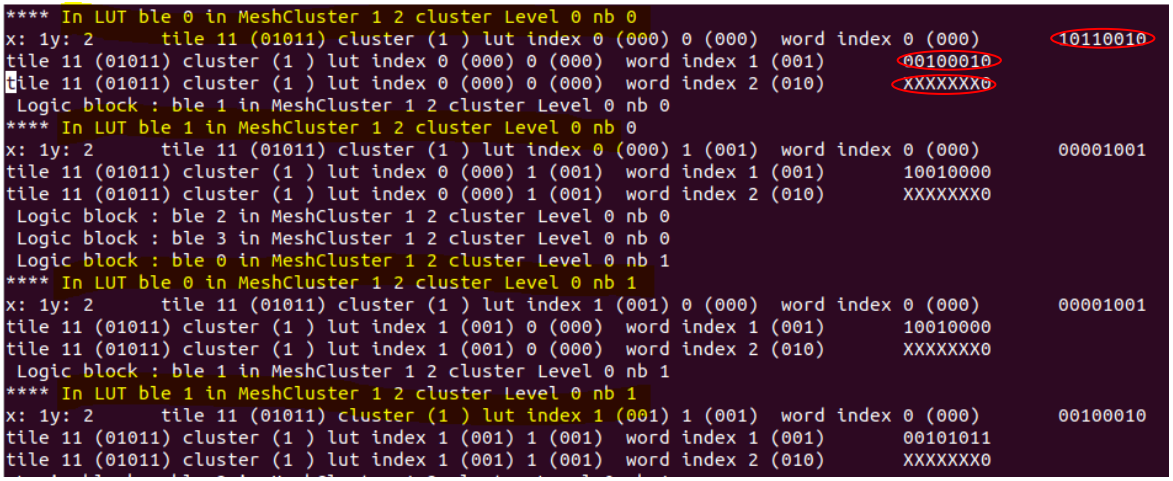


Figure 3.7: Bitstream description file example.

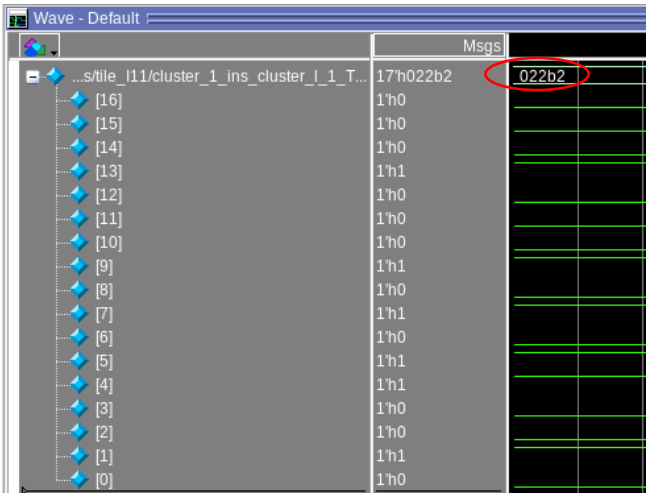


Figure 3.8: Example of the content of the SRAMs of a ble.

Take the SRAMs of a BLE as an example; as shown in the two figures 3.7 and 3.8 above, we checked the equality of the two values circled in red, one in simulation and the other in the bitstream description file. This procedure allows us to check the loader's accuracy.

In the second step, we examine the connectivity within the clusters, specifically between its sub-clusters, UMSB, DMSB, and BLE. To do this, we must examine the routing paths in the routing.log file, then go into the simulation and force the entry of this path, and see if we get the same result at the end of the path to validate the connectivity.

In a third step, we check the connectivity within the switchboxs between the DMSBs of the two levels, as well as between the UMSB and all of the DMSBs. As a result of the investigation, we discovered an error in the link between the UMSB feedback and the level 2 DMSB inputs.

The fourth step is to double-check the connections between the clusters and the switchboxs, as well as between switchboxs. During this stage, we discovered numerous disagreements and mismatches between the hardware and software teams regarding the link between the switchbox and the surrounding clusters, specifically the order of the connection pins between the switchbox outputs and the inputs of the four surrounding clusters. On the other hand, we have synchronized the connections order of the inpads and outpads between the softflow and hardflow codes.

All of these steps of connectivity verification necessitate checking in parallel the driver and destinations of the nets in the router.log file, then go and force the driver of the net pads in the simulation to "1" and then check if we have the same result in the destination net pads as shown in the 2 figures below.

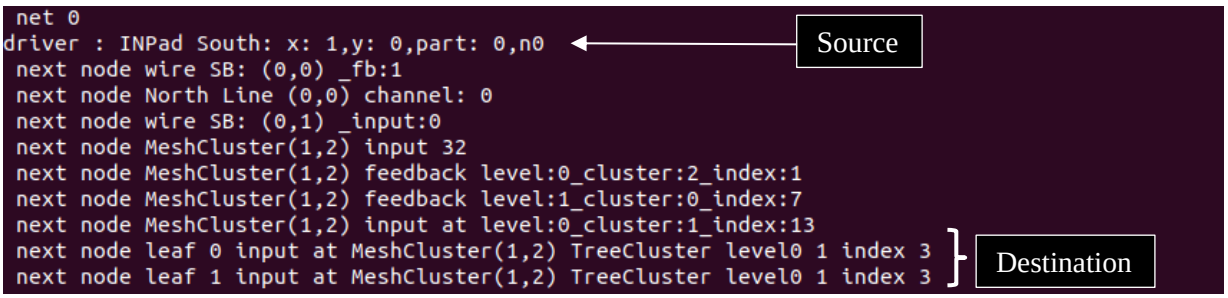


Figure 3.9: Example of a net in the router.log file.

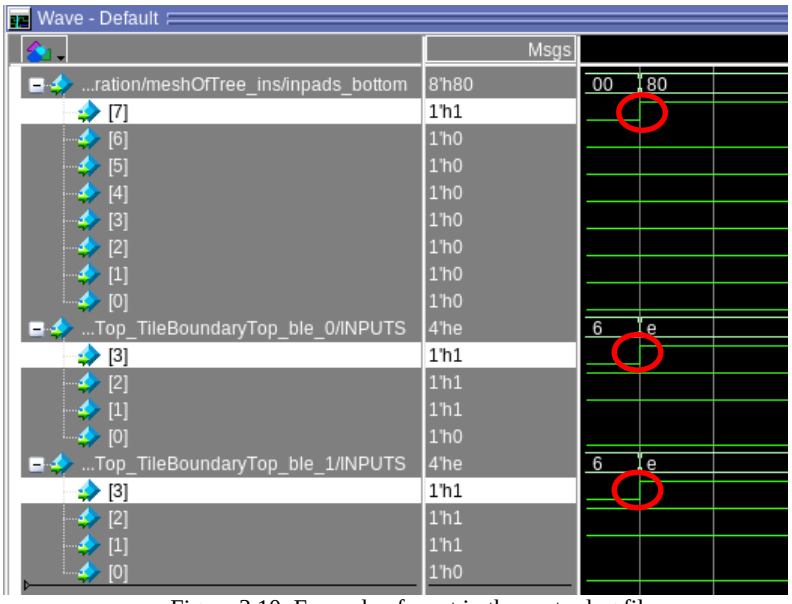


Figure 3.10: Example of a net in the router.log file.

As a final step, we verified the correct operation of the design loaded in the architecture. To perform this step, we read an input file and assign the corresponding values to the corresponding input blocks and after that we check the content of the output blocks and verify if it matches the expected values mentioned in the output.gold file.

As mentioned in the previous chapter and in order to automate the simulation process, we added a method to our generator code that allows it to generate a Verilog testbench from the initial architecture description parameters and a GenericTestBench file. This testbench reads the bitstream file and loads its contents into the Mesh Of Tree instance, then reads the input file and assigns each data to the appropriate input, and finally writes the outputs into an output.txt file.

Donload bitstream

```
//download bitstream
initial
begin
   $readmemb("bitstream.final",read_bitstream);
   j=0;
   for(i=0; i<size_bitstream*2; i=i+2)
   begin
        read_address[j] = read_bitstream[i];
        read_data[j] = read_bitstream[i+1] [data_size-1:0];
        j=j+1;
   end
   assign reset = 1'b1;
   assign configure =1'b0;
   for (i=0; i<size_bitstream ; i=i+1)
   begin
        address = read_address[i];
        data = read_data[i];
        #160;
    $display(address);
    $display(data);
   end
   $display("******************** bitstream loaded successfully
********************");
```

Read the inputs

```
  assign configure =1'b1;
  $readmemb("input.final",read_input);
  for (i=0; i<test_size ; i=i+1)
  begin
  in_t <= read_input[(i*4)+0][inTB_size-1:0];
  in_r <= read_input[(i*4)+1][inRL_size-1:0];
  in_b <= read_input[(i*4)+2][inTB_size-1:0];
  in_l <= read_input[(i*4)+3][inRL_size-1:0];
```

Write the outputs

```
#400;
$display(out_t);
$display(out_r);
$display(out_b);
$display(out_l);
read_output[(i*4)+0][outTB_size-1:0] = out_t;
read_output[(i*4)+1][outRL_size-1:0] = out_r;
read_output[(i*4)+2][outTB_size-1:0] = out_b;
read_output[(i*4)+3][outRL_size-1:0] = out_l;


$writememb("output.txt", read_output);
end
```

Mesh Of Tree instantiation

```
MeshOfTree  MOT
( .clock(clk), .conf(conf), .reset(reset), .address(address), .data(data), .inpads_top(in_t), .inpads_right(in_r), .inpads_bottom(in_b),
 .inpads_left(in_l), .outpads_top(out_t), .outpads_right(out_r), .outpads_bottom(out_b), .outpads_left(out_l));
```

Figure 3.11: TestBench.

Following the generation of the Verilog files, such as the Verilog architecture file and the testbench file, we have a run_simulation.tcl script that will automatically create a questasim simulation project that adds into it all the files required for the simulation. All this process is taken over in a shell script, which will run the tcl file and finally compare the generated output.txt file to the output.gold file that contains the predicted contents of toutputs pads.

```
#! /bin/sh
vsim -do run_simulation.tcl
file1="output.txt"
file2="output.gold"

if cmp -s "$file1" "$file2"; then
    printf 'The file "%s" is the same as "%s"\n' "$file1" "$file2"
    printf 'Validated Design\n'
else
    printf 'The file "%s" is different from "%s"\n' "$file1" "$file2"
    printf 'Design not validated\n'

fi
```

Figure 3.12: Shell script.

```
amine@amine-VirtualBox:~/mesh-of-tree/hard_flow$ ./simulation.sh
The file "output.txt" is the same as "output.gold"
Validated Design
```

Figure 3.13: Result of comparison.

3.3 FPGA Integration into MPSOC

As mentioned in the previous chapter, we intend to integrate the FPGA IP into an MPSoC architecture as a configurable accelerator to more explore the new FPGA architecture as a device or as an accelerator and to exploit the advantageous of both the Mesh_of_Tree design and the many-cores architectures. In fact, in this section, we will create the AXI slave interface that will communicate with the AXI master, which in our case will be openpiton.

3.3.1 AXI slave interface generation

As described in the previous chapter, we have developed using HNL a code that generates an AXI slave interface that instantiates an encapsulated AXI bridge that is based on a generic and parametric AXI bridge model and contains, on the other hand, an instance of Mesh_of_Tree, and these two are automatically linked together through an HNL interconnection.

In fact, the generic AXI bridge is a generic model that will generate an address mapper based on initial parameters, in order to assign data to the corresponding pins or read data from a specific pins. A simple description of the transaction specification is described below:

- First of all, before any transmission, master and slave must share signals of handshake (ready and valid signals).

- Transmission of control signals, address and data begin.
- Write and read signals to handle their corresponding operations.

Write operation

```
//write operation
        always @(posedge aclk)
        begin
            $monitor("posedge clock in write");
                if(awready == 1 && awready_rg == 1)
                begin
                    $monitor("awready equal to 1");
                        awready_rg = 1'b0;
                        wready_rg = 1'b0;
                        if(awvalid && wvalid)
                        begin
                            $monitor("WRITe");
                            strobes[DATA_WIDTH-1:0] = convert_strobe(wstrb);
                                dataToWrite[DATA_WIDTH-1:0] <= wdata & convert_strobe(wstrb);
                                //$monitor("WRITe 1");
                                #5;
                                case (decimal(awaddr))
                                0: configure  <= dataToWrite[0];
                                1: clock <= dataToWrite[0];
                                2: reset <= dataToWrite[0];
                                3: address       <= dataToWrite[0 +: address_size];
                                4: data <= dataToWrite[0 +: data_size];
                                5: inpads_top <=  dataToWrite[0 +: inpad_size];
                                6: inpads_right <=  dataToWrite[0 +: inpad_size];
                                7: inpads_bottom        <=  dataToWrite[0 +: inpad_size];
                                8: inpads_left  <=  dataToWrite[0 +: inpad_size];
                            default: $monitor("Do nothing");
                            endcase
                            if(decimal(awaddr) < 9)
                            begin
                                bresp_rg <= 2'b11;
                                bvalid_rg <= 1'b1;
                            end
                        end

                        awready_rg = 1'b1;
                        wready_rg = 1'b1;
                end
        end
```

Read operation

```
//read operation
        always @(posedge aclk)
        begin
            $monitor("posedge clock in read");
                if(arready_rg)
                begin
                        arready_rg = 1'b0;
                        rvalid_rg = 1'b0;
                        rresp_rg = 1'b0;

                        if(arvalid && rready)
                begin
                    case (decimal(araddr))
                        0: rdata_rg[0 +: outpad_size] = outpads_top;
                        1: rdata_rg[0 +: outpad_size] = outpads_right;
                        2: rdata_rg[0 +: outpad_size] = outpads_bottom;
                        3: rdata_rg[0 +: outpad_size] = outpads_left;
                        default: rdata_rg = 32'h00000000;
                    endcase
                    if(decimal(araddr) > 4)
                        rresp_rg = 1'b0;
                else
                        rresp_rg = 1'b1;
                end

                    arready_rg = 1'b1;
                rvalid_rg = 1'b1;
            end
        end
```

Figure 3.14: Write and read operation mapping.

Figure 3.14 shows the two sections of code that are responsible for mapping between the address and each corresponding input or output.

For example, as shown in Figure 3.14, when the master requests a read operation of address_to_read araddr = 0x00, the slave receives it and maps it to the FPGA's outpads_top and sends the value of the outpads_top register back to the master in rdata (data_to_read). As for write operations, the master will send both the awaddr (address_to_write) and the rdata (data_to_write). If the requested awaddr = 0x00 then the slave will write wdata to the FPGA input conf.

3.3.2 Simulation of the eFPGA within the AXI Slave Interface

On the other hand, we are going to configure the FPGA by configuring its SRAMs and input pads registers using the AXI protocol, so we have implemented the write operation. On the other hand, we have implemented the read operation to read the FPGA's output from its output pads.

3.3.2.1 Simulation of the write operation

In order to simulate the AXI protocol write operation, we simulated the handshake between the slave and the master. Indeed, the master sends a valid address signal to announce that the address_to_be_written (named awaddr in the simulation) is a new valid address (named awvalid) as in Figure 3.15.

As for the slave, it sends a ready signal to the master to announce that it can receive a new address (named awready) as in Figure 3.15.

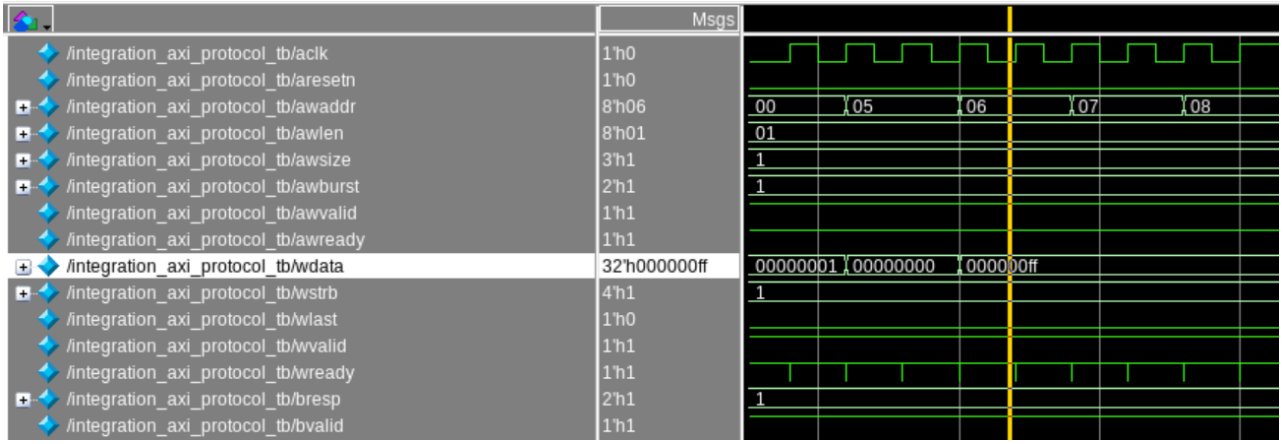


Figure 3.15: Simulating the write operation.

The Mesh-of-Tree instance is successfully configured, as shown in Figure 3.16, and the data to write is successfully written to the corresponding address.

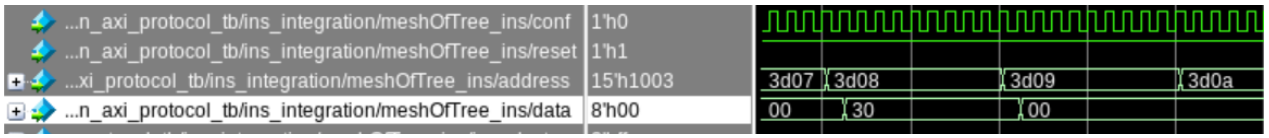


Figure 3.16: Mesh-of-Tree instance configuration.

As shown in Figure 3.17, and after configuring the FPGA, we have assigned the corresponding value to each of its inpads. In fact, each of these inpads is addressed by specific address as mentioned in Figure 2.36.



Figure 3.17: Mesh-of-Tree inpads assignment.

3.3.2.2 Simulation of the read operation

In order to simulate the read operation of the AXI protocol, we have simulated handshake between the slave and the master. Indeed, the master sends a valid address signal to announce that the address_to_read (named araddr in the simulation) is a new valid address (named arvalid in Figure 3.18).

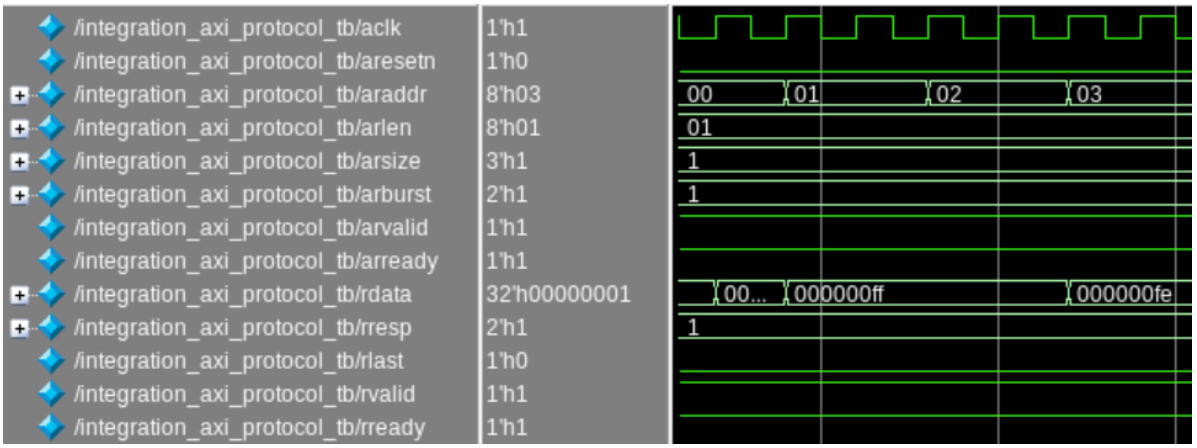


Figure 3.18: Simulating the read operation.

As for the slave, it sends an availability signal to the master to announce that it can receive a new address (called arready in Figure 3.18).

Thus, as we can see in the figure 3.19, the data read have been successfully read from its corresponding address.

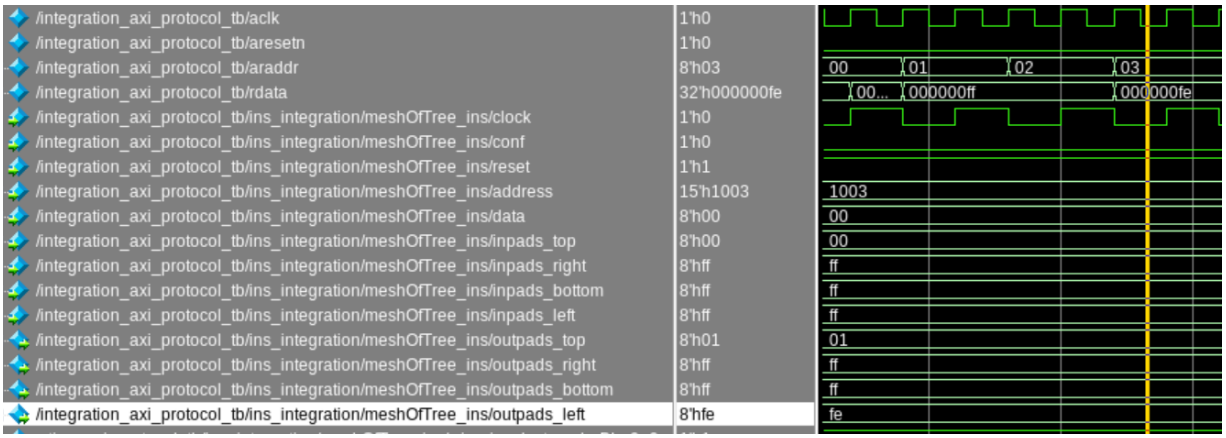


Figure 3.19: Read Data from outpads.

3.3.3 Connection of the eFPGA to openpiton

After we have prepared and validated the AXI Slave interface, we must now connect it to an AXI master, which in our case is openpiton. So, in order to attend this object, we will propose some changes to the openpiton architecture that will allow it to communicate with our eFPGA via an AXI protocol.

3.3.3.1 Add the eFPGA as a connected device to I/O Crosssbar

The I/O crossbar handles routing of NoC requests in the chipset. All I/O devices and the chip are given a port on the crossbar. Each device (and by extension) its port is associated with a range of memory addresses. Devices connected to the crossbar that send requests on the NoC are also given a packet filter, which edits the destination port in the header flit, so the crossbar can correctly route the packet to the right port. So seen that we would add another device to our I/O crossbar, an entry has to be added to the XML configuration file devices.xml which is the base for the dynamic generation of crossbar ports, associated wires, and packet filter logic that maps memory addresses to ports. Adding a new XML entry with the appropriate fields of the new device means specifying a name for it in the name tag and defining its memory region.

```
    <port>
        <!-- this is the shared bootrom (contains DTB that is generated for the current platform config) -->
        <name>ariane_bootrom</name>
        <base>0xfff1010000</base>
        <length>0x10000</length>
        <stream_accessible/>
    </port>
    <port>
        <!-- core local interrupt controller with real-time counter. this is shared among all cores -->
        <name>ariane_clint</name>
        <base>0xfff1020000</base>
        <length>0xc0000</length>
        <stream_accessible/>
    </port>
    <!-- platform level interrupt controller -->
    <port>
        <name>ariane_plic</name>
        <base>0xfff1100000</base>
        <length>0x4000000</length>
        <stream_accessible/>
    </port>
    <!-- end ariane specific periphs -->
    <!-- Mesh Of Tree specific periphs -->
    <port>
        <name>MOT</name>
        <base>0xfff5100000</base>
        <length>0xb00000</length>
    </port>
    <!-- end Mesh Of Tree specific periphs -->
</devices>
```

Figure 3.20: Changes in the XML file.

As shown in the figure 3.20, we chose MOT as the name of our device and 0xfff5100000 as base memory address that is used by the packet filter in conjunction with the length tag of 0xb00000 for the routing logic.

As previously stated, the change in the XML file affects the configuration of the module io_xbar, so when generating the module chipset.impl, we will see new instantiations and new wires that are linked to the newly added MOT device. Figure 3.21 depicts the changes observed in the chipset.impl module, which will be instantiated in the chipset module later. In fact, we notice the addition of new pins in the io_bar_noc2 and io_xbar_noc3 crossbars, which are related to the new Mesh of Tree device.

Associated wires

```
wire [`DATA_WIDTH-1:0] MOT_buf_xbar_noc2_data;
wire                   MOT_buf_xbar_noc2_valid;
wire                   MOT_buf_xbar_noc2_yummy;
wire [`DATA_WIDTH-1:0] xbar_buf_MOT_noc2_data;
wire                   xbar_buf_MOT_noc2_valid;
wire                   xbar_buf_MOT_noc2_yummy;

wire [`DATA_WIDTH-1:0] buf_MOT_noc2_data;
wire                   buf_MOT_noc2_valid;
wire                   MOT_buf_noc2_ready;

wire [`DATA_WIDTH-1:0] MOT_buf_noc2_data;
wire                   MOT_buf_noc2_valid;
wire                   buf_MOT_noc2_ready;


wire [`DATA_WIDTH-1:0] MOT_buf_xbar_noc3_data;
wire                   MOT_buf_xbar_noc3_valid;
wire                   MOT_buf_xbar_noc3_yummy;
wire [`DATA_WIDTH-1:0] xbar_buf_MOT_noc3_data;
wire                   xbar_buf_MOT_noc3_valid;
wire                   xbar_buf_MOT_noc3_yummy;

wire [`DATA_WIDTH-1:0] buf_MOT_noc3_data;
wire                   buf_MOT_noc3_valid;
wire                   MOT_buf_noc3_ready;

wire [`DATA_WIDTH-1:0] MOT_buf_noc3_data;
wire                   MOT_buf_noc3_valid;
wire                   buf_MOT_noc3_ready;
```

io_xbar_noc2

```
io_xbar_top_wrap io_xbar_noc2 (
    .clk                (chipset_clk),
    .reset_in           (~chipset_rst_n),

    .myChipID                   (14'b10000000000000),    // the first chip
    .myLocX                     (8'b0),  // not used
    .myLocY                     (8'b0),  // not used

    .dataIn_0(chip_buf_xbar_noc2_data),
    .validIn_0(chip_buf_xbar_noc2_valid),
    .yummyIn_0(chip_buf_xbar_noc2_yummy),
    .dataOut_0(xbar_buf_chip_noc2_data),
    .validOut_0(xbar_buf_chip_noc2_valid),
    .yummyOut_0(xbar_buf_chip_noc2_yummy),

                                        .
                                        .
                                        .
                                        .
                                        .

    .dataIn_8(MOT_buf_xbar_noc2_data),
    .validIn_8(MOT_buf_xbar_noc2_valid),
    .yummyIn_8(MOT_buf_xbar_noc2_yummy),
    .dataOut_8(xbar_buf_MOT_noc2_data),
    .validOut_8(xbar_buf_MOT_noc2_valid),
    .yummyOut_8(xbar_buf_MOT_noc2_yummy)
);
```

io_xbar_noc3

```
io_xbar_top_wrap io_xbar_noc3 (
    .clk                (chipset_clk),
    .reset_in           (~chipset_rst_n),

    .myChipID                   (14'b10000000000000),    // the first chip
    .myLocX                     (8'b0),  // not used
    .myLocY                     (8'b0),  // not used

    .dataIn_0(chip_buf_xbar_noc3_data),
    .validIn_0(chip_buf_xbar_noc3_valid),
    .yummyIn_0(chip_buf_xbar_noc3_yummy),
    .dataOut_0(xbar_buf_chip_noc3_data),
    .validOut_0(xbar_buf_chip_noc3_valid),
    .yummyOut_0(xbar_buf_chip_noc3_yummy),
                                        .
                                        .
                                        .
     .dataIn_8(MOT_buf_xbar_noc3_data),
     .validIn_8(MOT_buf_xbar_noc3_valid),
     .yummyIn_8(MOT_buf_xbar_noc3_yummy),
     .dataOut_8(xbar_buf_MOT_noc3_data),
     .validOut_8(xbar_buf_MOT_noc3_valid),
     .yummyOut_8(xbar_buf_MOT_noc3_yummy),
     .thanksIn_8()
);
```

Figure 3.21: Generation of crossbar ports and associated wires.

Following the addition of this new MOT device, other new instantiation modules are noticed in the chipset impl module, which are noc2_MOT_to_xbar, noc2_xbar_to_MOT, noc3_MOT_to_xbar and noc3_xbar_to_MOT which are a val_to_credit/credit_to_val based modules that converts off chip val/rdy NOC interface to a credit based interface. Since the original chip design has a credit based interface, another converter from a credit based to val/rdy interface is added for compatibility (c.f Figure3.22).

noc2_xbar_to_MOT

```
credit_to_valrdy noc2_xbar_to_MOT(
      .clk(chipset_clk),
      .reset(~chipset_rst_n),

      .data_in(xbar_buf_MOT_noc2_data),
      .valid_in(xbar_buf_MOT_noc2_valid),
      .yummy_in(MOT_buf_xbar_noc2_yummy),

      .data_out(buf_MOT_noc2_data),           // Data
      .valid_out(buf_MOT_noc2_valid),       // Val signal from dynamic network to processor
      .ready_out(MOT_buf_noc2_ready)    // Rdy signal from processor to dynamic network
);
```

noc2_MOT_to_xbar

```
valrdy_to_credit noc2_MOT_to_xbar (
      .clk(chipset_clk),
      .reset(~chipset_rst_n),

      .data_in(MOT_buf_noc2_data),
      .valid_in(MOT_buf_noc2_valid),
      .ready_in(buf_MOT_noc2_ready),

      .data_out(MOT_buf_xbar_noc2_data),           // Data
      .valid_out(MOT_buf_xbar_noc2_valid),       // Val signal
      .yummy_out(xbar_buf_MOT_noc2_yummy)    // Yummy signal
);
```

noc3_xbar_to_MOT

```
credit_to_valrdy noc3_xbar_to_MOT(
      .clk(chipset_clk),
      .reset(~chipset_rst_n),

      .data_in(xbar_buf_MOT_noc3_data),
      .valid_in(xbar_buf_MOT_noc3_valid),
      .yummy_in(MOT_buf_xbar_noc3_yummy),

      .data_out(buf_MOT_noc3_data),           // Data
      .valid_out(buf_MOT_noc3_valid),       // Val signal from dynamic network to processor
      .ready_out(MOT_buf_noc3_ready)    // Rdy signal from processor to dynamic network
);
```

noc3_MOT_to_xbar

```
valrdy_to_credit noc3_MOT_to_xbar (
      .clk(chipset_clk),
      .reset(~chipset_rst_n),

      .data_in(MOT_buf_noc3_data),
      .valid_in(MOT_buf_noc3_valid),
      .ready_in(buf_MOT_noc3_ready),

      .data_out(MOT_buf_xbar_noc3_data),           // Data
      .valid_out(MOT_buf_xbar_noc3_valid),       // Val signal
      .yummy_out(xbar_buf_MOT_noc3_yummy)    // Yummy signal
);
```

Figure 3.22: Generation of valrdy to credit converter and credit to valrdy converter.

3.3.3.2 Add an AXI interface to the Top system

Because we intend to perform AXI communication between the Mesh_of_Tree ip and OpenPiton, and yet we do not have an AXI interface in the OpenPiton top model and the communication protocol used between OpenPiton components is exclusively NoC, we had to make some changes to the RTL code to support communication with our Mesh of Tree IP via AXI.

The first step, as shown in Figure 3.23, is to add additional interfaces to the chispet_impl model that will allow our device to communicate with ioxbar and vice versa. Since the MOT device only has val/rdy NOC interfaces, this means that it can only communicate with the ioxbar through the val_to_credit/credit_to_val converters, which pushes us to link up the interfaces of these converters to the new inputs and outputs of the chipset_imp model.

```
module chipset_impl(

`ifdef Mesh_Of_Tree // Only inckude these interfaces if we have the Mesh of Tree device
    output chipset_buf_MOT_noc2_data,
    output chipset_buf_MOT_noc2_valid,
    input chipset_MOT_buf_noc2_ready,

    input chipset_MOT_buf_noc3_data,
    input chipset_MOT_buf_noc3_valid,
    output chipset_buf_MOT_noc3_ready,
`endif
```

Conditional Compilation

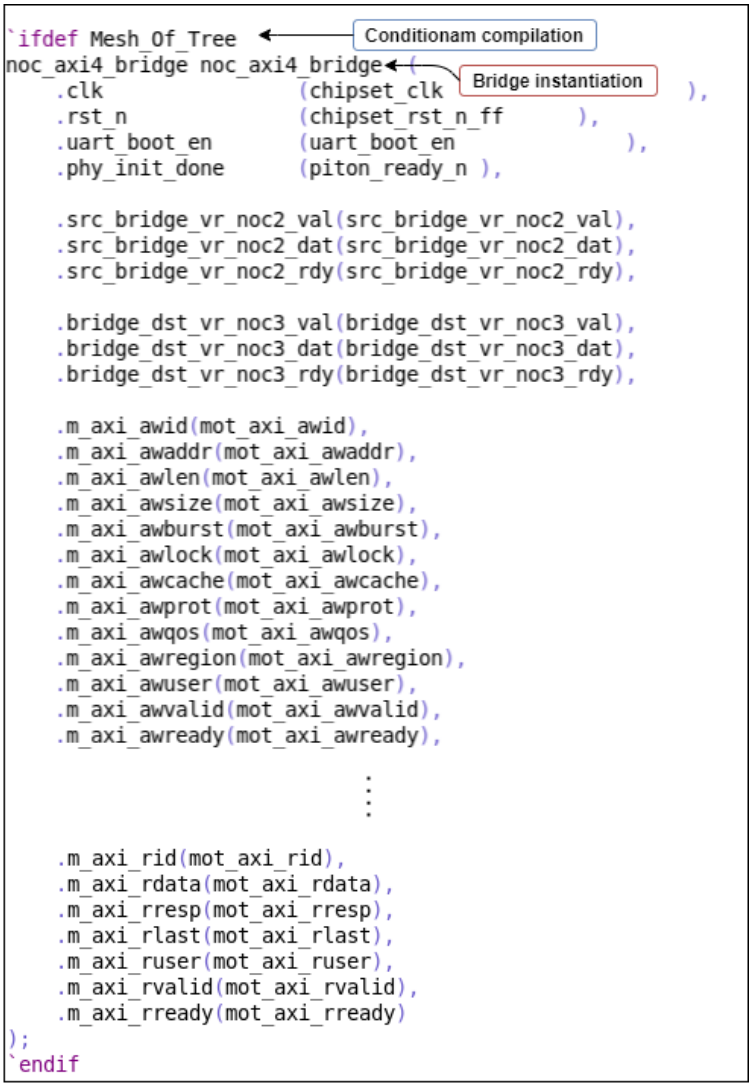
```
`ifdef Mesh_Of_Tree
assign chipset_buf_MOT_noc2_data = buf_MOT_noc2_data;
assign chipset_buf_MOT_noc2_valid = buf_MOT_noc2_valid;
assign MOT_buf_noc2_ready = chipset_MOT_buf_noc2_ready;

assign MOT_buf_noc3_data = chipset_MOT_buf_noc3_data;
assign MOT_buf_noc3_valid = chipset_MOT_buf_noc3_valid;
assign chipset_buf_MOT_noc3_ready = buf_MOT_noc3_ready;
`endif
```

Figure 3.23: Add interfaces to chipset_impl model.

But, before we proceed, it is important to note that the instantiation must be conditioned by a conditional compilation macro that appears immediately after the `ifdef compilation directive and checks for the definition of Mesh_of_Tree. If the Mesh_of_Tree macro is defined, the lines following the ifdef directive are included and compiled. Otherwise, the compiler will skip this section. So we will not need it until we have an AXI connection to our Mesh_of_Tree IP, at which point we will need to define the Mesh_of_Tree macro and pass it to Vivado to handle that part.

As shown in Figure 3.24, the second step is to instantiate a NOC/AXI bridge in the chipset which provides connectivity to a wide range of I/O devices by interfacing memory mapped I/O operations from the NoCs to AXI.




```
`ifdef Mesh_Of_Tree
noc_axi4_bridge noc_axi4_bridge(
    .clk                (chipset_clk                ),
    .rst_n              (chipset_rst_n_ff           ),
    .uart_boot_en       (uart_boot_en               ),
    .phy_init_done      (piton_ready_n ),

    .src_bridge_vr_noc2_val(src_bridge_vr_noc2_val),
    .src_bridge_vr_noc2_dat(src_bridge_vr_noc2_dat),
    .src_bridge_vr_noc2_rdy(src_bridge_vr_noc2_rdy),

    .bridge_dst_vr_noc3_val(bridge_dst_vr_noc3_val),
    .bridge_dst_vr_noc3_dat(bridge_dst_vr_noc3_dat),
    .bridge_dst_vr_noc3_rdy(bridge_dst_vr_noc3_rdy),

    .m_axi_awid(mot_axi_awid),
    .m_axi_awaddr(mot_axi_awaddr),
    .m_axi_awlen(mot_axi_awlen),
    .m_axi_awsize(mot_axi_awsize),
    .m_axi_awburst(mot_axi_awburst),
    .m_axi_awlock(mot_axi_awlock),
    .m_axi_awcache(mot_axi_awcache),
    .m_axi_awprot(mot_axi_awprot),
    .m_axi_awqos(mot_axi_awqos),
    .m_axi_awregion(mot_axi_awregion),
    .m_axi_awuser(mot_axi_awuser),
    .m_axi_awvalid(mot_axi_awvalid),
    .m_axi_awready(mot_axi_awready),

                                .
                                .
                                .

    .m_axi_rid(mot_axi_rid),
    .m_axi_rdata(mot_axi_rdata),
    .m_axi_rresp(mot_axi_rresp),
    .m_axi_rlast(mot_axi_rlast),
    .m_axi_ruser(mot_axi_ruser),
    .m_axi_rvalid(mot_axi_rvalid),
    .m_axi_rready(mot_axi_rready)
);
`endif
```

Figure 3.24: NOC/AXI bridge instantiation.

The next step is to add an AXI interface to our chipset module and connect them to the NOC/AXI bridge instance's AXI interface. In the other hand connect the NOC/AXI bridge instance's NOC interface to the MOT NOC interface of the chipset_impl instance (c.f Figure3.25 & Figure3.26).

```
module chipset(

`ifdef Mesh_Of_Tree // add AXI interface if we have Mesh_Of_Tree connection
    // AXI interface
    output wire [`AXI4_ID_WIDTH     -1:0]    mot_axi_awid,
    output wire [`AXI4_ADDR_WIDTH   -1:0]    mot_axi_awaddr,
    output wire [`AXI4_LEN_WIDTH    -1:0]    mot_axi_awlen,
    output wire [`AXI4_SIZE_WIDTH   -1:0]    mot_axi_awsize,
    output wire [`AXI4_BURST_WIDTH  -1:0]    mot_axi_awburst,
    output wire                              mot_axi_awlock,
    output wire [`AXI4_CACHE_WIDTH  -1:0]    mot_axi_awcache,
    output wire [`AXI4_PROT_WIDTH   -1:0]    mot_axi_awprot,
    output wire [`AXI4_QOS_WIDTH    -1:0]    mot_axi_awqos,
    output wire [`AXI4_REGION_WIDTH -1:0]    mot_axi_awregion,
    output wire [`AXI4_USER_WIDTH   -1:0]    mot_axi_awuser,
    output wire                              mot_axi_awvalid,
    input  wire                              mot_axi_awready,

    output wire  [`AXI4_ID_WIDTH     -1:0]    mot_axi_wid,
    output wire  [`AXI4_DATA_WIDTH   -1:0]    mot_axi_wdata,
    output wire  [`AXI4_STRB_WIDTH   -1:0]    mot_axi_wstrb,
    output wire                               mot_axi_wlast,
    output wire  [`AXI4_USER_WIDTH   -1:0]    mot_axi_wuser,
    output wire                               mot_axi_wvalid,
    input  wire                               mot_axi_wready,

    output wire  [`AXI4_ID_WIDTH     -1:0]    mot_axi_arid,
    output wire  [`AXI4_ADDR_WIDTH   -1:0]    mot_axi_araddr,
    output wire  [`AXI4_LEN_WIDTH    -1:0]    mot_axi_arlen,
    output wire  [`AXI4_SIZE_WIDTH   -1:0]    mot_axi_arsize,
    output wire  [`AXI4_BURST_WIDTH  -1:0]    mot_axi_arburst,
    output wire                               mot_axi_arlock,
    output wire  [`AXI4_CACHE_WIDTH  -1:0]    mot_axi_arcache,
    output wire  [`AXI4_PROT_WIDTH   -1:0]    mot_axi_arprot,
    output wire  [`AXI4_QOS_WIDTH    -1:0]    mot_axi_arqos,
    output wire  [`AXI4_REGION_WIDTH -1:0]    mot_axi_arregion,
    output wire  [`AXI4_USER_WIDTH   -1:0]    mot_axi_aruser,
    output wire                               mot_axi_arvalid,
    input  wire                               mot_axi_arready,

    input  wire  [`AXI4_ID_WIDTH     -1:0]    mot_axi_rid,
    input  wire  [`AXI4_DATA_WIDTH   -1:0]    mot_axi_rdata,
    input  wire  [`AXI4_RESP_WIDTH   -1:0]    mot_axi_rresp,
    input  wire                               mot_axi_rlast,
    input  wire  [`AXI4_USER_WIDTH   -1:0]    mot_axi_ruser,
    input  wire                               mot_axi_rvalid,
    output wire                               mot_axi_rready,

    input  wire  [`AXI4_ID_WIDTH     -1:0]    mot_axi_bid,
    input  wire  [`AXI4_RESP_WIDTH   -1:0]    mot_axi_bresp,
    input  wire  [`AXI4_USER_WIDTH   -1:0]    mot_axi_buser,
    input  wire                               mot_axi_bvalid,
    output wire                               mot_axi_bready,
`endif //Mesh Of Tree connection
```

Figure 3.25: Add AXI interface to chipset.

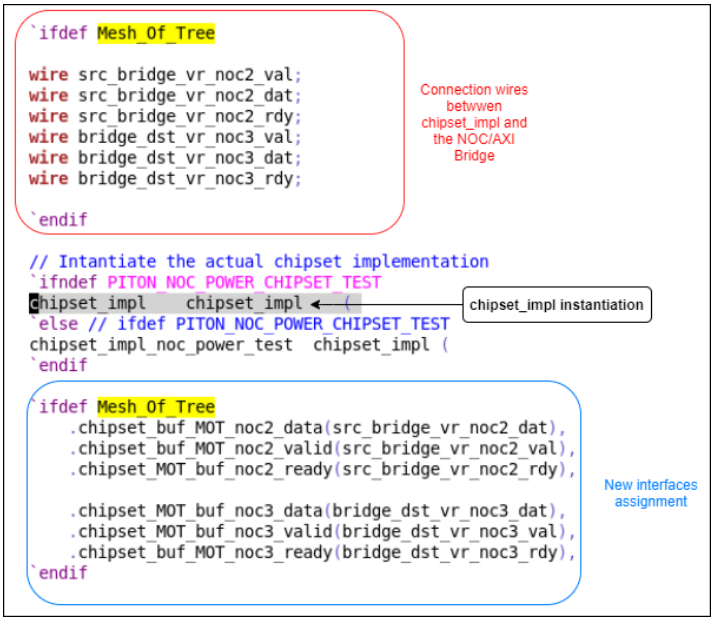


Figure 3.26: Add AXI interface to chipset.

The final step is to add an AXI interface to the Top system module and link them to interface AXI of the chipset instance.

```
module system(

`ifdef Mesh_Of_Tree // add AXI interface to Top system if we have Mesh_Of_Tree connection
    // AXI interface
    output wire [`AXI4_ID_WIDTH     -1:0]    mot_axi_awid,
    output wire [`AXI4_ADDR_WIDTH   -1:0]    mot_axi_awaddr,
    output wire [`AXI4_LEN_WIDTH    -1:0]    mot_axi_awlen,
    output wire [`AXI4_SIZE_WIDTH   -1:0]    mot_axi_awsize,
    output wire [`AXI4_BURST_WIDTH  -1:0]    mot_axi_awburst,
    output wire                              mot_axi_awlock,
    output wire [`AXI4_CACHE_WIDTH  -1:0]    mot_axi_awcache,
    output wire [`AXI4_PROT_WIDTH   -1:0]    mot_axi_awprot,
    output wire [`AXI4_QOS_WIDTH    -1:0]    mot_axi_awqos,
    output wire [`AXI4_REGION_WIDTH -1:0]    mot_axi_awregion,
    output wire [`AXI4_USER_WIDTH   -1:0]    mot_axi_awuser,
    output wire                              mot_axi_awvalid,
    input  wire                              mot_axi_awready,

    output wire  [`AXI4_ID_WIDTH     -1:0]   mot_axi_wid,
    output wire  [`AXI4_DATA_WIDTH   -1:0]   mot_axi_wdata,
    output wire  [`AXI4_STRB_WIDTH   -1:0]   mot_axi_wstrb,
    output wire                              mot_axi_wlast,
    output wire  [`AXI4_USER_WIDTH   -1:0]   mot_axi_wuser,
    output wire                              mot_axi_wvalid,
    input  wire                              mot_axi_wready,

    output wire  [`AXI4_ID_WIDTH     -1:0]   mot_axi_arid,
    output wire  [`AXI4_ADDR_WIDTH   -1:0]   mot_axi_araddr,
    output wire  [`AXI4_LEN_WIDTH    -1:0]   mot_axi_arlen,
    output wire  [`AXI4_SIZE_WIDTH   -1:0]   mot_axi_arsize,
    output wire  [`AXI4_BURST_WIDTH  -1:0]   mot_axi_arburst,
    output wire                              mot_axi_arlock,
    output wire  [`AXI4_CACHE_WIDTH  -1:0]   mot_axi_arcache,
    output wire  [`AXI4_PROT_WIDTH   -1:0]   mot_axi_arprot,
    output wire  [`AXI4_QOS_WIDTH    -1:0]   mot_axi_arqos,
    output wire  [`AXI4_REGION_WIDTH -1:0]   mot_axi_arregion,
    output wire  [`AXI4_USER_WIDTH   -1:0]   mot_axi_aruser,
    output wire                              mot_axi_arvalid,
    input  wire                              mot_axi_arready,

    input  wire  [`AXI4_ID_WIDTH     -1:0]   mot_axi_rid,
    input  wire  [`AXI4_DATA_WIDTH   -1:0]   mot_axi_rdata,
    input  wire  [`AXI4_RESP_WIDTH   -1:0]   mot_axi_rresp,
    input  wire                              mot_axi_rlast,
    input  wire  [`AXI4_USER_WIDTH   -1:0]   mot_axi_ruser,
    input  wire                              mot_axi_rvalid,
    output wire                              mot_axi_rready,

    input  wire  [`AXI4_ID_WIDTH     -1:0]   mot_axi_bid,
    input  wire  [`AXI4_RESP_WIDTH   -1:0]   mot_axi_bresp,
    input  wire  [`AXI4_USER_WIDTH   -1:0]   mot_axi_buser,
    input  wire                              mot_axi_bvalid,
    output wire                              mot_axi_bready,
`endif //Mesh_Of_Tree connection
```

```
// Piton chipset
chipset chipset(

`ifdef Mesh_Of_Tree // add AXI interface to chipset if we have Mesh_Of_Tree connection
    // AXI interface
    .mot_axi_awid(mot_axi_awid),
    .mot_axi_awaddr(mot_axi_awaddr),
    .mot_axi_awlen(mot_axi_awlen),
    .mot_axi_awsize(mot_axi_awsize),
    .mot_axi_awburst(mot_axi_awburst),
    .mot_axi_awlock(mot_axi_awlock),
    .mot_axi_awcache(mot_axi_awcache),
    .mot_axi_awprot(mot_axi_awprot),
    .mot_axi_awqos(mot_axi_awqos),
    .mot_axi_awregion(mot_axi_awregion),
    .mot_axi_awuser(mot_axi_awuser),
    .mot_axi_awvalid(mot_axi_awvalid),
    .mot_axi_awready(mot_axi_awready),

    .mot_axi_wid(mot_axi_wid),
    .mot_axi_wdata(mot_axi_wdata),
    .mot_axi_wstrb(mot_axi_wstrb),
    .mot_axi_wlast(mot_axi_wlast),
    .mot_axi_wuser(mot_axi_wuser),
    .mot_axi_wvalid(mot_axi_wvalid),
    .mot_axi_wready(mot_axi_wready),

    .mot_axi_arid(mot_axi_arid),
    .mot_axi_araddr(mot_axi_araddr),
    .mot_axi_arlen(mot_axi_arlen),
    .mot_axi_arsize(mot_axi_arsize),
    .mot_axi_arburst(mot_axi_arburst),
    .mot_axi_arlock(mot_axi_arlock),
    .mot_axi_arcache(mot_axi_arcache),
    .mot_axi_arprot(mot_axi_arprot),
    .mot_axi_arqos(mot_axi_arqos),
    .mot_axi_arregion(mot_axi_arregion),
    .mot_axi_aruser(mot_axi_aruser),
    .mot_axi_arvalid(mot_axi_arvalid),
    .mot_axi_arready(mot_axi_arready),

    .mot_axi_rid(mot_axi_rid),
    .mot_axi_rdata(mot_axi_rdata),
    .mot_axi_rresp(mot_axi_rresp),
    .mot_axi_rlast(mot_axi_rlast),
    .mot_axi_ruser(mot_axi_ruser),
    .mot_axi_rvalid(mot_axi_rvalid),
    .mot_axi_rready(mot_axi_rready),

    .mot_axi_bid(mot_axi_bid),
    .mot_axi_bresp(mot_axi_bresp),
    .mot_axi_buser(mot_axi_buser),
    .mot_axi_bvalid(mot_axi_bvalid),
    .mot_axi_bready(mot_axi_bready),
`endif //Mesh_Of_Tree connection
```

Figure 3.27: Add AXI interface to system and chipset instantiation.

3.3.3.3 Connect OpenPiton to the Mesh-of-Tree

After encapsulating the mesh of tree in an AXI slave interface and adding an AXI interface to OpenPiton, we need to connect them, so we will create a Top model in which we will instantiate the Top system model of OpenPiton and the AXI slave interface, then link them and generate an AXI communication. As shown in Figure 3.26 we have the two linked AXI instance.

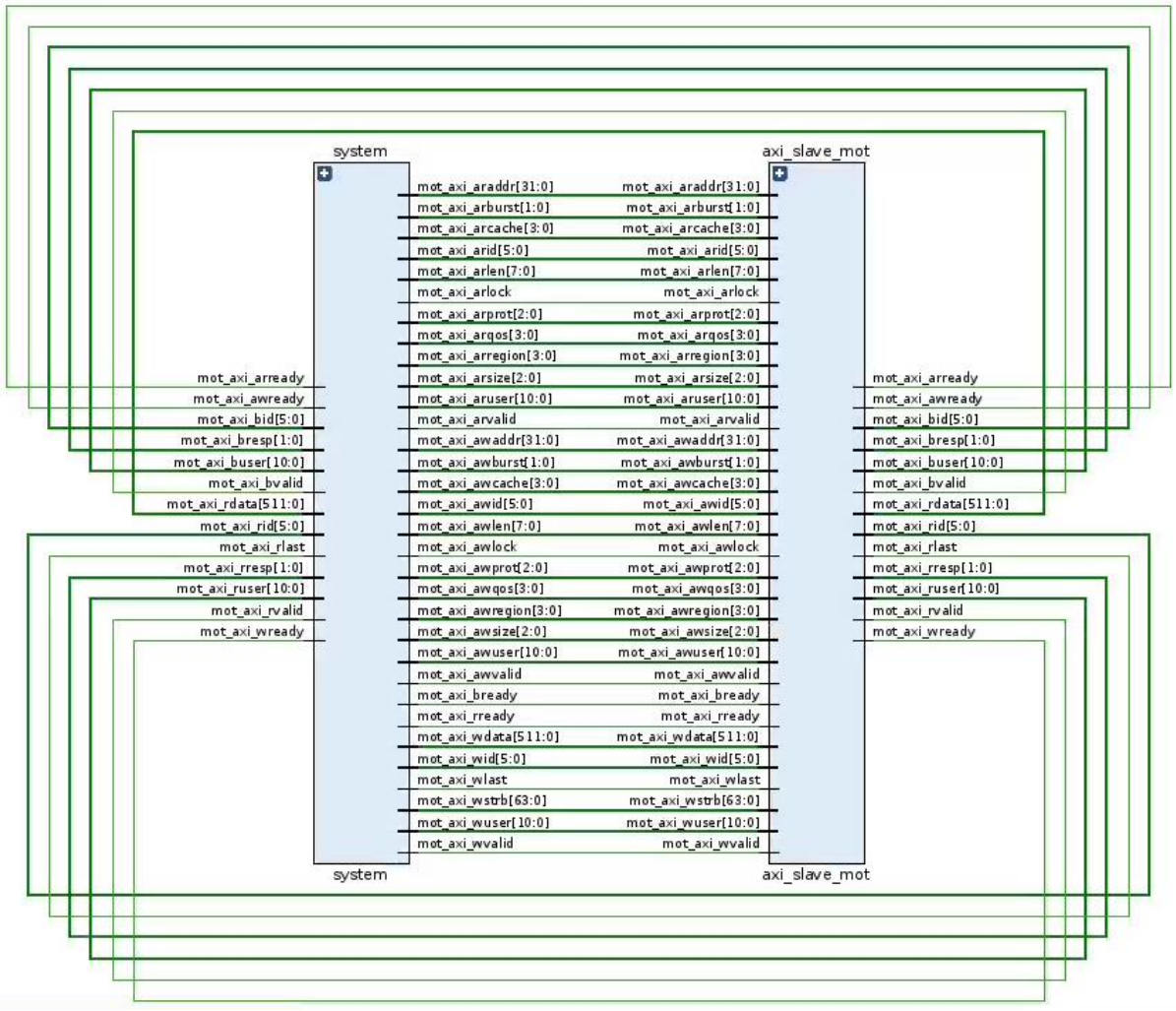


Figure 3.28: Top system and AXI slave interconnection.

3.3.3.4 Validation of the connection

In order to validate the connection between the OpenPiton and mesh-of-tree IP, we will wrote a C code that will be compiled using RISCV GNU tool chain, the binary generated by the compiler will be downloaded to the main memory of the processor which can be a BRAM or a DDR. In fact this C code will access the mesh-of-tree using pointer, for example we would assign a value to the conf input of the mesh-of-tree which as an address 0x00 , so we will pass to method the base address of the MOT device mentioned in the XML file and the data to write.

The figure 3.29, 3.30 and 3.31 are screen shots from the C code that is responsible of the configuration of the mesh-of-tree IP using OpenPiton.

```
write_reset(1);
write_conf(1);
// Download the Bitstream
for (j = 0; j < BitstreamSize; ++j) {
    write_adress(Address[j]);
    write_data(Data[j]);
}
write_conf(0);
write_clock(1);
for (j = 0; j < InpadFileSize; j=j+4) {
    // Assign the Inpads
    write_inpadTop(Inpads[j]);
    write_inpadRight(Inpads[j+1]);
    write_inpadBottom(Inpads[j+2]);
    write_inpadLeft(Inpads[j+3]);
    // Read from the Outpads
    int* outpadTop = read_outpadTop();
    Outpads[j]=outpadTop;
    int* outpadRight = read_outpadRight();
    Outpads[j+1]=outpadRight;
    int* outpadBottom = read_outpadBottom();
    Outpads[j+2]=outpadBottom;
    int* outpadLeft = read_outpadLeft();
    Outpads[j]=outpadLeft;
}
```

```
void write_conf(int conf){
    static int* MOT_base = (int*)(0xfff5100000);
    *MOT_base = conf;
    printf("write conf = %d \n", conf);
}
void write_clock(int* clock){
    static int* MOT_base = (int*)(0xfff5100001);
    *MOT_base = clock;
    printf("write clock = %d \n", clock);
}
void write_reset(int reset){
    static int* MOT_base = (int*)(0xfff5100002);
    *MOT_base = reset;
    printf("write reset = %d \n", reset);
}
```

```
void write_adress(int address[AddressSize]){
    static int* MOT_base = (int*)(0xfff5100003);
    unsigned i = 0;
    printf("write address = ");
    while(address[i] != NULL){
        *MOT_base = address[i];
        ++i;
        printf("%d", address[i]);
    }
    printf("\n");
}
void write_data(int data[DataSize]){
    static int* MOT_base = (int*)(0xfff5100004);
    unsigned i = 0;
    printf("write data = ");
    while(data[i] != NULL){
        *MOT_base = data[i];
        ++i;
        printf("%d", data[i]);
    }
    printf("\n");
}
```

Figure 3.29: Main, configuration and loading bitstream functions.

```
void write_inpadTop(int inpadTop[inpadSize]){
    static int* MOT_base = (int*)(0xfff5100005);
    unsigned i = 0;
    printf("write inpadsTop = ");
    while(inpadTop[i] != NULL){
        *MOT_base = inpadTop[i];
        ++i;
        printf("%d", inpadTop[i]);
    }
    printf("\n");
}
void write_inpadRight(int inpadRight[inpadSize]){
    static int* MOT_base = (int*)(0xfff5100006);
    unsigned i = 0;
    printf("write inpadsRight = ");
    while(inpadRight[i] != NULL){
        *MOT_base = inpadRight[i];
        ++i;
        printf("%d", inpadRight[i]);
    }
    printf("\n");
}
```

```
void write_inpadBottom(int inpadBottom[inpadSize]){
    static int* MOT_base = (int*)(0xfff5100007);
    unsigned i = 0;
    printf("write inpadsBottom = ");
    while(inpadBottom[i] != NULL){
        *MOT_base = inpadBottom[i];
        ++i;
        printf("%d", inpadBottom[i]);
    }
    printf("\n");
}
void write_inpadLeft(int inpadLeft[inpadSize]){
    static int* MOT_base = (int*)(0xfff5100008);
    unsigned i = 0;
    printf("write inpadsLeft = ");
    while(inpadLeft[i] != NULL){
        *MOT_base = inpadLeft[i];
        ++i;
        printf("%d", inpadLeft[i]);
    }
    printf("\n");
}
```

Figure 3.30: Inpads configuration functions.

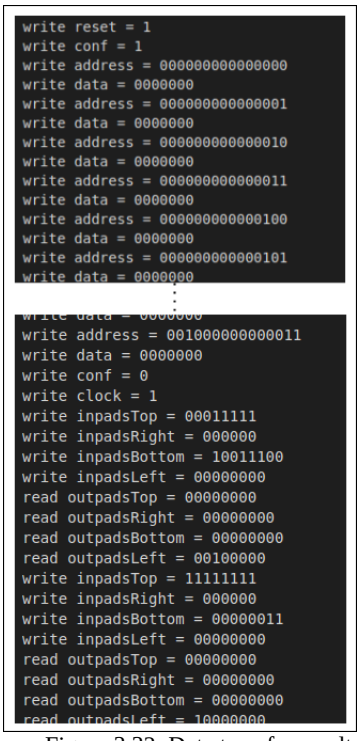


```
int* read_outpadTop(){
  static int *MOT_base = (int*)(0xfff5100000);
  static int outpadTop[outpadSize];
  unsigned i = 0;
  printf("read outpadsTop = ");
  while(MOT_base[i] != NULL){
    outpadTop[i]=MOT_base[i];
    ++i;
    printf("%d", MOT_base[i]);
  }
  printf("\n");
  return outpadTop;
}
int* read_outpadRight(){
  static int *MOT_base = (int*)(0xfff5100001);
  static int outpadRight[outpadSize];
  unsigned i = 0;
  printf("read outpadsRight = ");
  while(MOT_base[i] != NULL){
    outpadRight[i]=MOT_base[i];
    ++i;
    printf("%d", MOT_base[i]);
  }
  printf("\n");
  return outpadRight;
}
```

```
int* read_outpadBottom(){
  static int *MOT_base = (int*)(0xfff5100002);
  static int outpadBottom[outpadSize];
  unsigned i = 0;
  printf("read outpadsBottom = ");
  while(MOT_base[i] != NULL){
    outpadBottom[i]=MOT_base[i];
    ++i;
    printf("%d", MOT_base[i]);
  }
  printf("\n");
  return outpadLeft;
}
int* read_outpadLeft(){
  static int *MOT_base = (int*)(0xfff5100003);
  static int outpadLeft[outpadSize];
  unsigned i = 0;
  printf("read outpadsLeft = ");
  while(MOT_base[i] != NULL){
    outpadLeft[i]=MOT_base[i];
    ++i;
    printf("%d", MOT_base[i]);
  }
  printf("\n");
  return outpadLeft;
}
```

Figure 3.31: Outpads read functions.

The Figure 3.32 shows the result of compilation of the C code utilized to configure the FPGA IP. In fact, we began by initializing the reset input, then enabled the loader clock by activating the conf input. Then we parsed the bitstream file and loaded its contents into the IP FPGA via the AXI interface. We disabled the conf input and enabled the general mesh-of-tree FPGA clock after loading the bitstream. Then we instantiated the FPGA inputs pads and read its outputs pads states.



```
write reset = 1
write conf = 1
write address = 000000000000000
write data = 0000000
write address = 000000000000001
write data = 0000000
write address = 000000000000010
write data = 0000000
write address = 000000000000011
write data = 0000000
write address = 000000000000100
write data = 0000000
write address = 000000000000101
write data = 0000000
                ⋮
write address = 001000000000011
write data = 0000000
write conf = 0
write clock = 1
write inpadsTop = 00011111
write inpadsRight = 000000
write inpadsBottom = 10011100
write inpadsLeft = 00000000
read outpadsTop = 00000000
read outpadsRight = 00000000
read outpadsBottom = 00000000
read outpadsLeft = 00100000
write inpadsTop = 11111111
write inpadsRight = 000000
write inpadsBottom = 00000011
write inpadsLeft = 00000000
read outpadsTop = 00000000
read outpadsRight = 00000000
read outpadsBottom = 00000000
read outpadsLeft = 10000000
```

Figure 3.32: Data transfer result.

3.4 Conclusion

In this chapter, we validated our developed generator using a very specific procedure. Afterwards, we proposed the integration of the FPGA in the environment of a multiprocessor system on chip and precisely their interconnection with OpenPiton. Finally we confirmed and validated this interconnection.

Conclusion and perspectives

Throughout this project, we have undergone a deep research about existing FPGAs topologies and analyze their weaknesses and strengths. The tree topology presents a better area occupancy than the mesh but it is penalizing in terms of physical layout generation. As for the mesh, it enables us to scale up easily the FPGA layout. In the case of the mesh-of-tree topology, it unifies both tree and mesh strong points, since we benefit from the good interconnect of the tree and the scalability of the mesh.

After studying the topology, we have understand the existing FPGA IP generator that enables us to generate any FPGA architecture based on the mesh-of-tree topology. This existing IP generator was missing the essential element of the configuration which is the loader. Thus, we had to develop it in order to configure the implemented design. After completing the processing in the FPGA IP generator, we followed a specific debugging process to validate this IP generator. In fact, during this flow we discovered many errors and mismatches between our IP generator and the software flow synthesizer, which we fixed and corrected.

After validating the several components of the design, we have presented and validated the interface of interconnection within an MPSoC architecture.

Finally, we presented the interconnection of the FPGA IP to the OpenPiton manycore processor which will act as the master of our mesh-of-tree and configure it according to the attached bitstream file.

As future work, we aim to integrate the developed hardware design of the FPGA Mesh of Tree topology with the software flow and test different software designs on it. Moreover, we look forward to synthetizing our architecture and studying the generated netlist. As for integration in a multi-core architecture, we aim to integrate the eFPGA as a multi-slave component and get each slave responsible for a task to optimize the overall integration process.

References

[1] Mentor Graphics website. https://www.mentor.com/. Accessed Juin,03,2021.

[2] Umer Farooq, M.Hassan Aslan, Muhammad Usman. On the comparison of memristortransistor hybrid and transistor-only heterogeneous FPGAs, March 2018. Journal of King Saud University - Computer and Information Sciences 31 (2019) 514-27.

[3] Vinod Pangracious, Zied Marrakchi, and Habib Mehrez. Three-Dimensional Design Methodologies for Tree-based FPGA Architecture, 2015. Springer.

[4] Umer Farooq, Zied Marrakchi, and Habib Mehrez. Tree-based heterogeneous FPGA architectures: application specific exploration and optimization, 2012. Springer Science & Business Media.

[5] Zied Marrakchi, Hayder Mrabet, Umer Farooq, and Habib Mehrez. Fpga interconnect topologies exploration, 2009, 2009. International Journal of Reconfigurable Computing.

[6] Ashok B Mehta. Asic/soc functional design verification. 20 2018. Publ. Springer.

[7] Manhattandistance.https://www.sciencedirect.com/topics/mathematics/manhattandistance. Accessed March,15,2021.

[8] Zied Marrakchi, Hayder Mrabet, Christian Masson, and Habib Mehrez. Mesh of tree: unifying mesh and mfpga for better device performances. 2007. First International Symposium on Networks-on-Chip (NOCS'07), pages 243- 252, IEEE.

[9] Vinod Pangracious, Mulhim Al-Doori. Novel Three-Dimensional Embedded FPGA Technology and Achitecture, January 2017. ACM SIGARCH Computer Architecture News.

[10] V.Pangracious, Z.Marrakchi, E.Amouri, and H.Meherez. Analysis and Optimization of High Density Tree-based 3D Multilevel FPGA., March. ARC 2013, pp 197-209.

[11] G.Lemnieux, E.Lee, M.Tom, A.Yu. Directional and single-driver wires in FPGA interconnect, 2004. IEEE International Conference on Field-Programmable Technology, FPT-2004, pp. 41-48.

[12] W. Feng, S. Kaptanoglu. Designing efficient input interconnect blocks for LUT clusters using counting and entropy, 2007. International Symposium on Field Programmble Gate Array, (FPGA-2007), pp. 23-32.

[13] Julien Lamoureux and Steven J.E Wilton. On the interaction between PowerAware FPGA CAD Algorithms, 20 2003. Department of Electrical and Computer Engineering University of British Columbia Vancouver, B.C., Canada.

[14] DeHon. Rent's rule based switching requirements. 19 2001. In Proceedings of the 2001 international workshop on System-level interconnect prediction, pages 197–204.

[15] Muhammad Ali, Pedram Amini Rad, Diana Göhringer. RISC-V Based MPSoC Design Exploration for FPGAs: Area, Power and Performance. 2020. ARC 2020: 193-207.

[16] Microprocessor architecture. https://www.javatpoint.com/microprocessor-introduction. Accessed May,03,2021.

[17] Openpiton ASIC implmentaion. PowerPoint Presentation (hotchips.org). Accessed May,15,2021.

[18] Sheng Feng, Tong Liu,Jung-Cheun Lien. Inter-tile Buffer fystem for a field programmable gate array. 2006. Actel Corporation, Mountain View, CA, Appl. No.: 11/410,413.

[19] Ketan Padalia, Ryan Fung, Mark Bourgeault, Aaron Egier, and Jonathan Rose. Automatic Transistor and Physical Design of FPGA Tiles From An Architectural Specification, 2003. Department of ECE University of Toronto Ontario, Canada M5S 3G4.

[20] Umer Farooq, Zied Marrakchi, and Habib Mehrez. Tree-based heterogeneous FPGA architectures: application specific exploration and optimization, 2012. Springer Science & Business Media.

[21] Husain Parvez, Habib Mehrez. Application-Specific Mesh-based Heterogeneous FPGA Architectures., October 2010. Springer, New York, NY.

[22] idanagamachchi, S. D. Dewasurendra, R. G. Ragel M. Niranjan Member. Tile Optimization for Area in FPGA based Hardware Acceleration of Peptide Identification. 2011. International Conference on Industrial and Information Systems, ICIIS 2011, Aug. 16-19, 2011, Sri Lanka.

[23] Vinod Pangracious, Mulhim Al-Doori. Novel Three-Dimensional Embedded FPGA Technology and Achitecture, January 2017. ACM SIGARCH Computer Architecture News.

[24] Wentzlaff Parallel Research Group. OpenPiton Simulation Manual, March 2015, Princeton University.